BlueLagoon

ICPC Team Reference

GNU C++20 | 2026-10-18

本册源码与验证程序共用同一份算法文件。所有片段默认采用公共前置声明；图、区间下标按各段约定。校验短码忽略注释和非字符串空白，使用随附 hash.py 校验。测试范围与种子见 tests/report.json；本地回归通过不表示覆盖所有规模和数值退化。

目录

1 比赛环境与基础工具 . . . 2
1.1 公共前置声明 . . . 2
1.2 单测 / 多测入口 . . . 2
1.3 128 位读写与有符号整除 . . . 2
1.4 随机数与哈希 . . . 2
1.5 有序集合排名 / 第 k 小（PBDS） . . . 2
1.6 编译、调试、对拍与编辑器 . . . 2
2 数据结构与树 . . . 3
2.1 树状数组 / 前缀和 / 第 k 项 . . . 3
2.2 区间加 / 赋值 / 求和线段树 . . . 3
2.3 区间最值修改 / 加 / 求和（Segment Tree Beats） . . . 3
2.4 静态区间最值 / 幂等查询（ST 表） . . . 4
2.5 笛卡尔树 . . . 4
2.6 可持久化权值线段树 / 区间第 k 小 . . . 5
2.7 普通 / 带权并查集 . . . 5
2.8 可回滚并查集 . . . 5
2.9 序列分裂 / 合并 / 翻转（隐式 Treap） . . . 5
2.10 树链剖分 / LCA / 路径区间 . . . 6
2.11 虚树 . . . 6
2.12 点分治骨架 . . . 7
2.13 动态树路径异或（LCT） . . . 7
2.14 直线最小值查询（离散李超树） . . . 8
2.15 直线最大值查询（动态凸包） . . . 8
3 图论 . . . 8
3.1 强连通分量 . . . 8
3.2 布尔约束可行解（2-SAT） . . . 9
3.3 割点、桥、点双与边双 . . . 9
3.4 欧拉路径 . . . 10
3.5 非负权最短路 / 负环检测 . . . 10
3.6 有向最小生成树 / 最小树形图 . . . 10
3.7 支配树 . . . 11
3.8 必经与删边最短路选型 . . . 11
3.9 固定端点必经点 / 边 . . . 11
3.10 无向正权图单边删除最短路 . . . 12
3.11 无向图全局最小割 . . . 13
3.12 树同构 . . . 13
3.13 离线动态连通性 . . . 14
4 网络流与匹配 . . . 14
4.1 最大流 / 最小割（Dinic） . . . 14
4.2 有上下界可行流 / 最大流 / 最小流 . . . 14
4.3 最小费用流 / 最小费用最大流 . . . 15
4.4 无向图两点最小割树（Gomory-Hu） . . . 15
4.5 二分图最大匹配（Hopcroft-Karp） . . . 16
4.6 二分图最小点覆盖 / 最大独立集 . . . 16
4.7 二分图最小权匹配（KM / Hungarian） . . . 16
4.8 一般图最大匹配（带花树） . . . 16
4.9 二分图最优边染色 . . . 17
5 字符串 . . . 17
5.1 前缀函数 / KMP / Z 函数 . . . 17
5.2 双模字符串哈希 . . . 18
5.3 回文半径 / 最长回文（Manacher） . . . 18
5.4 多模式匹配（AC 自动机） . . . 18
5.5 后缀数组 + Height . . . 19
5.6 后缀 LCP 查询 . . . 19
5.7 后缀自动机 / 子串出现次数 . . . 19
5.8 回文树 / 不同回文与出现次数 . . . 19
5.9 Lyndon 分解 / 字符串最小表示 . . . 20
5.10 最长公共子序列长度 / 恢复（LCS） . . . 20
6 数论与组合数学 . . . 21
6.1 模运算 / 扩欧 / 扩展 CRT . . . 21
6.2 线性筛 / 质因子 / 欧拉与莫比乌斯函数 . . . 21
6.3 64 位判素 / 质因数分解 . . . 21
6.4 离散对数（BSGS / 扩展 BSGS） . . . 21
6.5 原根与模素数平方根 . . . 22
6.6 组合数取模（Lucas / 扩展 Lucas） . . . 22
6.7 整除取整求和（类欧几里得） . . . 22
6.8 杜教筛 phi / mu 前缀和 . . . 23
6.9 积性函数求和（Min_25 筛） . . . 23
6.10 整除分块与反演速查 . . . 23
7 多项式与线性代数 . . . 24
7.1 多项式乘法 / 卷积（NTT） . . . 24
7.2 任意模数卷积 . . . 24
7.3 形式幂级数求逆 / ln / exp / 幂 . . . 25
7.4 形式幂级数平方根 . . . 25
7.5 多项式除法 / 取模 / 多点求值 / 插值 . . . 26
7.6 连续点值插值 . . . 26
7.7 FWT / FMT 公式与正交补 . . . 26
7.8 位运算卷积 / 子集和 / 子集卷积 . . . 27
7.9 线性递推猜测 / 第 k 项（BM） . . . 27
7.10 实数 / 模素数高斯消元与行列式 . . . 28
7.11 异或线性基 . . . 28
7.12 生成树计数（Matrix-Tree） . . . 29
7.13 线性规划最优化（单纯形法） . . . 29
8 计算几何 . . . 30
8.1 二维点与向量 . . . 30
8.2 点在线段上 . . . 30
8.3 有向直线侧别 . . . 31
8.4 直线交点 . . . 31
8.5 线段交点 . . . 31
8.6 点到直线的投影 / 轴对称 . . . 31
8.7 点到直线距离 . . . 31
8.8 点到线段距离 . . . 31
8.9 点在多边形内 . . . 31
8.10 多边形有向面积 . . . 31
8.11 凸包 . . . 32
8.12 凸包直径 / 最远点对（旋转卡壳） . . . 32
8.13 最近点对 . . . 32
8.14 点在凸多边形内的查询 . . . 32
8.15 凸包极值点与直线交 . . . 32
8.16 两圆交点 . . . 33
8.17 圆与直线相交 . . . 33
8.18 圆的公切线 . . . 33
8.19 三角形外接圆 . . . 33
8.20 最小覆盖圆 . . . 33
8.21 圆与多边形交面积 . . . 34
8.22 三维向量 . . . 34
8.23 三维凸包 . . . 34
8.24 点到凸多边形的切线 . . . 35
8.25 凸多边形闵可夫斯基和 . . . 35
8.26 半平面交 . . . 35
8.27 几何类型与返回编码速查 . . . 36
9 优化、计数与公式速查 . . . 36
9.1 决策单调 DP 的分治优化 . . . 36
9.2 区间 DP 四边形不等式优化（Knuth） . . . 36
9.3 全单调矩阵行最小值（SMAWK） . . . 36
9.4 数值积分（自适应 Simpson） . . . 37
9.5 全覆盖循环序列（De Bruijn） . . . 37
9.6 SG、Nim 与常见博弈 . . . 37
9.7 组合与数学公式速查 . . . 37
10 常用补充 . . . 38
10.1 二维树状数组 / 矩形求和 . . . 38
10.2 莫队 / 静态区间查询 . . . 38
10.3 倍增祖先 / 路径第 k 点 . . . 38
10.4 模矩阵乘幂与实数 / 模矩阵求逆 . . . 39
10.5 异或方程组 / 解空间（GF(2)） . . . 39
10.6 最长递增子序列长度 / 恢复（LIS） . . . 40
10.7 整数分拆数 / 两类斯特林数 . . . 40
10.8 fread 快读快写 . . . 40
10.9 区间内分母最小的分数 . . . 41
10.10 多边形直线裁剪与重心 . . . 41
10.11 曼哈顿距离最小生成树候选边 . . . 41
11 进阶附录 . . . 42
11.1 有理函数第 k 项 / 快速递推（Bostan-Mori） . . . 42
11.2 一侧固定的在线卷积 . . . 42
11.3 形式幂级数复合 / 多项式平移 . . . 42
11.4 无权拟阵交 . . . 43
11.5 素数模离散 k 次根 . . . 43
11.6 正整数幂塔取模 . . . 43
11.7 有上下界最小费用环流（允许负环） . . . 44
11.8 K 短路 / K 最短游走（Eppstein） . . . 44
11.9 字符串极大周期段（runs） . . . 45
11.10 一般图 Δ+1 边染色 . . . 45
11.11 弦图识别与完美消除序 . . . 46
11.12 Nim 积 / 64 位 nimber 乘法 . . . 46
11.13 一般图最大权匹配（带权带花树） . . . 47
11.14 实系数多项式乘法 / 卷积（FFT） . . . 48
11.15 有向最小生成树 / 选边恢复（快速版） . . . 49
11.16 Delaunay 三角剖分 . . . 49
11.17 后缀树 / 最长公共子串（Ukkonen） . . . 51
11.18 离线动态最小生成树（边权修改） . . . 51
11.19 反对称矩阵 Pfaffian . . . 52
11.20 多项式系数递推猜测与延拓 . . . 52
11.21 合数模线性递推 / 第 k 项（Reeds-Sloane） . . . 53
12 性能工具与卡常技巧 . . . 55
12.1 性能工具选用与边界 . . . 55
12.2 动态模数快速取模（Barrett） . . . 56
12.3 奇模数连续模乘（Montgomery） . . . 56
12.4 64 除 32 位商余数（x86 divl） . . . 56
12.5 位扫描与等大小子集枚举 . . . 57
12.6 整数键开放寻址哈希表 . . . 57
12.7 固定容量内存池（Arena） . . . 57
12.8 数组异或（展开循环 / AVX2） . . . 57

1 比赛环境与基础工具

1.1 公共前置声明

功能：提供整型别名、标准库和模板共用的基础声明。
约定：所有片段默认依赖本段；GNU 扩展 __int128、PBDS 均需 GCC 兼容环境。
复杂度：$\mathcal{O}(1)$

BlueLagoon / CC0-1.0 / 独立编译 856401

```cpp
#include<bits/stdc++.h>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;
```

1.2 单测 / 多测入口

功能：提供 solve 与 main 的单测 / 多测程序入口。
约定：多测入口；单测改为直接 solve()。数组大小和每组清理范围按题目修改。
复杂度：按题目
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 独立编译 b5d84f

```cpp
const int N = 3e5 + 10;
int n, a[N];

void solve() {
    cin >> n;
    for (int i = 1; i <= n; i ++)
        cin >> a[i];
    // Reset the state used by this test case here.
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0),
        cout.tie(0);
    int tt;
    cin >> tt;
    while (tt --)
        solve();
    return 0;
}
```

1.3 128 位读写与有符号整除

功能：读写 128 位整数，并计算带符号整数的向下 / 向上整除。
约定：读入值必须在 i128 范围内。floor_div / ceil_div 要求 b 非零且商可表示。
复杂度：$\mathcal{O}$(位数)
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 947ab8

```cpp
istream &operator>>(istream &in, i128 &x) {
    string s;
    in >> s;
    x = 0;
    bool neg = !s.empty() && s[0] == '-';
    for (int i = neg || (!s.empty() && s[0] == '+');
         i < (int)s.size(); i ++)
        x = x * 10 - (s[i] - '0');
    if (!neg) x = -x;
    return in;
}
ostream &operator<<(ostream &out, i128 x) {
    if (!x) return out << '0';
    u128 y = x < 0 ? -u128(x) : u128(x);
    string s;
    while (y)
        s += char('0' + y % 10), y /= 10;
    if (x < 0) out << '-';
    reverse(s.begin(), s.end());
    return out << s;
}
i64 floor_div(i64 a, i64 b) {
    assert(b && !(a == LLONG_MIN && b == -1));
    i64 q = a / b, r = a % b;
    return q - (r && ((r < 0) != (b < 0)));
}
i64 ceil_div(i64 a, i64 b) {
    assert(b && !(a == LLONG_MIN && b == -1));
    i64 q = a / b, r = a % b;
    return q + (r && ((r < 0) == (b < 0)));
}
```

1.4 随机数与哈希

功能：生成随机数，为哈希表的整数键提供混洗哈希。
约定：SplitMix64 混洗；哈希碰撞仍有可能。对拍使用固定随机种子以复现。
复杂度：平均 $\mathcal{O}(1)$ 哈希
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 e9f202

```cpp
mt19937_64 rng(chrono::steady_clock::now()
                   .time_since_epoch()
                   .count());
i64 rnd(i64 l, i64 r) {
    return uniform_int_distribution<i64>(l, r)(rng);
}
struct Hash {
    static u64 mix(u64 x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(u64 x) const {
        static const u64 seed = rng();
        return mix(x + seed);
    }
};
// unordered_map<i64, int, Hash> mp;
```

1.5 有序集合排名 / 第 k 小（PBDS）

功能：维护有序集合，查询元素排名和第 k 小元素。
约定：比较器必须严格弱序，不能用 less_equal 模拟重复值；不存在的第 k 项返回 end()。
复杂度：$\mathcal{O}(\log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 2b903c

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using OrderedSet =
    tree<T, null_type, less<T>, rb_tree_tag,
         tree_order_statistics_node_update>;
// s.order_of_key(x): number of keys < x.
// s.find_by_order(k): iterator to 0-based kth key.
// Duplicates: store pair<value, unique_id>.
```

1.6 编译、调试、对拍与编辑器

提交编译：g++ main.cpp -std=c++20 -O2 -Wall -Wextra -o main

本地调试：g++ main.cpp -std=c++20 -O1 -g -D_GLIBCXX_DEBUG -fsanitize=address,undefined -o main

调试选项依考场安装情况使用；最终提交移除 DEBUG 和重定向。

Linux/Bash 对拍：for ((s=1;;s++)); do ./gen "$s" > in; ./main < in > out; ./brute < in > ans; diff -w out ans || break; done

生成器接受种子 s；保存失败输入，先排除暴力程序本身错误。

Vim：set nu ts=4 sw=4 noet autoindent | syntax on

Vim 编译：:w | !g++ % -std=c++20 -O2 -Wall -Wextra -o %:r

Vim 运行：:!./%:r < in

__builtin_ctzll(x)、clzll(x) 要求 x != 0；移位量必须小于位宽。

枚举子集：for (int s = m; s; s = (s - 1) & m)；零子集单独处理。

枚举全集所有子集共 $\mathcal{O}(3^n)$；SOS DP 为 $\mathcal{O}(n2^n)$。

检查乘法是否先转 i64 / i128；INF 加边权前确认不会溢出。

浮点输出设精度；比较误差按量纲和规模选择；几何中优先整数谓词。

DFS 深链注意栈；大数组注意内存；比赛开始确认编译器、时间限制和可用调试工具。

2 数据结构与树

2.1 树状数组 / 前缀和 / 第 k 项

功能：支持单点加、前缀和，以及按累计权值查找第 k 项。
约定：1-based；add 要求 1 <= x <= n。kth 要求每个位置的权值非负、1 <= k <= sum(n)。
复杂度：$\mathcal{O}(\log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 831032

```
struct Fenwick {
    int n;
    vector<i64> tr;
    Fenwick(int n) : n(n), tr(n + 1) {}
    void add(int x, i64 v) {
        for (; x <= n; x += x & -x)
            tr[x] += v;
    }
    i64 sum(int x) const {
        i64 ans = 0;
        for (; x; x -= x & -x)
            ans += tr[x];
        return ans;
    }
    int kth(i64 k) const {
        int x = 0;
        for (int d = 1 << __lg(max(1, n)); d; d >>= 1)
            if (x + d <= n && tr[x + d] < k)
                k -= tr[x += d];
        return x + 1;
    }
};
```

2.2 区间加 / 赋值 / 求和线段树

功能：维护区间加、区间赋值与区间和。
约定：初始全零，闭区间 1..n；从根参数 (1,1,n) 调用，要求 1 <= x <= y <= n。普通线段树可删懒标记。
复杂度：$\mathcal{O}(\log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 c62139

```
struct SegTree {
    struct Node {
        i64 sum = 0, add = 0, set = 0;
        bool has = false;
    };
    int n;
    vector<Node> tr;
    SegTree(int n) : n(n), tr(4 * n + 4) {}
    void apply(int p, int l, int r, i64 v, bool set) {
        if (set) {
            tr[p].sum = (r - l + 1) * v;
            tr[p].set = v;
            tr[p].has = true;
            tr[p].add = 0;
        } else {
            tr[p].sum += (r - l + 1) * v;
            tr[p].add += v;
        }
    }
    void push(int p, int l, int r) {
        int m = (l + r) / 2;
        if (tr[p].has) {
            apply(p * 2, l, m, tr[p].set, true);
            apply(p * 2 + 1, m + 1, r, tr[p].set, true);
            tr[p].has = false;
        }
        if (tr[p].add) {
            apply(p * 2, l, m, tr[p].add, false);
            apply(p * 2 + 1, m + 1, r, tr[p].add,
                  false);
            tr[p].add = 0;
        }
    }
    void update(int p, int l, int r, int x, int y,
                i64 v, bool set = false) {
        if (x <= l && r <= y)
            return apply(p, l, r, v, set);
        push(p, l, r);
        int m = (l + r) / 2;
        if (x <= m) update(p * 2, l, m, x, y, v, set);
        if (y > m)
            update(p * 2 + 1, m + 1, r, x, y, v, set);
        tr[p].sum = tr[p * 2].sum + tr[p * 2 + 1].sum;
    }
    i64 query(int p, int l, int r, int x, int y) {
        if (x <= l && r <= y) return tr[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        i64 ans = 0;
        if (x <= m) ans += query(p * 2, l, m, x, y);
        if (y > m)
            ans += query(p * 2 + 1, m + 1, r, x, y);
        return ans;
    }
};
```

2.3 区间最值修改 / 加 / 求和（Segment Tree Beats）

功能：维护区间 chmin、chmax、加，以及区间和、最小值、最大值。
约定：从 vector<i64> 建树；0-based 半开区间 [l,r)。chmin 令 a[i]=min(a[i],v)，chmax 同理。实际及内部临时极值、修改参数严格在 (-INF, INF) 内；中间区间和、累计加法标记须能装入 i64。空区间修改无效，查询返回 {0,INF,-INF}。完整版本为摊还界，不保证单次最坏 O(log n)。
调用：SegBeats t(vector<i64>{3,1,4}); t.chmin(0,3,2); t.add(1,3,5); auto q=t.query(0,3);
复杂度：建树 $\mathcal{O}(n)$，q 次操作总计 $\mathcal{O}((n+q)\log^2 n)$ 摊还，空间 $\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 4c23f3

```
struct SegBeats {
    static constexpr i64 INF = LLONG_MAX / 4;
    struct Node {
        i64 sum = 0, add = 0;
        // 最大、严格次大；最小、严格次小。无次极值用哨兵。
        i64 mx = -INF, sx = -INF, mn = INF, sn = INF;
        int cx = 0, cn = 0,
            len = 0; // 两种极值个数、长度。
    };
    struct Result {
        i64 sum = 0, mn = INF, mx = -INF;
    };
    int n;
    vector<Node> tr;
    explicit SegBeats(const vector<i64> &a)
        : n(a.size()), tr(4 * n + 4) {
        if (n) build(1, 0, n, a);
    }
    void pull(int p) {
        auto &a = tr[p * 2], &b = tr[p * 2 + 1];
        Node c;
        c.len = a.len + b.len;
        c.sum = i128(a.sum) + b.sum;
        c.mx = max(a.mx, b.mx), c.mn = min(a.mn, b.mn);
        c.cx = (a.mx == c.mx ? a.cx : 0) +
               (b.mx == c.mx ? b.cx : 0);
        c.cn = (a.mn == c.mn ? a.cn : 0) +
               (b.mn == c.mn ? b.cn : 0);
        c.sx = max(a.mx == c.mx ? a.sx : a.mx,
                   b.mx == c.mx ? b.sx : b.mx);
        c.sn = min(a.mn == c.mn ? a.sn : a.mn,
                   b.mn == c.mn ? b.sn : b.mn);
```

```
        tr[p] = c;
    }
    void build(int p, int l, int r,
               const vector<i64> &a) {
        if (r - l == 1) {
            assert(-INF < a[l] && a[l] < INF);
            auto &x = tr[p];
            x.sum = x.mx = x.mn = a[l];
            x.cx = x.cn = x.len = 1;
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a),
            build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void apply_add(int p, i64 v) {
        auto &x = tr[p];
        x.sum = i128(x.sum) + i128(v) * x.len;
        x.mx += v, x.mn += v, x.add += v;
        if (x.sx != -INF) x.sx += v;
        if (x.sn != INF) x.sn += v;
    }
    // 内部捷径：仅最大值受影响，必须 sx<v<mx。
    void apply_min(int p, i64 v) {
        auto &x = tr[p];
        if (x.mx <= v) return;
        assert(x.sx < v);
        x.sum = i128(x.sum) + i128(v - x.mx) * x.cx;
        if (x.mn == x.mx)
            x.mn = v;
        else if (x.sn == x.mx)
            x.sn = v;
        x.mx = v;
    }
    // 内部捷径：仅最小值受影响，必须 mn<v<sn。
    void apply_max(int p, i64 v) {
        auto &x = tr[p];
        if (x.mn >= v) return;
        assert(v < x.sn);
        x.sum = i128(x.sum) + i128(v - x.mn) * x.cn;
        if (x.mx == x.mn)
            x.mx = v;
        else if (x.sx == x.mn)
            x.sx = v;
        x.mn = v;
    }
    void push(int p) {
        // 先下传加法，再按父结点现有极值限制孩子。
        for (int c : { p * 2, p * 2 + 1 }) {
            apply_add(c, tr[p].add);
            apply_min(c, tr[p].mx);
            apply_max(c, tr[p].mn);
        }
        tr[p].add = 0;
    }
    // op=0 加，op=1 chmin，op=2
    // chmax；其余函数为公开入口。
    void update(int p, int l, int r, int ql, int qr,
                i64 v, int op) {
        if (qr <= l || r <= ql) return;
        auto &x = tr[p];
        if ((op == 1 && x.mx <= v) ||
            (op == 2 && x.mn >= v))
            return;
        if (ql <= l && r <= qr) {
            if (op == 0) return apply_add(p, v);
            if (op == 1 && x.sx < v)
                return apply_min(p, v);
            if (op == 2 && v < x.sn)
                return apply_max(p, v);
        }
        push(p);
        int m = (l + r) / 2;
        update(p * 2, l, m, ql, qr, v, op);
        update(p * 2 + 1, m, r, ql, qr, v, op);
        pull(p);
    }
    void change(int l, int r, i64 v, int op) {
        assert(0 <= l && l <= r && r <= n);
        assert(-INF < v && v < INF);
        if (l < r) update(1, 0, n, l, r, v, op);
    }
    void add(int l, int r, i64 v) {
        change(l, r, v, 0);
    }
    void chmin(int l, int r, i64 v) {
        change(l, r, v, 1);
    }
    void chmax(int l, int r, i64 v) {
        change(l, r, v, 2);
    }
    Result get(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return {};
        if (ql <= l && r <= qr)
            return { tr[p].sum, tr[p].mn, tr[p].mx };
        push(p);
        int m = (l + r) / 2;
        auto a = get(p * 2, l, m, ql, qr);
        auto b = get(p * 2 + 1, m, r, ql, qr);
        return { i64(i128(a.sum) + b.sum),
                 min(a.mn, b.mn), max(a.mx, b.mx) };
    }
    Result query(int l, int r) {
        assert(0 <= l && l <= r && r <= n);
        return l == r ? Result{} : get(1, 0, n, l, r);
    }
};
```

2.4 静态区间最值 / 幂等查询（ST 表）

功能：预处理静态数组，常数时间查询区间 min、max 等幂等运算。
约定：0-based 闭区间；仅适合幂等运算，如 min/max/gcd，不能直接改成 sum。
复杂度：预处理 $\mathcal{O}(n \log n)$，查询 $\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 2e8d76

```
struct ST {
    vector<vector<int>> st;
    ST(const vector<int> &a) {
        int n = a.size();
        assert(n);
        st.push_back(a);
        for (int k = 1; (1 << k) <= n; k ++) {
            st.emplace_back(n - (1 << k) + 1);
            for (int i = 0; i + (1 << k) <= n; i ++)
                st[k][i] =
                    min(st[k - 1][i],
                        st[k - 1][i + (1 << (k - 1))]);
        }
    }
    int query(int l, int r) const {
        int k = __lg(r - l + 1);
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

2.5 笛卡尔树

功能：将数组构成下标中序、权值满足最小堆性质的二叉树。
约定：0-based；最小堆、下标中序。相等值取左侧为祖先；空输入根为 -1。
复杂度：$\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 01487c

```
pair<int, vector<array<int, 2>>>
cartesian(const vector<int> &a) {
    int n = a.size();
    vector<array<int, 2>> ch(n, { -1, -1 });
    vector<int> st;
    for (int i = 0; i < n; i ++) {
        int last = -1;
```

```
        while (!st.empty() && a[st.back()] > a[i]) {
            last = st.back();
            st.pop_back();
        }
        if (!st.empty()) ch[st.back()][1] = i;
        ch[i][0] = last;
        st.push_back(i);
    }
    return { st.empty() ? -1 : st[0], ch };
}
```

2.6 可持久化权值线段树 / 区间第 k 小

功能：维护各版本权值频次，通过前缀版本之差查询区间第 k 小。
约定：权值离散化到闭区间 1..M；0 为全零根。前缀根差 kth(a,b) 要求差值计数非负且 k 有效。
复杂度：每次 $\mathcal{O}(\log M)$，空间 $\mathcal{O}(\text{更新数}\log M)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 0ecbd2

```
struct Persistent {
    struct Node {
        int l = 0, r = 0, cnt = 0;
    };
    vector<Node> tr = { Node() };
    int add(int p, int l, int r, int x, int v = 1) {
        int q = tr.size();
        tr.push_back(tr[p]);
        tr[q].cnt += v;
        if (l != r) {
            int m = (l + r) / 2;
            if (x <= m)
                tr[q].l = add(tr[p].l, l, m, x, v);
            else
                tr[q].r = add(tr[p].r, m + 1, r, x, v);
        }
        return q;
    }
    int kth(int a, int b, int l, int r, int k) {
        if (l == r) return l;
        int cnt = tr[tr[b].l].cnt - tr[tr[a].l].cnt;
        int m = (l + r) / 2;
        if (k <= cnt)
            return kth(tr[a].l, tr[b].l, l, m, k);
        return kth(tr[a].r, tr[b].r, m + 1, r, k - cnt);
    }
};
```

2.7 普通 / 带权并查集

功能：维护集合连通关系，以及合并时给出的两点数值差约束。
约定：0-based；d[x] 表示 value[x]-value[parent[x]]；find 后为到根差值。join 返回约束是否一致，普通合并用 w=0。
复杂度：均摊 $\mathcal{O}(\alpha(n))$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 151a21

```
struct DSU {
    vector<int> p, sz;
    vector<i64> d;
    DSU(int n) : p(n), sz(n, 1), d(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        if (p[x] == x) return x;
        int y = p[x];
        p[x] = find(y);
        d[x] += d[y];
        return p[x];
    }
    // Enforce value[y] - value[x] = w.
    bool join(int x, int y, i64 w = 0) {
        int a = find(x), b = find(y);
        w += d[x] - d[y];
        if (a == b) return w == 0;
        if (sz[a] < sz[b]) swap(a, b), w = -w;
        p[b] = a;
        d[b] = w;
        sz[a] += sz[b];
        return true;
    }
};
```

2.8 可回滚并查集

功能：维护连通关系，并回退到此前保存的合并状态。
约定：0-based；保存 time()，用 rollback(t) 回退。find 不做路径压缩；join 返回是否合并。
复杂度：$\mathcal{O}(\log n)$ 查找，$\mathcal{O}(\text{回退数})$ 回退
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 033892

```
struct RollbackUF {
    vector<int> e;
    vector<pair<int, int>> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) {
        return e[x] < 0 ? x : find(e[x]);
    }
    int time() { return (int)(st).size(); }
    void rollback(int t) {
        for (int i = time(); i -- > t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({ a, e[a] });
        st.push_back({ b, e[b] });
        e[a] += e[b];
        e[b] = a;
        return true;
    }
};
```

2.9 序列分裂 / 合并 / 翻转（隐式 Treap）

功能：按位置分裂和合并序列，维护区间翻转与区间和。
约定：0 为空节点；split(root,k) 分出前 k 个元素。区间 [l,r) 用两次 split，再 flip / 读 sum，最后 merge。
复杂度：期望 $\mathcal{O}(\log n)$ 每次
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 96a9bd

```
struct Treap {
    struct Node {
        int l = 0, r = 0, sz = 1;
        u64 pri;
        i64 val, sum;
        bool rev = false;
    };
    vector<Node> tr = { { 0, 0, 0, 0, 0, 0, false } };
    mt19937_64 rng{ 712367 };
    int node(i64 v) {
        tr.push_back({ 0, 0, 1, rng(), v, v, false });
        return tr.size() - 1;
    }
    void flip(int p) {
        if (p) swap(tr[p].l, tr[p].r), tr[p].rev ^= 1;
    }
    void push(int p) {
        if (tr[p].rev) {
            flip(tr[p].l);
            flip(tr[p].r);
            tr[p].rev = false;
        }
    }
    void pull(int p) {
        auto &a = tr[p];
        a.sz = tr[a.l].sz + tr[a.r].sz + 1;
```

```
        a.sum = tr[a.l].sum + tr[a.r].sum + a.val;
    }
    pair<int, int> split(int p, int k) {
        if (!p) return { 0, 0 };
        push(p);
        if (tr[tr[p].l].sz >= k) {
            auto [a, b] = split(tr[p].l, k);
            tr[p].l = b;
            pull(p);
            return { a, p };
        }
        auto [a, b] =
            split(tr[p].r, k - tr[tr[p].l].sz - 1);
        tr[p].r = a;
        pull(p);
        return { p, b };
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (tr[a].pri < tr[b].pri) {
            push(a);
            tr[a].r = merge(tr[a].r, b);
            pull(a);
            return a;
        }
        push(b);
        tr[b].l = merge(a, tr[b].l);
        pull(b);
        return b;
    }
};
```

2.10 树链剖分 / LCA / 路径区间

功能：将树上路径和子树转为数组区间，求 LCA，支持按路径方向聚合。
约定：0-based，无向树；子树为 [in[u],out[u])。path 回调半开区间，适合可交换运算；edge=true 表示边权存在较深端点。深链改迭代 DFS。path_ordered(u,v,edge) 按 u→v 顺序返回 {l,r,rev}；rev=true 时反向读取 seq[l..r)，适用于非交换聚合；边模式排除 LCA。例：拼接各段字符得到整条有向路径字符串。
复杂度：预处理 $\mathcal{O}(n)$，LCA $\mathcal{O}(\log n)$，路径 $\mathcal{O}(\log n)$ 段
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 独立编译 67366d

```
struct HLD {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> p, dep, sz, son, top, in, out, seq;
    HLD(int n)
        : n(n), g(n), p(n, -1), dep(n), sz(n),
          son(n, -1), top(n), in(n), out(n), seq(n) {}
    void dfs(int u) {
        sz[u] = 1;
        for (int v : g[u])
            if (v != p[u]) {
                p[v] = u;
                dep[v] = dep[u] + 1;
                dfs(v);
                sz[u] += sz[v];
                if (son[u] == -1 || sz[v] > sz[son[u]])
                    son[u] = v;
            }
    }
    void build(int u, int t) {
        top[u] = t;
        in[u] = timer;
        seq[timer ++] = u;
        if (son[u] != -1) build(son[u], t);
        for (int v : g[u])
            if (v != p[u] && v != son[u]) build(v, v);
        out[u] = timer;
    }
    void init(int r = 0) {
        timer = 0;
        p[r] = -1;
        dep[r] = 0;
        fill(son.begin(), son.end(), -1);
        dfs(r);
        build(r, r);
    }
    int lca(int u, int v) const {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = p[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
    struct Segment {
        int l, r;
        bool rev;
    };
    vector<Segment>
    path_ordered(int u, int v,
                 bool edge = false) const {
        vector<Segment> a, b;
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]]) {
                a.push_back(
                    { in[top[u]], in[u] + 1, true });
                u = p[top[u]];
            } else {
                b.push_back(
                    { in[top[v]], in[v] + 1, false });
                v = p[top[v]];
            }
        }
        if (dep[u] >= dep[v]) {
            if (in[v] + edge < in[u] + 1)
                a.push_back(
                    { in[v] + edge, in[u] + 1, true });
        } else if (in[u] + edge < in[v] + 1)
            b.push_back(
                { in[u] + edge, in[v] + 1, false });
        reverse(b.begin(), b.end());
        a.insert(a.end(), b.begin(), b.end());
        return a;
    }
    template <class F>
    void path(int u, int v, F f,
              bool edge = false) const {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            f(in[top[u]], in[u] + 1);
            u = p[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (in[u] + edge <= in[v])
            f(in[u] + edge, in[v] + 1);
    }
};
```

2.11 虚树

功能：压缩树上关键点及必要 LCA，保留关键点之间的祖先关系。
约定：输入关键点；输出祖先 → 后代边，补入相邻 LCA。单点时无边，节点自身由输入保存；边长为 dep[v]-dep[u]。
复杂度：$\mathcal{O}(k \log k + k \log n)$
依赖：§1.1, §2.10（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 205c64

```
vector<pair<int, int>> virtual_tree(vector<int> a,
                                    const HLD &h) {
    if (a.empty()) return {};
    auto cmp = [&](int x, int y) {
        return h.in[x] < h.in[y];
    };
    sort(a.begin(), a.end(), cmp);
    int k = a.size();
    for (int i = 1; i < k; i ++)
        a.push_back(h.lca(a[i - 1], a[i]));
    sort(a.begin(), a.end(), cmp);
    a.erase(unique(a.begin(), a.end()), a.end());
    vector<int> st;
```

```
    vector<pair<int, int>> ed;
    for (int u : a) {
        while (!st.empty() &&
               h.out[st.back()] <= h.in[u])
            st.pop_back();
        if (!st.empty()) ed.push_back({ st.back(), u });
        st.push_back(u);
    }
    return ed;
}
```

2.12 点分治骨架

功能：按重心拆分树，提供逐子树统计路径贡献的回调骨架。
约定：无向树，0-based。work 统计经过重心的贡献：逐子树查询已有信息，再加入当前子树；仅访问!dead 节点。深链注意递归栈。
复杂度：分治框架 $\mathcal{O}(n \log n)$，另加 work 开销
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 5e1e5c

```
struct Centroid {
    vector<vector<int>> g;
    vector<int> sz;
    vector<bool> dead;
    Centroid(int n) : g(n), sz(n), dead(n) {}
    int size(int u, int p) {
        sz[u] = 1;
        for (int v : g[u])
            if (v != p && !dead[v]) sz[u] += size(v, u);
        return sz[u];
    }
    int center(int u, int p, int tot) {
        for (int v : g[u])
            if (v != p && !dead[v])
                if (sz[v] > tot / 2)
                    return center(v, u, tot);
        return u;
    }
    template <class F> void solve(int u, F &work) {
        u = center(u, -1, size(u, -1));
        work(u, g, dead);
        dead[u] = true;
        for (int v : g[u])
            if (!dead[v]) solve(v, work);
    }
};
```

2.13 动态树路径异或（LCT）

功能：动态连边、断边、修改点权，并查询森林中两点路径异或。
约定：1-based，0 是空节点。query 要求两点连通；初始点权用 set 设置。支持森林，link 防止成环，cut 仅删除已有直接边。
复杂度：均摊 $\mathcal{O}(\log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 4c2aae

```
struct LCT {
    struct Node {
        int ch[2] = {}, p = 0;
        bool rev = false;
        i64 val = 0, sum = 0;
    };
    vector<Node> tr;
    LCT(int n) : tr(n + 1) {}
    bool root(int x) {
        int p = tr[x].p;
        return !p ||
               (tr[p].ch[0] != x && tr[p].ch[1] != x);
    }
    void pull(int x) {
        tr[x].sum = tr[tr[x].ch[0]].sum ^ tr[x].val ^
                    tr[tr[x].ch[1]].sum;
    }
    void flip(int x) {
        if (x)
            swap(tr[x].ch[0], tr[x].ch[1]),
                tr[x].rev ^= 1;
    }
    void push(int x) {
        if (tr[x].rev) {
            flip(tr[x].ch[0]);
            flip(tr[x].ch[1]);
            tr[x].rev = false;
        }
    }
    void rotate(int x) {
        int y = tr[x].p, z = tr[y].p;
        int k = tr[y].ch[1] == x, w = tr[x].ch[k ^ 1];
        if (!root(y)) tr[z].ch[tr[z].ch[1] == y] = x;
        tr[x].p = z;
        tr[x].ch[k ^ 1] = y;
        tr[y].p = x;
        tr[y].ch[k] = w;
        if (w) tr[w].p = y;
        pull(y);
        pull(x);
    }
    void splay(int x) {
        vector<int> st = { x };
        for (int y = x; !root(y);)
            st.push_back(y = tr[y].p);
        while (!st.empty())
            push(st.back()), st.pop_back();
        while (!root(x)) {
            int y = tr[x].p, z = tr[y].p;
            if (!root(y))
                rotate((tr[y].ch[1] == x) ==
                               (tr[z].ch[1] == y)
                           ? y
                           : x);
            rotate(x);
        }
    }
    void access(int x) {
        for (int y = 0; x; y = x, x = tr[x].p) {
            splay(x);
            tr[x].ch[1] = y;
            pull(x);
        }
    }
    void makeroot(int x) {
        access(x);
        splay(x);
        flip(x);
    }
    int findroot(int x) {
        access(x);
        splay(x);
        push(x);
        while (tr[x].ch[0])
            x = tr[x].ch[0], push(x);
        splay(x);
        return x;
    }
    bool link(int x, int y) {
        makeroot(x);
        if (findroot(y) == x) return false;
        tr[x].p = y;
        return true;
    }
    bool cut(int x, int y) {
        makeroot(x);
        access(y);
        splay(y);
        if (tr[y].ch[0] != x || tr[x].ch[1])
            return false;
        tr[y].ch[0] = tr[x].p = 0;
        pull(y);
        return true;
    }
    void set(int x, i64 v) {
        access(x);
        splay(x);
        tr[x].val = v;
```

```
        pull(x);
    }
    i64 query(int x, int y) {
        makeroot(x);
        access(y);
        splay(y);
        return tr[y].sum;
    }
};
```

2.14 直线最小值查询（离散李超树）

功能：插入直线，在预先登记的横坐标处查询所有直线的最小值。
约定：所有查询横坐标提前收集；query 的 x 是 xs 中的下标，根参数 (1,0,xs.size()-1)。返回最小值 i128；至少加一条线后查询。
复杂度：$\mathcal{O}(\log n)$ 每次
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 b875ba

```
struct LiChao {
    struct Line {
        i64 k = 0, b = 0;
        i128 get(i64 x) const {
            return i128(k) * x + b;
        }
    };
    vector<i64> xs;
    vector<Line> tr;
    vector<bool> has;
    LiChao(vector<i64> a) : xs(a) {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()),
                 xs.end());
        assert(!xs.empty());
        tr.resize(xs.size() * 4);
        has.resize(tr.size());
    }
    void add(Line v, int p, int l, int r) {
        if (!has[p]) {
            tr[p] = v;
            has[p] = true;
            return;
        }
        int m = (l + r) / 2;
        bool left = v.get(xs[l]) < tr[p].get(xs[l]);
        bool mid = v.get(xs[m]) < tr[p].get(xs[m]);
        if (mid) swap(v, tr[p]);
        if (l == r) return;
        if (left != mid)
            add(v, p * 2, l, m);
        else
            add(v, p * 2 + 1, m + 1, r);
    }
    void add(i64 k, i64 b) {
        add({ k, b }, 1, 0, xs.size() - 1);
    }
    i128 query(int x, int p, int l, int r) const {
        i128 ans =
            has[p] ? tr[p].get(xs[x]) : i128(1) << 126;
        if (l == r) return ans;
        int m = (l + r) / 2;
        return min(ans, x <= m ? query(x, p * 2, l, m)
                               : query(x, p * 2 + 1,
                                       m + 1, r));
    }
};
```

2.15 直线最大值查询（动态凸包）

功能：动态插入直线，查询指定横坐标处最大的 kx+b。
约定：add(k,b) 后 query(x) 求最大 kx+b；求最小值可取负。差值、交点整除和求值均须不溢出 i64；至少插入一条线。
复杂度：均摊 $\mathcal{O}(\log n)$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 97d94c

```
struct Line {
    mutable i64 k, m, p;
    bool operator<(const Line &o) const {
        return k < o.k;
    }
    bool operator<(i64 x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const i64 inf = LLONG_MAX;
    i64 div(i64 a, i64 b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(i64 k, i64 m) {
        auto z = insert({ k, m, 0 }), y = z ++, x = y;
        while (isect(y, z))
            z = erase(z);
        if (x != begin() && isect(--x, y))
            isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    i64 query(i64 x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

3 图论

3.1 强连通分量

功能：分解有向图的强连通分量，提供逆拓扑序分量编号。
约定：0-based。comp 编号为逆拓扑序；solve 一次，修改图后重建对象。深链注意栈。
复杂度：$\mathcal{O}(n+m)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 805bae

```
struct SCC {
    int timer = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, comp, st;
    SCC(int n) : g(n), dfn(n), low(n), comp(n, -1) {}
    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        for (int v : g[u]) {
            if (!dfn[v])
                dfs(v), low[u] = min(low[u], low[v]);
            else if (comp[v] == -1)
                low[u] = min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u]) {
            int v;
            do {
                v = st.back();
                st.pop_back();
                comp[v] = cnt;
            } while (v != u);
            cnt ++;
        }
    }
    void solve() {
        for (int i = 0; i < (int)g.size(); i ++)
            if (!dfn[i]) dfs(i);
    }
};
```

3.2 布尔约束可行解（2-SAT）

功能：求解由两文字析取组成的布尔约束，返回一组可行赋值。
约定：变量 0..n-1；文字 2*x+v 表示 x=v，取反用 ^1。either 添加析取，imply 同时添加逆否命题；求解一次。
复杂度：$\mathcal{O}(n+m)$
依赖：§1.1, §3.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 7cee7d

```
struct TwoSat {
    int n;
    SCC s;
    vector<bool> ans;
    TwoSat(int n) : n(n), s(2 * n), ans(n) {}
    // Literal 2*x+v means variable x takes value v.
    void imply(int a, int b) {
        s.g[a].push_back(b);
        s.g[b ^ 1].push_back(a ^ 1);
    }
    void either(int a, int b) { imply(a ^ 1, b); }
    void force(int a) { either(a, a); }
    bool solve() {
        s.solve();
        for (int i = 0; i < n; i ++) {
            if (s.comp[2 * i] == s.comp[2 * i + 1])
                return false;
            ans[i] = s.comp[2 * i + 1] < s.comp[2 * i];
        }
        return true;
    }
};
```

3.3 割点、桥、点双与边双

功能：枚举割点、桥、点双与边双连通分量，并构建圆方森林。
约定：顶点编号 0..n-1，无向图可不连通、可有平行边；自环需在 add 前过滤。先 BCC b(n)，逐条 b.add(u,v)，再 b.solve()，最后读取结果或调用 b.block_tree()。每个对象只 solve 一次，修改图后需重建对象。dfs、color 为内部过程；深链注意递归栈。cut[u] 判割点，bridge[id] 判桥，ec[u] 为边双编号，blocks[i] 为第 i 个点双的顶点集。点双可共享割点；每个顶点只属于一个边双。block_tree() 返回点双的圆方森林。
复杂度：DFS $\mathcal{O}(n+m)$，点双去重额外 $\mathcal{O}(m \log m)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 c898cb

```
struct BCC {
    // n: 原图顶点数；timer: DFS 时间戳，从 1 开始分配。
    int n, timer = 0;
    // g[u] 存 { 邻点, 无向边编号 }；
    ↪ 一条边的两个方向共用同一编号。
    vector<vector<pair<int, int>>> g;
    // ed[id] = { u, v }: 编号 id 的边的两个端点，编号按 add
    ↪ 顺序从 0 开始。
    vector<pair<int, int>> ed;
    // dfn[u]: 首次访问时间，0 表示未访问。
    // low[u]: 从 u 的 DFS 子树经至多一条返祖边可达的最小
    ↪ dfn。
    // st: 尚未归入点双的边编号栈，不是顶点栈。
    // ec[u]: u 所属边双的编号，求解前为 -1，求解后从 0
    ↪ 连续编号。
    // 同一边双内任意两点删除任意一条边后仍连通；
    ↪ 孤立点也单独成分量。
    vector<int> dfn, low, st, ec;
    // cut[u]: 删除顶点 u 后，图的连通分量数是否增加（割点）。
    // bridge[id]: 删除编号 id 的边后，
    ↪ 图的连通分量数是否增加（桥）。
    vector<bool> cut, bridge;
    // blocks[i]: 第 i 个点双的顶点列表，已排序去重，编号从
    ↪ 0 开始。
    // 割点可出现在多个列表中；桥的两端构成一个二点块，
    ↪ 孤立点构成单点块。
    // blocks 的编号与 ec 的编号无关，blocks.size()
    ↪ 是点双数，不是边双数。
    vector<vector<int>> blocks;
    // 构造 n 个顶点的空图。
    BCC(int n)
        : n(n), g(n), dfn(n), low(n), ec(n, -1),
          cut(n) {}
    // 加一条无向边，只调用一次，不必再 add(v,u)；要求 u !=
    ↪ v。
    void add(int u, int v) {
        assert(u != v);
        int id = ed.size();
        ed.push_back({ u, v });
        g[u].push_back({ v, id });
        g[v].push_back({ u, id });
    }
    // 内部 Tarjan DFS；pe 为父边编号，DFS 根传 -1。
    // 按父边编号跳过反向边，保证平行边仍能作为返祖边更新
    ↪ low。
    void dfs(int u, int pe) {
        dfn[u] = low[u] = ++timer;
        // child: u 在 DFS 树中的直接孩子数，
        ↪ 用于判断根是否为割点。
        int child = 0;
        for (auto [v, id] : g[u])
            if (id != pe) {
                if (!dfn[v]) {
                    child ++;
                    st.push_back(id);
                    dfs(v, id);
                    low[u] = min(low[u], low[v]);
                    // v 的子树无法绕回 u 或其祖先，则树边
                    ↪ (u,v) 是桥。
                    bridge[id] = low[v] > dfn[u];
                    if (low[v] >= dfn[u]) {
                        // 子树无法绕过 u 回到其祖先：
                        ↪ 弹出一个点双。
                        // 非根 u 是割点；根仅在有至少两个
                        ↪ DFS 孩子时是割点。
                        cut[u] = pe != -1 || child > 1;
                        // b 收集该点双的端点，x
                        ↪ 是从边栈弹出的边编号。
                        vector<int> b;
                        int x;
                        do {
                            x = st.back();
                            st.pop_back();
                            b.push_back(ed[x].first);
                            b.push_back(ed[x].second);
                        } while (x != id);
                        sort(b.begin(), b.end());
                        b.erase(
                            unique(b.begin(), b.end()),
                            b.end());
                        blocks.push_back(b);
                    }
                } else if (dfn[v] < dfn[u]) {
                    st.push_back(id);
                    low[u] = min(low[u], dfn[v]);
                }
            }
    }
    // 内部染色：忽略所有桥，把 u 所在的连通块标为边双 c。
    void color(int u, int c) {
        ec[u] = c;
        for (auto [v, id] : g[u])
            if (!bridge[id] && ec[v] == -1) color(v, c);
    }
    // 处理所有连通分量，填好 cut、bridge、blocks、ec。
    void solve() {
        bridge.resize(ed.size());
        for (int u = 0; u < n; u ++)
            if (!dfn[u]) {
                dfs(u, -1);
                if (g[u].empty())
                    blocks.push_back({ u });
            }
        // c 是已分配的边双数；结束后未存为成员。
        // 外部可用 n ? *max_element(ec.begin(), ec.end())
        ↪ + 1 : 0 求边双数。
        int c = 0;
        for (int u = 0; u < n; u ++)
```

```
            if (ec[u] == -1) color(u, c ++);
        }
        // solve 后调用，返回圆方森林的邻接表，共 n + blocks.size() 个结点。
        // 圆点 0..n-1 对应原图顶点；方点 n+i 对应 blocks[i]。
        // u 属于 blocks[i] 时连边 u <-> n+i；孤立点也会连向自己的单点块。
        // 这是点双圆方森林；若要边双缩点后的桥森林，另枚举 bridge[id]，
        // 将 ed[id] 的两个端点映射到 ec 后连边即可。
        vector<vector<int>> block_tree() {
            vector<vector<int>> t(n + blocks.size());
            for (int i = 0; i < (int)blocks.size(); i ++)
                for (int u : blocks[i]) {
                    t[u].push_back(n + i);
                    t[n + i].push_back(u);
                }
            return t;
        }
    };
```

3.4 欧拉路径

功能：求有向或无向图中恰好经过每条边一次的路径。
约定：0-based；邻接项 (终点,边编号)，边编号 0..m-1。无向边两端使用同一编号。有向路径从出入度差为 1 的点起；无向从奇度点起；无边时返回起点。
复杂度：$\mathcal{O}(n+m)$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 66504d

```
vector<int>
eulerWalk(vector<vector<pair<int, int>>> &gr,
          int nedges, int src = 0) {
    int n = (int)(gr).size();
    vector<int> D(n), its(n), eu(nedges), ret,
        s = { src };
    D[src] ++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x],
            end = (int)(gr[x]).size();
        if (it == end) {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it ++];
        if (!eu[e]) {
            D[x] --, D[y] ++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D)
        if (x < 0 || (int)(ret).size() != nedges + 1)
            return {};
    return { ret.rbegin(), ret.rend() };
}
```

3.5 非负权最短路 / 负环检测

功能：求单源非负权最短路，并提供任意位置负环的检测与恢复。
约定：0-based。Dijkstra 要求非负边和最短路小于 INF；负环检测相当于接超级源，返回环顶点（不重复首点），支持任意位置负环。
复杂度：Dijkstra $\mathcal{O}((n+m)\log n)$，负环 $\mathcal{O}(nm)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 b7d3e4

```
vector<i64>
dijkstra(const vector<vector<pair<int, i64>>> &g,
         int s) {
    const i64 inf = LLONG_MAX / 4;
    vector<i64> d(g.size(), inf);
    priority_queue<pair<i64, int>,
                   vector<pair<i64, int>>,
                   greater<pair<i64, int>>>
        q;
    d[s] = 0;
    q.push({ 0, s });
    while (!q.empty()) {
        auto [dis, u] = q.top();
        q.pop();
        if (dis != d[u]) continue;
        for (auto [v, w] : g[u])
            if (d[v] > dis + w) {
                d[v] = dis + w;
                q.push({ d[v], v });
            }
    }
    return d;
}
vector<int>
negative_cycle(int n,
               const vector<tuple<int, int, i64>> &ed) {
    vector<i128> d(n);
    vector<int> p(n, -1);
    int x = -1;
    for (int i = 0; i < n; i ++) {
        x = -1;
        for (auto [u, v, w] : ed)
            if (d[v] > d[u] + w)
                d[v] = d[u] + w, p[v] = u, x = v;
    }
    if (x == -1) return {};
    for (int i = 0; i < n; i ++)
        x = p[x];
    vector<int> a = { x };
    for (int v = p[x]; v != x; v = p[v])
        a.push_back(v);
    reverse(a.begin(), a.end());
    return a;
}
```

3.6 有向最小生成树 / 最小树形图

功能：求从指定根可达所有点的有向生成树最小总权值。
约定：Chu-Liu/Edmonds；0-based，求 root 到所有点的最小树形图权值，失败 nullopt。支持负权和平行边；不恢复边。边权和总和须不溢出。
复杂度：$\mathcal{O}(nm)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 21f5d7

```
struct DEdge {
    int u, v;
    i64 w;
};
optional<i64> directed_mst(int n, int root,
                           vector<DEdge> ed) {
    i64 ans = 0;
    const i64 inf = LLONG_MAX / 4;
    while (true) {
        vector<i64> in(n, inf);
        vector<int> pre(n), id(n, -1), vis(n, -1);
        for (auto e : ed)
            if (e.u != e.v && e.w < in[e.v])
                in[e.v] = e.w, pre[e.v] = e.u;
        in[root] = 0;
        for (i64 w : in) {
            if (w == inf) return nullopt;
            ans += w;
        }
        int cnt = 0;
        for (int i = 0; i < n; i ++) {
            int v = i;
            while (v != root && id[v] == -1 &&
                   vis[v] != i)
                vis[v] = i, v = pre[v];
            if (v != root && id[v] == -1) {
                for (int u = pre[v]; u != v; u = pre[u])
                    id[u] = cnt;
                id[v] = cnt ++;
            }
```

```
        }
        if (!cnt) return ans;
        for (int i = 0; i < n; i ++)
            if (id[i] == -1) id[i] = cnt ++;
        for (auto &e : ed) {
            i64 w = in[e.v];
            e.u = id[e.u];
            e.v = id[e.v];
            if (e.u != e.v) e.w -= w;
        }
        root = id[root];
        n = cnt;
    }
};
```

3.7 支配树

功能：求从入口出发到达某点时必经的支配关系，返回直接支配点。
约定：0-based 有向图；返回直接支配点，根指向自身，不可达为 -1。每个对象求解一次；递归栈按图深度准备。
复杂度：$\mathcal{O}((n+m)\log n)$
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 df3804

```
struct Dominator {
    int timer = 0;
    vector<vector<int>> g, rg, bucket;
    vector<int> dfn, seq, par, best, semi, parent, dom;
    Dominator(int n)
        : g(n), rg(n + 1), bucket(n + 1), dfn(n),
          seq(n + 1), par(n + 1), best(n + 1),
          semi(n + 1), parent(n + 1), dom(n + 1) {}
    int find(int x, int depth = 0) {
        if (par[x] == x) return depth ? -1 : x;
        int p = find(par[x], depth + 1);
        if (p == -1) return depth ? par[x] : best[x];
        if (semi[best[x]] > semi[best[par[x]]])
            best[x] = best[par[x]];
        par[x] = p;
        return depth ? p : best[x];
    }
    void dfs(int u) {
        int x = dfn[u] = ++timer;
        seq[x] = u;
        par[x] = best[x] = semi[x] = x;
        for (int v : g[u]) {
            if (!dfn[v]) dfs(v), parent[dfn[v]] = x;
            rg[dfn[v]].push_back(x);
        }
    }
    vector<int> solve(int root) {
        dfs(root);
        for (int i = timer; i; i --) {
            for (int v : rg[i])
                semi[i] = min(semi[i], semi[find(v)]);
            if (i > 1) bucket[semi[i]].push_back(i);
            for (int v : bucket[i]) {
                int p = find(v);
                dom[v] = semi[p] == i ? i : p;
            }
            if (i > 1) par[i] = parent[i];
        }
        vector<int> ans(g.size(), -1);
        ans[root] = root;
        for (int i = 2; i <= timer; i ++) {
            if (semi[i] != dom[i]) dom[i] = dom[dom[i]];
            ans[seq[i]] = seq[dom[i]];
        }
        return ans;
    }
};
```

3.8 必经与删边最短路选型

先分清：问的是所有路径还是所有最短路径，端点是否固定，删除后是否恢复，以及图是否有向、边权是否严格为正。以下“必经”均要求 s 原本能到 t；不通时返回不可达，不把所有点边都算成必经。端点本身算必经点；s=t 时允许空路径。

无向图所有路径的必经边：先找桥、缩边双，得到桥森林；s、t 所在边双之间的树路径上的桥才是该点对的必经边。并非图中任意一座桥都必经。必经点可用 BCC.h 的圆方森林：原顶点 s 到 t 的树路径上，内部圆点恰是分离两者的割点，另加两个端点。多组询问可在森林上预处理 LCA；不同树表示不连通。

有向图所有路径的必经点：以 s 为根建立支配树，t 的祖先恰好是必经点。Dominator.h 返回的是直接支配点数组，不是每点的全部必经点。固定 s、多目标查询时只需建一次支配树；若问 u 是否是到 v 的必经点，检查支配树上 u 是否为 v 的祖先。

必经边通过拆边转成必经点：将原边 id 的 u→v 改成 u→(n+id)→v。每条边有独立新点，平行边不能合并。支配 t 的新点对应必经边。Mandatory.h 的 mandatory_st 已封装固定 s,t 的点边标记；若需一个源到所有目标的关系，直接用拆边后的 Dominator，避免为每个目标重复建树。原图支配树的一条父子边，并不自动等于原图必经边。

仅写“删除一条 s-t 路径，剩下跑 Tarjan”，没有说明维护何种结构、如何处理原路径及旁路，不能直接当成完整模板。单看删路后图的 SCC 也不能据此判断原图的全部必经边。这里统一采用定义明确的支配树方案。

最短路必经点边：非负权有向图先从 s 跑 Dijkstra 得 ds，从 t 在反图跑 Dijkstra 得 dt。令 D=ds[t]，仅保留 ds[u]+w+dt[v]=D 的边，然后在该子图上求必经点边。Mandatory.h 的 mandatory_shortest 采用此法，输出仍按原图的边 ID 对齐；支持零权边和零权环，结果不依赖哈希。

若边权严格为正，最短路子图按 ds 严格递增，是 DAG。可以数 s→u 与 v→t 的最短路条数，检查两者乘积是否等于 s→t 的总条数来判断边 u→v；对顶点类似。模数计数相等有碰撞可能，双模也不保证绝对正确；精确大整数计数另有位复杂度。存在零权环时，不能直接按距离排序做上述 DAG 路径计数；也不能只把 SCC 缩点后就丢掉 SCC 内部的必经关系。

无向图独立删边最短路：ReplacementPaths.h 固定 s,t，每问删一条边，回答后恢复。先选一条最短路 P；不在 P 上的边删掉后仍可走 P，所以答案不变。模板要求严格正权，用两棵最短路树以及区间取 min 同时处理 P 上所有边。最短路树本身没有给出答案，还需要枚举备用边。

具体地，让 s 树与 t 树都包含同一条 P；从树上去掉 P 的边后，ls[u]、lt[v] 表示各自分支连接 P 的位置。对不属于 P 的原边 u-v，候选 ds[u]+w+dt[v] 可避开 P 上编号在 [ls[u],lt[v]) 的边，两种方向分别处理。只排除 P 的边 ID，平行的备用边必须保留。正权保证两树的路径方向一致，且每个点满足 ls<=lt；最优替换路径从前缀区域走到后缀区域时，总有一条跨区边提供不劣候选。因此对区间取 min 即得答案。结果只含距离，不支持累计删除、删点或任意改变端点。

有向图独立删边最短路：也可以求解，不能概括成“做不了”。通用竞赛基线是选一条简单最短路 P，只对 P 上至多 n-1 条边逐次禁用并重跑 Dijkstra；其他边的答案不变。用惰性堆、允许多重边时，总时界为 O(n(n+m) log(n+m))，单位权图可改 BFS。无向两树区间法的论证不能直接搬到有向图；大规模有向图需另看权值、稠密度和题目限制，不在本模板范围内。

参考与进一步说明：支配关系、正权替换路及 Beats 的来源链接见 SOURCES.md。本页说明的是静态图、单次失效模型；在线动态最短路不能直接套用。

3.9 固定端点必经点 / 边

功能：用拆边和支配树精确求有向图 s 到 t 的必经点、边，也支持非负权最短路。
约定：0-based，允许平行边、自环；边按输入顺序编号。reachable=false 时所有标记为 false。可达时端点也算必经点；s=t 允许空路径，只有 s 必经。每条边拆成新点，可能有 O(n+m) 递归深度。mandatory_shortest 要求 0<=w<LLONG_MAX/4，所有需表示的有限最短距离小于该上界；支持零权环。

调用：auto a=mandatory_st(3,{{0,1},{1,2}},0,2); // a.vertex[1]、a.edge[0] 为 true。
复杂度：$\mathcal{O}((n+m)\log(n+m))$，空间 $\mathcal{O}(n+m)$
依赖：§1.1, §3.7, §3.5（递归收齐依赖）。

BlueLagoon / CC0-1.0 (wrapper only; dependencies retain their licenses) / 本地回归 / 对拍
0ae814

```cpp
struct MandatoryResult {
    bool reachable;
    vector<bool> vertex,
        edge; // 原顶点 / 原边是否出现在每条 s-t 路径上。
};

MandatoryResult
mandatory_st(int n, const vector<pair<int, int>> &ed,
             int s, int t) {
    assert(0 <= s && s < n && 0 <= t && t < n);
    int m = ed.size();
    MandatoryResult ans{ false, vector<bool>(n),
                         vector<bool>(m) };
    Dominator d(n + m);
    for (int i = 0; i < m; i ++) {
        auto [u, v] = ed[i];
        assert(0 <= u && u < n && 0 <= v && v < n);
        // 每条边各有一个新点，平行边也不能共用。
        d.g[u].push_back(n + i);
        d.g[n + i].push_back(v);
    }
    auto p = d.solve(s);
    if (p[t] == -1) return ans;
    ans.reachable = true;
    // 支配树上 t 到根的祖先恰好是所有必经点。
    for (int x = t;; x = p[x]) {
        if (x < n)
            ans.vertex[x] = true;
        else
            ans.edge[x - n] = true;
        if (x == s) break;
    }
    return ans;
}

MandatoryResult mandatory_shortest(
    int n, const vector<tuple<int, int, i64>> &ed,
    int s, int t) {
    assert(0 <= s && s < n && 0 <= t && t < n);
    const i64 inf = LLONG_MAX / 4;
    vector<vector<pair<int, i64>>> g(n), rg(n);
    for (auto [u, v, w] : ed) {
        assert(0 <= u && u < n && 0 <= v && v < n &&
               0 <= w && w < inf);
        g[u].push_back({ v, w });
        rg[v].push_back({ u, w });
    }
    auto a = dijkstra(g, s), b = dijkstra(rg, t);
    MandatoryResult ans{ false, vector<bool>(n),
                         vector<bool>(ed.size()) };
    if (a[t] == inf) return ans;
    vector<pair<int, int>> tight;
    vector<int> id;
    for (int i = 0; i < (int)ed.size(); i ++) {
        auto [u, v, w] = ed[i];
        if (a[u] != inf && b[v] != inf &&
            a[u] + w + b[v] == a[t]) {
            tight.push_back({ u, v });
            id.push_back(i);
        }
    }
    auto z = mandatory_st(n, tight, s, t);
    ans.reachable = true;
    ans.vertex = move(z.vertex);
    for (int i = 0; i < (int)id.size(); i ++)
        ans.edge[id[i]] = z.edge[i];
    return ans;
}
```

3.10 无向正权图单边删除最短路

功能：固定 s,t，预处理每条边独立删除后的 s-t 最短距离。
约定：0-based，无向严格正权图，可有平行边和自环；不是累计删边，也不是删点。solve(s,t) 返回按边 ID 排列的答案，断开返回 INF；s=t 时均为 0，可重复 solve。要求 0<w<INF，所有需表示的有限最短距离小于 INF。只返回距离，不恢复替换路径。
调用：ReplacementPaths g(3); g.add(0,1,2); g.add(1,2,3); g.add(0,2,9); auto a=g.solve(0,2);
复杂度：$\mathcal{O}((n+m)\log(n+m))$ 预处理，查表 $\mathcal{O}(1)$，空间 $\mathcal{O}(n+m)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍
17ae12

```cpp
struct ReplacementPaths {
    static constexpr i64 INF = LLONG_MAX / 4;
    struct Edge {
        int u, v;
        i64 w;
    };
    int n;
    vector<Edge> ed;
    vector<vector<pair<int, int>>>
        g; // {邻点，边编号}。
    explicit ReplacementPaths(int n) : n(n), g(n) {}
    int add(int u, int v, i64 w) {
        assert(0 <= u && u < n && 0 <= v && v < n &&
               0 < w && w < INF);
        int id = ed.size();
        ed.push_back({ u, v, w });
        g[u].push_back({ v, id }),
            g[v].push_back({ u, id });
        return id;
    }
    struct Tree {
        vector<i64> d;
        vector<int> par,
            pe; // 父顶点、父边；根与不可达点均为 -1。
    };
    Tree shortest(int root) const {
        Tree a{ vector<i64>(n, INF), vector<int>(n, -1),
                vector<int>(n, -1) };
        priority_queue<pair<i64, int>,
                       vector<pair<i64, int>>,
                       greater<pair<i64, int>>>
            q;
        a.d[root] = 0;
        q.push({ 0, root });
        while (!q.empty()) {
            auto [d, u] = q.top();
            q.pop();
            if (d != a.d[u]) continue;
            for (auto [v, id] : g[u])
                if (d + ed[id].w < a.d[v]) {
                    a.d[v] = d + ed[id].w;
                    a.par[v] = u, a.pe[v] = id;
                    q.push({ a.d[v], v });
                }
        }
        return a;
    }
    // 删除树上的 P 边后，每个点所属分支挂在 P
    // 的哪个顶点。
    vector<int> labels(const Tree &a, int root,
                       const vector<int> &pos) const {
        vector<vector<int>> ch(n);
        for (int v = 0; v < n; v ++)
            if (a.par[v] != -1)
                ch[a.par[v]].push_back(v);
        vector<int> lab(n, -1), st{ root };
        lab[root] = pos[root];
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : ch[u]) {
                lab[v] = pos[v] == -1 ? lab[u] : pos[v];
                st.push_back(v);
            }
```

```
        }
        return lab;
    }
    vector<i64> solve(int s, int t) const {
        assert(0 <= s && s < n && 0 <= t && t < n);
        auto a = shortest(s);
        vector<i64> ans(ed.size(), a.d[t]);
        if (s == t || a.d[t] == INF) return ans;
        auto b = shortest(t);
        vector<int> path, pos(n, -1);
        for (int v = t; v != -1; v = a.par[v])
            path.push_back(v);
        reverse(path.begin(), path.end());
        int k = path.size() - 1;
        vector<bool> on(ed.size());
        for (int i = 0; i <= k; i ++) {
            int u = path[i];
            pos[u] = i;
            if (i < k) {
                int v = path[i + 1];
                // 强制 t 树包含同一条
                // P；正权保证不会形成父指针环。
                b.par[u] = v, b.pe[u] = a.pe[v];
                on[a.pe[v]] = true;
            }
        }
        auto l = labels(a, s, pos),
             r = labels(b, t, pos);
        vector<i64> tag(4 * k, INF);
        auto update = [&](auto self, int p, int x,
                          int y, int ql, int qr,
                          i64 v) -> void {
            if (qr <= x || y <= ql) return;
            if (ql <= x && y <= qr) {
                tag[p] = min(tag[p], v);
                return;
            }
            int m = (x + y) / 2;
            self(self, p * 2, x, m, ql, qr, v);
            self(self, p * 2 + 1, m, y, ql, qr, v);
        };
        for (int id = 0; id < (int)ed.size(); id ++) {
            if (on[id])
                continue; // 只排除这条原边
                          // ID，不能排除平行边。
            auto [u, v, w] = ed[id];
            for (int z = 0; z < 2; z ++, swap(u, v))
                if (l[u] != -1 && r[v] != -1 &&
                    l[u] < r[v]) {
                    // s 树前缀 + (u,v) + t 树后缀，绕过
                    // P 的 [l[u],r[v]) 边。
                    i64 d = a.d[u] + w + b.d[v];
                    update(update, 1, 0, k, l[u], r[v],
                           d);
                }
        }
        auto collect = [&](auto self, int p, int x,
                           int y, i64 v) -> void {
            v = min(v, tag[p]);
            if (y - x == 1) {
                ans[a.pe[path[y]]] = v;
                return;
            }
            int m = (x + y) / 2;
            self(self, p * 2, x, m, v);
            self(self, p * 2 + 1, m, y, v);
        };
        collect(collect, 1, 0, k, INF);
        return ans;
    }
};
```

3.11 无向图全局最小割

功能：在所有非平凡顶点划分中求最小割权，恢复割的一侧顶点。
约定：Stoer-Wagner；无向非负权邻接矩阵、对称、n>=2，平行边先求和。返回割权和一侧顶点；对角线忽略，总权须不溢出。
复杂度：$\mathcal{O}(n^3)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 30c90b

```
pair<i64, vector<int>> min_cut(vector<vector<i64>> a) {
    int n = a.size();
    assert(n >= 2);
    vector<int> active(n);
    iota(active.begin(), active.end(), 0);
    vector<vector<int>> co(n);
    for (int i = 0; i < n; i ++)
        co[i] = { i };
    pair<i64, vector<int>> ans = { LLONG_MAX, {} };
    while (active.size() > 1) {
        vector<i64> w(n);
        vector<bool> used(n);
        int pre = -1;
        for (int i = 0; i < (int)active.size(); i ++) {
            int v = -1;
            for (int u : active)
                if (!used[u] &&
                    (v == -1 || w[u] > w[v]))
                    v = u;
            if (i + 1 == (int)active.size()) {
                if (w[v] < ans.first)
                    ans = { w[v], co[v] };
                for (int u : active)
                    if (u != pre && u != v)
                        a[pre][u] = a[u][pre] +=
                            a[u][v];
                co[pre].insert(co[pre].end(),
                               co[v].begin(),
                               co[v].end());
                active.erase(find(active.begin(),
                                  active.end(), v));
                break;
            }
            used[v] = true;
            pre = v;
            for (int u : active)
                if (!used[u]) w[u] += a[v][u];
        }
    }
    return ans;
}
```

3.12 树同构

功能：确定性判断两棵无向树是否同构。
约定：输入无向简单树；使用同一个字典分配子树类型，确定性判同构，无哈希碰撞。深链注意栈。
复杂度：保守 $\mathcal{O}(n \log^2 n)$，空间 $\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 ad870e

```
vector<int> tree_centers(const vector<vector<int>> &g) {
    int n = g.size();
    if (n <= 2) {
        vector<int> a(n);
        iota(a.begin(), a.end(), 0);
        return a;
    }
    vector<int> d(n), q;
    for (int i = 0; i < n; i ++)
        if ((d[i] = g[i].size()) == 1) q.push_back(i);
    int head = 0, left = n;
    while (left > 2) {
        int tail = q.size();
        left -= tail - head;
        while (head < tail)
            for (int v : g[q[head ++]])
                if (--d[v] == 1) q.push_back(v);
```

```
    }
    return vector<int>(q.begin() + head, q.end());
}
int rooted_id(int u, int p,
              const vector<vector<int>> &g,
              map<vector<int>, int> &mp) {
    vector<int> a;
    for (int v : g[u])
        if (v != p) a.push_back(rooted_id(v, u, g, mp));
    sort(a.begin(), a.end());
    if (!mp.count(a)) mp[a] = mp.size();
    return mp[a];
}
bool tree_iso(const vector<vector<int>> &a,
              const vector<vector<int>> &b) {
    if (a.size() != b.size()) return false;
    if (a.empty()) return true;
    map<vector<int>, int> mp;
    int x = rooted_id(tree_centers(a)[0], -1, a, mp);
    for (int r : tree_centers(b))
        if (rooted_id(r, -1, b, mp) == x) return true;
    return false;
}
```

3.13 离线动态连通性

功能：离线处理边的加入 / 删除时段，查询各时刻的连通关系。
约定：t>=1，时刻 0..t-1；边有效期 [x,y) 加入根 (1,0,t)。删除时配对同一条边的加入时刻；重复边需计数或分别编号；叶回调查询 DSU。
复杂度：$\mathcal{O}$(边有效期数 $\log t \log n$ + 查询数 $\log n$)
依赖：§1.1, §2.8（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 4ece16

```
struct DynamicConnectivity {
    int t;
    RollbackUF dsu;
    vector<vector<pair<int, int>>> ed;
    DynamicConnectivity(int n, int t)
        : t(t), dsu(n), ed(4 * t + 4) {}
    void add(int p, int l, int r, int x, int y,
             pair<int, int> e) {
        if (x >= r || y <= l) return;
        if (x <= l && r <= y) {
            ed[p].push_back(e);
            return;
        }
        int m = (l + r) / 2;
        add(p * 2, l, m, x, y, e);
        add(p * 2 + 1, m, r, x, y, e);
    }
    template <class F>
    void solve(int p, int l, int r, F &f) {
        int tm = dsu.time();
        for (auto [u, v] : ed[p])
            dsu.join(u, v);
        if (r - l == 1)
            f(l, dsu);
        else {
            int m = (l + r) / 2;
            solve(p * 2, l, m, f);
            solve(p * 2 + 1, m, r, f);
        }
        dsu.rollback(tm);
    }
};
```

4 网络流与匹配

4.1 最大流 / 最小割（Dinic）

功能：求 s 到 t 最大流，恢复原边流量，并定位最小割的源侧。
约定：0-based，非负容量，返回新增流量；默认 lim=LLONG_MAX/4。add 返回邻接边编号。终止后另跑 bfs 得最小割可达侧；反向初始容量非零时 used 是净流。
复杂度：$\mathcal{O}(n^2m)$，二分图单位网络 $\mathcal{O}(m\sqrt{n})$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 a5f972

```
struct Dinic {
    struct Edge {
        int to, rev;
        i64 cap, init;
    };
    vector<vector<Edge>> g;
    vector<int> dep, cur;
    Dinic(int n) : g(n), dep(n), cur(n) {}
    int add(int u, int v, i64 c, i64 rc = 0) {
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({ v, b + (u == v), c, c });
        g[v].push_back({ u, a, rc, rc });
        return a;
    }
    bool bfs(int s, int t) {
        fill(dep.begin(), dep.end(), -1);
        queue<int> q;
        dep[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto e : g[u])
                if (e.cap && dep[e.to] == -1)
                    dep[e.to] = dep[u] + 1,
                    q.push(e.to);
        }
        return dep[t] != -1;
    }
    i64 dfs(int u, int t, i64 f) {
        if (u == t) return f;
        for (int &i = cur[u]; i < (int)g[u].size();
             i ++) {
            auto &e = g[u][i];
            if (!e.cap || dep[e.to] != dep[u] + 1)
                continue;
            i64 v = dfs(e.to, t, min(f, e.cap));
            if (v) {
                e.cap -= v;
                g[e.to][e.rev].cap += v;
                return v;
            }
        }
        return 0;
    }
    i64 flow(int s, int t, i64 lim = LLONG_MAX / 4) {
        assert(s != t);
        i64 ans = 0;
        while (ans < lim && bfs(s, t)) {
            fill(cur.begin(), cur.end(), 0);
            while (i64 f = dfs(s, t, lim - ans))
                ans += f;
        }
        return ans;
    }
    i64 used(int u, int id) const {
        return g[u][id].init - g[u][id].cap;
    }
};
```

4.2 有上下界可行流 / 最大流 / 最小流

功能：求满足边流量上下界的可行环流及非负源汇流的最大 / 最小值。
约定：0-based；无源汇环流调用 feasible。solve 求非负 s→t 流的最大 / 最小值；对象只求解一次。原边流量 = 下界 +d.used(u,id)。容量与平衡量须小于 INF。
复杂度：一次可行性最大流＋一次残量最大流
依赖：§1.1, §4.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 a6af3a

```
struct LowerBound {
    int n, ss, tt;
    Dinic d;
```

```
    vector<i64> bal;
    LowerBound(int n)
        : n(n), ss(n), tt(n + 1), d(n + 2), bal(n) {}
    int add(int u, int v, i64 l, i64 r) {
        assert(0 <= l && l <= r);
        bal[u] -= l;
        bal[v] += l;
        return d.add(u, v, r - l);
    }
    bool feasible() {
        i64 need = 0;
        for (int i = 0; i < n; i ++) {
            if (bal[i] > 0)
                d.add(ss, i, bal[i]), need += bal[i];
            if (bal[i] < 0) d.add(i, tt, -bal[i]);
        }
        return d.flow(ss, tt) == need;
    }
    optional<i64> solve(int s, int t,
                        bool maximum = true) {
        int id = d.add(t, s, LLONG_MAX / 4);
        if (!feasible()) return nullopt;
        i64 base = d.used(t, id);
        for (int u = 0; u < n + 2; u ++)
            for (auto &e : d.g[u])
                if (u >= n || e.to >= n) e.cap = 0;
        auto &e = d.g[t][id];
        d.g[s][e.rev].cap = e.cap = 0;
        return maximum ? base + d.flow(s, t)
                       : base - d.flow(t, s, base);
    }
};
```

4.3 最小费用流 / 最小费用最大流

功能：在流量上限内尽量增广并最小化费用，可用于最小费用最大匹配。
约定：非负整数容量，支持负费用但初始正容量残量图不得有负环。返回新增流量 / 费用，lim 控制需求；费用、势能、距离与乘积需不溢出。
复杂度：$\mathcal{O}(nm + \text{增广次数} m \log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 63b856

```
struct MCF {
    struct Edge {
        int to, rev;
        i64 cap, cost;
    };
    int n;
    vector<vector<Edge>> g;
    MCF(int n) : n(n), g(n) {}
    void add(int u, int v, i64 cap, i64 cost) {
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({ v, b + (u == v), cap, cost });
        g[v].push_back({ u, a, 0, -cost });
    }
    pair<i64, i64> flow(int s, int t,
                        i64 lim = LLONG_MAX / 4) {
        assert(s != t);
        const i64 inf = LLONG_MAX / 4;
        vector<i64> h(n), dis(n);
        vector<int> pv(n), pe(n);
        // Virtual source reaches every vertex. No
        // negative cycle.
        for (int it = 0; it < n; it ++) {
            bool changed = false;
            for (int u = 0; u < n; u ++)
                for (auto e : g[u])
                    if (e.cap &&
                        h[e.to] > h[u] + e.cost)
                        h[e.to] = h[u] + e.cost,
                        changed = true;
            if (!changed) break;
            assert(it + 1 < n);
        }
        i64 f = 0, cost = 0;
        while (f < lim) {
            fill(dis.begin(), dis.end(), inf);
            dis[s] = 0;
            priority_queue<pair<i64, int>,
                           vector<pair<i64, int>>,
                           greater<pair<i64, int>>>
                q;
            q.push({ 0, s });
            while (!q.empty()) {
                auto [du, u] = q.top();
                q.pop();
                if (du != dis[u]) continue;
                for (int i = 0; i < (int)g[u].size();
                     i ++) {
                    auto e = g[u][i];
                    i64 nd =
                        du + e.cost + h[u] - h[e.to];
                    if (e.cap && nd < dis[e.to]) {
                        dis[e.to] = nd;
                        pv[e.to] = u;
                        pe[e.to] = i;
                        q.push({ nd, e.to });
                    }
                }
            }
            if (dis[t] == inf) break;
            for (int u = 0; u < n; u ++)
                h[u] += min(dis[u], dis[t]);
            i64 z = lim - f;
            for (int v = t; v != s; v = pv[v])
                z = min(z, g[pv[v]][pe[v]].cap);
            for (int v = t; v != s; v = pv[v]) {
                auto &e = g[pv[v]][pe[v]];
                e.cap -= z;
                g[v][e.rev].cap += z;
                cost += z * e.cost;
            }
            f += z;
        }
        return { f, cost };
    }
};
```

4.4 无向图两点最小割树（Gomory-Hu）

功能：构造一棵树，用树路径最小边权表示原图任意两点的最小割。
约定：无向非负容量图，0-based；任意两点最小割等于树路径最小边权。总容量小于 Dinic 默认流量上限。
复杂度：n-1 次最大流
依赖：§1.1, §4.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 2f5df4

```
vector<tuple<int, int, i64>>
gomory_hu(int n,
          const vector<tuple<int, int, i64>> &ed) {
    vector<int> p(n);
    vector<tuple<int, int, i64>> ans;
    for (int i = 1; i < n; i ++) {
        Dinic d(n);
        for (auto [u, v, w] : ed)
            if (u != v) d.add(u, v, w, w);
        i64 f = d.flow(i, p[i]);
        d.bfs(i, p[i]);
        ans.push_back({ i, p[i], f });
        for (int j = i + 1; j < n; j ++)
            if (p[j] == p[i] && d.dep[j] != -1)
                p[j] = i;
    }
    return ans;
}
```

4.5 二分图最大匹配（Hopcroft-Karp）

功能：求二分图最大基数匹配，返回匹配数和右侧各点的配对。
约定：左侧邻接 g，右侧匹配数组 r 初始全 -1；两侧分别 0-based。返回匹配数，r[v] 是匹配左点。
复杂度：$\mathcal{O}(m\sqrt{n})$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 5215db

```
int hopcroftKarp(vector<vector<int>> &g,
                 vector<int> &r) {
    int n = (int)(g).size(), res = 0;
    vector<int> l(n, -1), q(n), d(n);
    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 ||
                (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
        for (int i = 0; i < n; i ++)
            if (l[i] == -1) q[t ++] = i, d[i] = 1;
        for (int i = 0; i < t; i ++)
            for (int v : g[q[i]]) {
                if (r[v] == -1)
                    f = 1;
                else if (!d[r[v]])
                    d[r[v]] = d[q[i]] + 1,
                    q[t ++] = r[v];
            }
        if (!f) return res;
        for (int i = 0; i < n; i ++)
            if (l[i] == -1) res += dfs(dfs, i);
    }
}
```

4.6 二分图最小点覆盖 / 最大独立集

功能：求二分图最小点覆盖；其顶点补集是最大独立集。
约定：输入左侧邻接、左右点数 n,m；结果编码左侧 0..n-1，右侧 n..n+m-1。补集为最大独立集。
复杂度：$\mathcal{O}(m\sqrt{n})$
依赖：§1.1, §4.5（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 a84f18

```
vector<int> cover(vector<vector<int>> &g, int n,
                  int m) {
    vector<int> match(m, -1);
    int res = hopcroftKarp(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match)
        if (it != -1) lfound[it] = false;
    vector<int> q, cover;
    for (int i = 0; i < n; i ++)
        if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back();
        q.pop_back();
        lfound[i] = 1;
        for (int e : g[i])
            if (!seen[e] && match[e] != -1) {
                seen[e] = true;
                q.push_back(match[e]);
            }
    }
    for (int i = 0; i < n; i ++)
        if (!lfound[i]) cover.push_back(i);
    for (int i = 0; i < m; i ++)
        if (seen[i]) cover.push_back(n + i);
    assert((int)(cover).size() == res);
    return cover;
}
```

4.7 二分图最小权匹配（KM / Hungarian）

功能：求覆盖左侧所有点的最小权匹配；两侧等大时为最小权完美匹配。
约定：输入 cost 为 n×m 费用矩阵，要求 n<=m、左侧每点都匹配到不同右点；返回最小总费用与左侧对应的右侧下标（0-based）。n=m 为完美匹配；支持负权，最大权可把费用取负。不负责先最大化匹配数量；稀疏图或不存在边的情形优先使用最小费用流。费用差与总费用须可表示；空矩阵合法。
调用：auto [cost,match]=weightedMatching(a);，例如 a={{3,1},{2,4}} 得费用 3，match={1,0}。
复杂度：$\mathcal{O}(n^2m)$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 c05e93

```
template <class T>
pair<T, vector<int>>
weightedMatching(vector<vector<T>> &C) {
    int i = (int)(C).size(),
        m = i ? (int)(C[0]).size() : 0, c, s, r;
    vector<T> dist(m), pot(m);
    vector<int> match(i), rev(m, -1), cols(m), prev(m);
    T d = 0, nd, cost = 0;
    while (i --) {
        for (int c = 0; c < m; c ++)
            dist[c] = C[i][c], cols[c] = c, prev[c] = i;
        for (s = 0;;) {
            for (int j = s; j < m; j ++) {
                c = cols[j], nd = dist[c] - pot[c];
                if (j == s || d > nd)
                    d = nd, swap(cols[s], cols[j]);
            }
            if ((r = rev[c = cols[s ++]]) == -1) break;
            for (int j = 0; j < m; j ++)
                if (dist[j] >
                    (nd = C[r][j] - C[r][c] + dist[c]))
                    dist[j] = nd, prev[j] = r;
        }
        cost += dist[c];
        while (s --)
            pot[cols[s]] = dist[cols[s]] - d;
        for (; r != i; swap(c, match[r]))
            r = rev[c] = prev[c];
    }
    return { cost, match };
}
```

4.8 一般图最大匹配（带花树）

功能：求无向一般图的最大基数匹配，并恢复配对。
约定：1-based，n<=2020；init(n)，addEdge(u,v)，Match() 返回匹配对数，match 保存配对。忽略自环；对象较大建议全局分配。
复杂度：$\mathcal{O}(nm)$ 至 $\mathcal{O}(n^3)$ 上界
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 fc70ba

```
struct Blossom {
    static const int MAXN = 2020 + 1;
    // 1-based Vertex index
    int vis[MAXN], par[MAXN], orig[MAXN], match[MAXN],
        aux[MAXN], t, N;
    vector<int> conn[MAXN];
    queue<int> Q;
    void addEdge(int u, int v) {
        conn[u].push_back(v);
        conn[v].push_back(u);
    }
    void init(int n) {
        N = n;
        t = 0;
        orig[0] = 0;
        for (int i = 0; i <= n; ++i) {
            conn[i].clear();
            match[i] = aux[i] = par[i] = 0;
        }
    }
```

```
    void augment(int u, int v) {
        int pv = v, nv;
        do {
            pv = par[v];
            nv = match[pv];
            match[v] = pv;
            match[pv] = v;
            v = nv;
        } while (u != pv);
    }
    int lca(int v, int w) {
        ++t;
        while (true) {
            if (v) {
                if (aux[v] == t) return v;
                aux[v] = t;
                v = orig[par[match[v]]];
            }
            swap(v, w);
        }
    }
    void blossom(int v, int w, int a) {
        while (orig[v] != a) {
            par[v] = w;
            w = match[v];
            if (vis[w] == 1) Q.push(w), vis[w] = 0;
            orig[v] = orig[w] = a;
            v = par[w];
        }
    }
    bool bfs(int u) {
        fill(vis + 1, vis + 1 + N, -1);
        iota(orig + 1, orig + N + 1, 1);
        Q = queue<int>();
        Q.push(u);
        vis[u] = 0;
        while (!Q.empty()) {
            int v = Q.front();
            Q.pop();
            for (int x : conn[v]) {
                if (vis[x] == -1) {
                    par[x] = v;
                    vis[x] = 1;
                    if (!match[x])
                        return augment(u, x), true;
                    Q.push(match[x]);
                    vis[match[x]] = 0;
                } else if (vis[x] == 0 &&
                           orig[v] != orig[x]) {
                    int a = lca(orig[v], orig[x]);
                    blossom(x, v, a);
                    blossom(v, x, a);
                }
            }
        }
        return false;
    }
    int Match() {
        int ans = 0;
        // find random matching (not necessary, constant
        // improvement)
        vector<int> V(N);
        iota(V.begin(), V.end(), 1);
        shuffle(V.begin(), V.end(), mt19937(0x94949));
        for (auto x : V)
            if (!match[x]) {
                for (auto y : conn[x])
                    if (!match[y]) {
                        match[x] = y, match[y] = x;
                        ++ans;
                        break;
                    }
            }
        for (int i = 1; i <= N; ++i)
            if (!match[i] && bfs(i)) ++ans;
        return ans;
    }
};
```

4.9 二分图最优边染色

功能：用最大度数 Δ 种颜色给二分图边染色，使同点关联边颜色不同。

约定：两侧分别 0-based，允许平行边；输出边颜色 0..Delta-1。使用交替路径换色，内存 O((l+r)Delta)；适合中等规模。

复杂度：$\mathcal{O}(m(n+Delta))$，n=l+r

依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 d3e6a7

```
vector<int>
bipartite_edge_color(int l, int r,
                     const vector<pair<int, int>> &ed) {
    int n = l + r, m = ed.size(), delta = 0;
    vector<int> deg(n);
    for (auto [u, v] : ed)
        delta = max({ delta, ++deg[u], ++deg[l + v] });
    vector<vector<int>> at(n, vector<int>(delta, -1));
    vector<int> col(m, -1);
    auto other = [&](int id, int u) {
        return ed[id].first ^ (l + ed[id].second) ^ u;
    };
    for (int id = 0; id < m; id ++) {
        auto [u, v] = ed[id];
        v += l;
        int a = 0, b = 0;
        while (at[u][a] != -1)
            a ++;
        while (at[v][b] != -1)
            b ++;
        if (a != b) {
            vector<int> path;
            int x = v, c = a;
            while (at[x][c] != -1) {
                int e = at[x][c];
                path.push_back(e);
                x = other(e, x);
                c ^= a ^ b;
            }
            for (int e : path) {
                auto [s, t] = ed[e];
                t += l;
                at[s][col[e]] = at[t][col[e]] = -1;
            }
            for (int e : path) {
                auto [s, t] = ed[e];
                t += l;
                col[e] ^= a ^ b;
                at[s][col[e]] = at[t][col[e]] = e;
            }
        }
        col[id] = a;
        at[u][a] = at[v][a] = id;
    }
    return col;
}
```

5 字符串

5.1 前缀函数 / KMP / Z 函数

功能：计算前缀函数和 Z 数组，使用 KMP 枚举模式串出现位置。

约定：0-based，z[0]=0；KMP 返回每个匹配的起始下标，模式串非空。最短周期候选 n-p[n-1]，整周期需整除 n。

复杂度：$\mathcal{O}(n+m)$

依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 0736b1

```
vector<int> prefix_function(const string &s) {
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i ++) {
        int j = p[i - 1];
        while (j && s[i] != s[j])
            j = p[j - 1];
        p[i] = j + (s[i] == s[j]);
```

```
    }
    return p;
}
vector<int> kmp(const string &s, const string &t) {
    assert(!t.empty());
    auto p = prefix_function(t);
    vector<int> ans;
    for (int i = 0, j = 0; i < (int)s.size(); i ++) {
        while (j && s[i] != t[j])
            j = p[j - 1];
        j += s[i] == t[j];
        if (j == (int)t.size())
            ans.push_back(i - j + 1), j = p[j - 1];
    }
    return ans;
}
vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; i ++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i] ++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}
```

5.2 双模字符串哈希

功能：计算子串双模哈希，用于快速比较子串。
约定：0-based 半开区间 [l,r)。比较哈希时同时比较长度；存在碰撞，固定 base 易被针对，必要时在全局随机选同一 base。
复杂度：预处理 $\mathcal{O}(n)$，查询 $\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 3fc004

```
struct StringHash {
    static const int p = 1000000007, q = 1000000009;
    static const int base = 911382323;
    vector<array<int, 2>> h, pw;
    StringHash(const string &s)
        : h(s.size() + 1), pw(s.size() + 1) {
        pw[0] = { 1, 1 };
        for (int i = 0; i < (int)s.size(); i ++)
            for (int j = 0; j < 2; j ++) {
                int mod = j ? q : p;
                pw[i + 1][j] =
                    i64(pw[i][j]) * base % mod;
                h[i + 1][j] =
                    (i64(h[i][j]) * base +
                     (unsigned char)s[i] + 1) %
                    mod;
            }
    }
    array<int, 2> get(int l, int r) const {
        array<int, 2> ans;
        for (int j = 0; j < 2; j ++) {
            int mod = j ? q : p;
            ans[j] =
                (h[r][j] -
                 i64(h[l][j]) * pw[r - l][j] % mod +
                 mod) %
                mod;
        }
        return ans;
    }
};
```

5.3 回文半径 / 最长回文（Manacher）

功能：求每个奇 / 偶中心的最长回文半径，可据此求最长回文子串。
约定：odd[i] 对应长度 2*odd[i]-1；even[i] 对应以 i-1 与 i 之间为中心的长度 2*even[i]。
复杂度：$\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 e084cd

```
pair<vector<int>, vector<int>>
manacher(const string &s) {
    int n = s.size();
    vector<int> odd(n), even(n);
    for (int i = 0, l = 0, r = -1; i < n; i ++) {
        int k =
            i > r ? 1 : min(odd[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n &&
               s[i - k] == s[i + k])
            k ++;
        odd[i] = k --;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i ++) {
        int k =
            i > r ? 0
                  : min(even[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n &&
               s[i - k - 1] == s[i + k])
            k ++;
        even[i] = k --;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return { odd, even };
}
```

5.4 多模式匹配（AC 自动机）

功能：同时匹配多个模式串，统计每个模式串在文本中的出现次数。
约定：小写 a..z，非空模式；add 返回终点编号，重复模式共享终点。全部添加后 build 一次；count 返回各终点出现次数，可重复查询。
复杂度：$\mathcal{O}(26 \cdot$ 节点数 + 文本长度$)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 5267b3

```
struct AC {
    struct Node {
        array<int, 26> ch{};
        int fail = 0;
    };
    vector<Node> tr = { Node() };
    vector<int> order;
    int add(const string &s) {
        assert(!s.empty());
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!tr[u].ch[x]) {
                int v = tr.size();
                tr[u].ch[x] = v;
                tr.push_back(Node());
            }
            u = tr[u].ch[x];
        }
        return u;
    }
    void build() {
        order.clear();
        queue<int> q;
        for (int v : tr[0].ch)
            if (v) q.push(v);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int c = 0; c < 26; c ++) {
                int &v = tr[u].ch[c];
                if (v)
                    tr[v].fail = tr[tr[u].fail].ch[c],
                    q.push(v);
                else
                    v = tr[tr[u].fail].ch[c];
            }
        }
    }
    vector<i64> count(const string &s) const {
```

```
        vector<i64> cnt(tr.size());
        int u = 0;
        for (char c : s)
            u = tr[u].ch[c - 'a'], cnt[u] ++;
        for (int i = (int)order.size() - 1; i >= 0; i --)
            cnt[tr[order[i]].fail] += cnt[order[i]];
        return cnt;
    }
};
```

5.5 后缀数组＋ Height

功能：按字典序排序全部后缀，计算相邻后缀最长公共前缀。
约定：输入字节串不得含零字节；0-based。sa 大小 n+1，sa[0]=n 是空后缀；lcp[i]=LCP(sa[i-1],sa[i])。LCP 查询对 rank 之间的 lcp 做 RMQ。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1（递归收齐依赖）。

KACTL / Unknown / 独立编译 efda1e

```
struct SuffixArray {
    vector<int> sa, lcp;
    SuffixArray(string s,
                int lim = 256) { // or vector<int>
        s.push_back(0);
        int n = (int)(s).size(), k = 0, a, b;
        vector<int> x(n), y(n), ws(max(n, lim));
        for (int i = 0; i < n; i ++)
            x[i] = (unsigned char)s[i];
        sa = lcp = y, iota((sa).begin(), (sa).end(), 0);
        for (int j = 0, p = 0; p < n;
             j = max(1, j * 2), lim = p) {
            p = j, iota((y).begin(), (y).end(), n - j);
            for (int i = 0; i < n; i ++)
                if (sa[i] >= j) y[p ++] = sa[i] - j;
            fill((ws).begin(), (ws).end(), 0);
            for (int i = 0; i < n; i ++)
                ws[x[i]] ++;
            for (int i = 1; i < lim; i ++)
                ws[i] += ws[i - 1];
            for (int i = n; i --;)
                sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            for (int i = 1; i < n; i ++)
                a = sa[i - 1], b = sa[i],
                x[b] = (y[a] == y[b] &&
                        y[a + j] == y[b + j])
                           ? p - 1
                           : p ++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i ++]] = k)
            for (k &&k --, j = sa[x[i] - 1];
                 s[i + k] == s[j + k]; k ++)
                ;
    }
};
```

5.6 后缀 LCP 查询

功能：查询任意两个后缀的最长公共前缀长度。
约定：后缀起点 0..n，n 表示空后缀。返回两个后缀的 LCP；rank 包括空后缀。
复杂度：额外预处理 $\mathcal{O}(n \log n)$，查询 $\mathcal{O}(1)$
依赖：§1.1, §5.5, §2.4（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 fd9373

```
struct SuffixLCP {
    SuffixArray sa;
    ST st;
    vector<int> rank;
    SuffixLCP(const string &s)
        : sa(s), st(sa.lcp), rank(s.size() + 1) {
        for (int i = 0; i < (int)sa.sa.size(); i ++)
            rank[sa.sa[i]] = i;
    }
    int lcp(int i, int j) const {
        if (i == j) return (int)rank.size() - 1 - i;
        int l = rank[i], r = rank[j];
        if (l > r) swap(l, r);
        return st.query(l + 1, r);
    }
};
```

5.7 后缀自动机 / 子串出现次数

功能：构建表示所有子串的状态图，统计各状态子串的出现次数。
约定：小写 a..z；逐字符 add。count 在构造完成后仅调用一次；状态代表长度 (len[link],len] 的子串，共享 endpos 与出现次数。
复杂度：构造 / 计数 $\mathcal{O}(26n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 211011

```
struct SAM {
    struct Node {
        array<int, 26> ch;
        int len = 0, link = -1;
        i64 cnt = 0;
        Node() { ch.fill(-1); }
    };
    vector<Node> tr = { Node() };
    int last = 0;
    void add(char c) {
        int x = c - 'a', u = tr.size(), p = last;
        tr.push_back(Node());
        tr[u].len = tr[last].len + 1;
        tr[u].cnt = 1;
        while (p != -1 && tr[p].ch[x] == -1)
            tr[p].ch[x] = u, p = tr[p].link;
        if (p == -1)
            tr[u].link = 0;
        else {
            int q = tr[p].ch[x];
            if (tr[q].len == tr[p].len + 1)
                tr[u].link = q;
            else {
                int v = tr.size();
                tr.push_back(tr[q]);
                tr[v].len = tr[p].len + 1;
                tr[v].cnt = 0;
                while (p != -1 && tr[p].ch[x] == q)
                    tr[p].ch[x] = v, p = tr[p].link;
                tr[q].link = tr[u].link = v;
            }
        }
        last = u;
    }
    void count() {
        int n = tr[last].len;
        vector<int> cnt(n + 1), a(tr.size());
        for (auto u : tr)
            cnt[u.len] ++;
        for (int i = 1; i <= n; i ++)
            cnt[i] += cnt[i - 1];
        for (int i = 0; i < (int)tr.size(); i ++)
            a[--cnt[tr[i].len]] = i;
        for (int i = (int)a.size() - 1; i > 0; i --)
            tr[tr[a[i]].link].cnt += tr[a[i]].cnt;
    }
};
```

5.8 回文树 / 不同回文与出现次数

功能：在线维护不同回文子串，统计各回文的出现次数。
约定：小写 a..z；节点 0 是奇根，1 是偶根，其余对应不同回文。add 返回当前最长回文后缀节点；count 完成后调用一次。
复杂度：$\mathcal{O}(26n)$ 空间，线性构造
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 667fac

```
struct Eertree {
    struct Node {
        array<int, 26> ch{};
        int len = 0, link = 0;
```

```
        i64 cnt = 0;
    };
    vector<Node> tr = vector<Node>(2);
    vector<int> s = { -1 };
    int last = 1;
    Eertree() { tr[0].len = -1; }
    int get(int u) {
        int p = s.size() - 1;
        while (s[p - tr[u].len - 1] != s[p])
            u = tr[u].link;
        return u;
    }
    int add(char c) {
        int x = c - 'a';
        s.push_back(x);
        int p = get(last);
        if (!tr[p].ch[x]) {
            int u = tr.size();
            tr.push_back(Node());
            tr[u].len = tr[p].len + 2;
            tr[u].link =
                tr[u].len == 1
                    ? 1
                    : tr[get(tr[p].link)].ch[x];
            tr[p].ch[x] = u;
        }
        last = tr[p].ch[x];
        tr[last].cnt ++;
        return last;
    }
    void count() {
        for (int i = (int)tr.size() - 1; i >= 2; i --)
            tr[tr[i].link].cnt += tr[i].cnt;
    }
};
```

5.9 Lyndon 分解 / 字符串最小表示

功能：求 Lyndon 因子分解，以及字典序最小循环位移的起点。
约定：duval 返回各 Lyndon 因子的半开区间；min_rotation 返回字典序最小循环位移的起点，空串返回 0。按 char 比较，建议 ASCII 字符集。
复杂度：$\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 75c58a

```
vector<pair<int, int>> duval(const string &s) {
    int n = s.size();
    vector<pair<int, int>> ans;
    for (int i = 0; i < n;) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            k = s[k] < s[j] ? i : k + 1;
            j ++;
        }
        while (i <= k)
            ans.push_back({ i, i + j - k }), i += j - k;
    }
    return ans;
}
int min_rotation(const string &s) {
    int n = s.size();
    if (!n) return 0;
    string t = s + s;
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        if (t[i + k] == t[j + k]) {
            k ++;
            continue;
        }
        if (t[i + k] > t[j + k]) {
            i += k + 1;
            if (i <= j) i = j + 1;
        } else {
            j += k + 1;
            if (j <= i) j = i + 1;
        }
        k = 0;
    }
    return min(i, j);
}
```

5.10 最长公共子序列长度 / 恢复（LCS）

功能：用位运算求最长公共子序列长度，用 Hirschberg 恢复一条子序列。
约定：任意字节串；bitset_lcs 只求长度，lcs_restore 另作恢复。可将较短串作为 b 以减少空间；unsigned 计算跨字借位。lcs_restore(a, b) 返回一条 LCS；使用 string_view、不限制字节字母表。例：lcs_restore("abc","ac") 得"ac"。
复杂度：$\mathcal{O}(|a|\,\mathrm{ceil}(|b|/64))$，空间 $\mathcal{O}(256\,\mathrm{ceil}(|b|/64))$；恢复 $\mathcal{O}(nm)$ 时间、$\mathcal{O}(m+\log n)$ 额外空间（另计结果字符串）
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 0315b4

```
string lcs_restore(string_view a, string_view b) {
    if (a.empty() || b.empty()) return "";
    if (a.size() == 1)
        return b.find(a[0]) == string_view::npos
                   ? ""
                   : string(1, a[0]);
    int mid = a.size() / 2, m = b.size(), k = 0;
    {
        auto row = [&](string_view x, bool rev) {
            vector<int> d(m + 1);
            for (int i = 0; i < (int)x.size(); i ++) {
                int pre = 0;
                for (int j = 1; j <= m; j ++) {
                    int old = d[j];
                    char u =
                        x[rev ? x.size() - 1 - i : i];
                    char v = b[rev ? m - j : j - 1];
                    d[j] = u == v ? pre + 1
                                  : max(d[j], d[j - 1]);
                    pre = old;
                }
            }
            return d;
        };
        auto l = row(a.substr(0, mid), false);
        auto r = row(a.substr(mid), true);
        for (int j = 1; j <= m; j ++)
            if (l[j] + r[m - j] > l[k] + r[m - k])
                k = j;
    }
    return lcs_restore(a.substr(0, mid),
                       b.substr(0, k)) +
           lcs_restore(a.substr(mid), b.substr(k));
}

int bitset_lcs(const string &a, const string &b) {
    int n = b.size(), m = (n + 63) / 64;
    vector<vector<u64>> mask(256, vector<u64>(m));
    for (int i = 0; i < n; i ++)
        mask[(unsigned char)b[i]][i / 64] |=
            1ULL << (i % 64);
    vector<u64> s(m);
    for (unsigned char c : a) {
        u64 carry = 1, borrow = 0;
        for (int j = 0; j < m; j ++) {
            u64 x = s[j] | mask[c][j];
            u64 y = (s[j] << 1) | carry;
            carry = s[j] >> 63;
            u64 z = x - y - borrow;
            borrow = x < y || (borrow && x == y);
            s[j] = x & ~z;
        }
    }
    int ans = 0;
    for (u64 x : s)
        ans += __builtin_popcountll(x);
    return ans;
}
```

6 数论与组合数学

6.1 模运算 / 扩欧 / 扩展 CRT

功能：提供模运算、扩展欧几里得、模逆与非互质同余方程合并。
约定：modulus 正数；CRT 合并 x=a (mod m)、x=b (mod n)，失败 nullopt，返回最小非负解与 lcm；要求 lcm<=LLONG_MAX。inverse 不存在返回-1。
复杂度：$\mathcal{O}(\log \text{modulus})$；乘法 $\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 902701

```
u64 mul_mod(u64 a, u64 b, u64 m) {
    return u128(a) * b % m;
}
u64 pow_mod(u64 a, u64 b, u64 m) {
    u64 ans = 1 % m;
    for (; b; b >>= 1, a = mul_mod(a, a, m))
        if (b & 1) ans = mul_mod(ans, a, m);
    return ans;
}
i128 exgcd(i128 a, i128 b, i128 &x, i128 &y) {
    if (!b) {
        x = 1;
        y = 0;
        return a;
    }
    i128 g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
i64 inverse(i64 a, i64 m) {
    i128 x, y;
    i128 g = exgcd((i128(a) % m + m) % m, m, x, y);
    if (g != 1) return -1;
    return (x % m + m) % m;
}
optional<pair<i64, i64>> crt(i64 a, i64 m, i64 b,
                             i64 n) {
    assert(m > 0 && n > 0);
    a = (i128(a) % m + m) % m;
    b = (i128(b) % n + n) % n;
    i128 x, y, g = exgcd(m, n, x, y), d = i128(b) - a;
    if (d % g) return nullopt;
    i128 mod = i128(m) / g * n;
    assert(mod <= LLONG_MAX);
    i128 k = (d / g * x % (n / g) + n / g) % (n / g);
    i128 v = (a + i128(m) * k) % mod;
    return pair<i64, i64>{ (i64)v, (i64)mod };
}
```

6.2 线性筛 / 质因子 / 欧拉与莫比乌斯函数

功能：线性预处理质数、最小质因子、欧拉函数及莫比乌斯函数。
约定：n>=0；lp 最小质因子，phi 欧拉函数，mu 莫比乌斯函数。反复除 lp 可分解 n 以内整数。
复杂度：$\mathcal{O}(n)$，空间 $\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 0cbf94

```
struct Sieve {
    vector<int> primes, lp, phi, mu;
    Sieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1) {
        if (n >= 1) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i ++) {
            if (!lp[i])
                lp[i] = i, primes.push_back(i),
                phi[i] = i - 1, mu[i] = -1;
            for (int p : primes) {
                if (p > n / i) break;
                lp[i * p] = p;
                if (i % p == 0) {
                    phi[i * p] = phi[i] * p;
                    break;
                }
                phi[i * p] = phi[i] * (p - 1);
                mu[i * p] = -mu[i];
            }
        }
    }
};
```

6.3 64 位判素 / 质因数分解

功能：用 Miller-Rabin 判素，用 Pollard-Rho 分解 64 位无符号整数。
约定：支持完整 u64 范围，factor 要求 n>=1，输出未排序的质因子列表；判素基底对 64 位确定正确，Rho 耗时随机。
复杂度：MR $\mathcal{O}(\log n)$；Rho 期望约 $\mathcal{O}(n^{1/4})$
依赖：§1.1, §6.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 1fa72a

```
bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p :
         { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37 })
        if (n % p == 0) return n == p;
    u64 d = n - 1;
    int s = __builtin_ctzll(d);
    d >>= s;
    for (u64 a :
         { 2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL,
           9780504ULL, 1795265022ULL }) {
        if (a % n == 0) continue;
        u64 x = pow_mod(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (int i = 1; i < s; i ++) {
            x = mul_mod(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
        if (!ok) return false;
    }
    return true;
}
u64 rho(u64 n) {
    if (n % 2 == 0) return 2;
    static mt19937_64 rng(712367);
    while (true) {
        u64 c = rng() % (n - 1) + 1,
            x = rng() % (n - 2) + 2, y = x, d = 1;
        auto f = [&](u64 v) {
            return u64((u128(mul_mod(v, v, n)) + c) %
                       n);
        };
        while (d == 1) {
            x = f(x);
            y = f(f(y));
            d = gcd(x > y ? x - y : y - x, n);
        }
        if (d != n) return d;
    }
}
void factor(u64 n, vector<u64> &a) {
    if (n == 1) return;
    if (is_prime(n)) {
        a.push_back(n);
        return;
    }
    u64 p = rho(n);
    factor(p, a);
    factor(n / p, a);
}
```

6.4 离散对数（BSGS / 扩展 BSGS）

功能：求模指数方程的最小非负指数，支持底数与模数不互质。
约定：求最小 $x \ge 0$ 使 $a^x \equiv b \pmod m$，m<=LLONG_MAX；支持不互质，0^0 按 1 处理。$\sqrt{m}$ 内存是实际限制。
复杂度：期望 $\mathcal{O}(\sqrt{m})$ 时间 / 空间
依赖：§1.1, §6.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 6a1444

```
optional<u64> bsgs(u64 a, u64 b, u64 m) {
    assert(m >= 1 && m <= LLONG_MAX);
    a %= m;
    b %= m;
    if (m == 1 || b == 1) return 0;
    u64 cnt = 0, k = 1, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k) return cnt;
        if (b % g) return nullopt;
        b /= g;
        m /= g;
        cnt ++;
        k = mul_mod(k, a / g, m);
        if (m == 1) return cnt;
    }
    b = mul_mod(b, inverse(k, m), m);
    u64 s = sqrt((long double)m);
    while (u128(s) * s < m)
        s ++;
    unordered_map<u64, u64> mp;
    u64 v = 1;
    for (u64 j = 0; j < s; j ++) {
        mp.emplace(v, j);
        v = mul_mod(v, a, m);
    }
    u64 step = inverse(v, m);
    for (u64 i = 0; i <= s; i ++) {
        if (mp.count(b)) return i * s + mp[b] + cnt;
        b = mul_mod(b, step, m);
    }
    return nullopt;
}
```

6.5 原根与模素数平方根

功能：求素数模的原根，以及模素数二次剩余的一个平方根。
约定：p 为素数；Tonelli-Shanks 支持 p<=LLONG_MAX。返回任一根或 nullopt，另一个根为 (p-x)%p；原根先分解 p-1。
复杂度：平方根 $\mathcal{O}(\log^2 p)$，原根依赖分解
依赖：§1.1, §6.3（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 96c607

```
u64 primitive_root(u64 p) {
    assert(is_prime(p));
    if (p == 2) return 1;
    vector<u64> a;
    factor(p - 1, a);
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    for (u64 g = 2;; g ++) {
        bool ok = true;
        for (u64 q : a)
            if (pow_mod(g, (p - 1) / q, p) == 1) {
                ok = false;
                break;
            }
        if (ok) return g;
    }
}
optional<u64> mod_sqrt(u64 a, u64 p) {
    assert(p <= LLONG_MAX && is_prime(p));
    a %= p;
    if (!a || p == 2) return a;
    if (pow_mod(a, (p - 1) / 2, p) != 1) return nullopt;
    if (p % 4 == 3) return pow_mod(a, (p + 1) / 4, p);
    u64 q = p - 1, z = 2;
    int s = __builtin_ctzll(q);
    q >>= s;
    while (pow_mod(z, (p - 1) / 2, p) == 1)
        z ++;
    u64 c = pow_mod(z, q, p),
        x = pow_mod(a, (q + 1) / 2, p),
        t = pow_mod(a, q, p);
    while (t != 1) {
        int i = 0;
        u64 v = t;
        while (v != 1)
            v = mul_mod(v, v, p), i ++;
        u64 b = pow_mod(c, 1ULL << (s - i - 1), p);
        x = mul_mod(x, b, p);
        c = mul_mod(b, b, p);
        t = mul_mod(t, c, p);
        s = i;
    }
    return x;
}
```

6.6 组合数取模（Lucas / 扩展 Lucas）

功能：计算大 n 的组合数模素数或合数，可复用质数幂预处理。
约定：扩展 Lucas：分解模数到质数幂、剔除 p 因子阶乘，再 CRT。每个 p^e 都须能建立 $\mathcal{O}(p^e)$ 表，建议 <=2e6；n<=LLONG_MAX。素数情形即 Lucas，可复用预处理对象。
复杂度：预处理 $\mathcal{O}(sump^e)$，每次 $\mathcal{O}(\log n \log \mathrm{mod})$，另加分解
依赖：§1.1, §6.3（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 60e379

```
struct PrimePowerBinomial {
    i64 p, mod;
    vector<i64> fac;
    PrimePowerBinomial(i64 p, i64 pk)
        : p(p), mod(pk), fac(pk + 1, 1) {
        for (i64 i = 1; i <= mod; i ++)
            fac[i] = i % p ? mul_mod(fac[i - 1], i, mod)
                           : fac[i - 1];
    }
    i64 exponent(u64 n) const {
        i64 ans = 0;
        while (n)
            n /= p, ans += n;
        return ans;
    }
    i64 fact(u64 n) const {
        if (!n) return 1;
        return mul_mod(
            mul_mod(pow_mod(fac[mod], n / mod, mod),
                    fac[n % mod], mod),
            fact(n / p), mod);
    }
    i64 choose(u64 n, u64 k) const {
        if (k > n) return 0;
        i64 e =
            exponent(n) - exponent(k) - exponent(n - k);
        u64 v = mul_mod(fact(n), inverse(fact(k), mod),
                        mod);
        v = mul_mod(v, inverse(fact(n - k), mod), mod);
        return mul_mod(v, pow_mod(p, e, mod), mod);
    }
};
i64 binomial_mod(u64 n, u64 k, i64 mod) {
    assert(mod > 0);
    if (mod == 1 || k > n) return 0;
    vector<u64> a;
    factor(mod, a);
    sort(a.begin(), a.end());
    i64 ans = 0, m = 1;
    for (int i = 0; i < (int)a.size();) {
        i64 p = a[i], pk = 1;
        while (i < (int)a.size() && a[i] == (u64)p)
            pk *= p, i ++;
        i64 v = PrimePowerBinomial(p, pk).choose(n, k);
        auto z = crt(ans, m, v, pk);
        tie(ans, m) = *z;
    }
    return ans;
}
```

6.7 整除取整求和（类欧几里得）

功能：求线性函数除以模数后向下取整的前 n 项之和。
约定：求 $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$，n>=0、m>0；支持负 a,b，结果 i128，要求最终结果在范围内。
复杂度：$\mathcal{O}(\log m)$
依赖：§1.1, §1.3（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 94f866

```cpp
i128 floor_sum(i64 n, i64 m, i64 a, i64 b) {
    assert(n >= 0 && m > 0);
    i64 qa = floor_div(a, m), qb = floor_div(b, m);
    i128 ans =
        i128(n) * (n - 1) / 2 * qa + i128(n) * qb;
    a = i128(a) - i128(qa) * m;
    b = i128(b) - i128(qb) * m;
    while (true) {
        if (a >= m)
            ans += i128(n) * (n - 1) / 2 * (a / m),
                a %= m;
        if (b >= m) ans += i128(n) * (b / m), b %= m;
        i128 y = i128(a) * n + b;
        if (y < m) return ans;
        n = y / m;
        b = y % m;
        swap(m, a);
    }
}
```

6.8 杜教筛 phi / mu 前缀和

功能：求大范围欧拉函数之和与莫比乌斯函数之和。
约定：n>=0 且 n<LLONG_MAX；建议 lim 约 $n^{2/3}$，根据内存调整。相同对象复用缓存；phi 前缀返回 i128，mu 前缀返回 i64。
复杂度：标准 cutoff 下约 $\mathcal{O}(n^{2/3})$，map 另有对数因子
依赖：§1.1, §6.2（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 5e05a5

```cpp
struct DuSieve {
    int lim;
    vector<i64> phi, mu;
    map<i64, i128> mp;
    map<i64, i64> mm;
    DuSieve(int lim)
        : lim(lim), phi(lim + 1), mu(lim + 1) {
        Sieve s(lim);
        for (int i = 1; i <= lim; i ++)
            phi[i] = phi[i - 1] + s.phi[i],
            mu[i] = mu[i - 1] + s.mu[i];
    }
    i128 sum_phi(i64 n) {
        if (n <= lim) return phi[n];
        if (mp.count(n)) return mp[n];
        i128 ans = i128(n) * (n + 1) / 2;
        for (i64 l = 2, r; l <= n; l = r + 1) {
            r = n / (n / l);
            ans -= i128(r - l + 1) * sum_phi(n / l);
        }
        return mp[n] = ans;
    }
    i64 sum_mu(i64 n) {
        if (n <= lim) return mu[n];
        if (mm.count(n)) return mm[n];
        i64 ans = 1;
        for (i64 l = 2, r; l <= n; l = r + 1) {
            r = n / (n / l);
            ans -= (r - l + 1) * sum_mu(n / l);
        }
        return mm[n] = ans;
    }
};
```

6.9 积性函数求和（Min_25 筛）

功能：统计素数个数 / 和，并求指定形式积性函数的前缀和。
约定：n>=1；cnt[0] 与 sum[0] 是素数个数 / 和。multiplicative_sum 要求 $f(1) = 1$、积性、$f(p) = ap + b$，并传入 `f(p,e)` $= f(p^e)$。仅对构造时同一个 n 的整除分块值调用 id；i128 结果不溢出。
复杂度：典型 $\mathcal{O}(n^{3/4}/\log n)$ 筛阶段，递归依赖 f；空间 $\mathcal{O}(\sqrt{n})$
依赖：§1.1, §6.2（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 d24bbb

```cpp
struct Min25 {
    i64 n, sq;
    vector<i64> w, cnt;
    vector<i128> sum, pref;
    vector<int> lo, hi, pr;
    Min25(i64 n)
        : n(n), sq(sqrt((long double)n)), lo(sq + 2),
          hi(sq + 2) {
        assert(n >= 1);
        while ((sq + 1) <= n / (sq + 1))
            sq ++;
        while (sq > n / sq)
            sq --;
        lo.resize(sq + 2);
        hi.resize(sq + 2);
        for (i64 l = 1, r; l <= n; l = r + 1) {
            r = n / (n / l);
            i64 x = n / l;
            int id = w.size();
            w.push_back(x);
            cnt.push_back(x - 1);
            sum.push_back(i128(x) * (x + 1) / 2 - 1);
            if (x <= sq)
                lo[x] = id;
            else
                hi[n / x] = id;
        }
        Sieve s(sq);
        pr = s.primes;
        pref = { 0 };
        for (int i = 0; i < (int)pr.size(); i ++) {
            i64 p = pr[i];
            pref.push_back(pref.back() + p);
            for (int j = 0;
                 j < (int)w.size() && p <= w[j] / p;
                 j ++) {
                int k = id(w[j] / p);
                cnt[j] -= cnt[k] - i;
                sum[j] -= i128(p) * (sum[k] - pref[i]);
            }
        }
    }
    int id(i64 x) const {
        return x <= sq ? lo[x] : hi[n / x];
    }
    template <class F>
    i128 rec(i64 x, int k, i64 a, i64 b, F &f) const {
        if (x < 2 || (k && x <= pr[k - 1])) return 0;
        int z = id(x);
        i128 ans = a * (sum[z] - pref[k]) +
                   i128(b) * (cnt[z] - k);
        for (int i = k;
             i < (int)pr.size() && pr[i] <= x / pr[i];
             i ++) {
            i64 p = pr[i], pe = p;
            for (int e = 1; pe <= x / p; e ++, pe *= p)
                ans += f(p, e) *
                           rec(x / pe, i + 1, a, b, f) +
                       f(p, e + 1);
        }
        return ans;
    }
    template <class F>
    i128 multiplicative_sum(i64 a, i64 b, F f) const {
        return 1 + rec(n, 0, a, b, f);
    }
};
```

6.10 整除分块与反演速查

整除分块：从 $l = 1$ 开始，令 $r = \lfloor n/\lfloor n/l \rfloor \rfloor$，则 $[l, r]$ 内 $\lfloor n/i \rfloor$ 不变；下一段从 $r + 1$ 开始。

双变量：令 $a = \lfloor n/l \rfloor$、$b = \lfloor m/l \rfloor$，区间右端点为

$$r = \min\left(\left\lfloor \frac{n}{a} \right\rfloor, \left\lfloor \frac{m}{b} \right\rfloor\right),$$

循环到 $\min(n, m)$。

常用恒等式：

$$\sum_{d \mid n} \varphi(d) = n, \qquad \sum_{d \mid n} \mu(d) = [n = 1],$$

$$\sum_{d \mid n} \mu(d) \frac{n}{d} = \varphi(n).$$

Möbius 反演：

$$F(n)=\sum_{d\mid n} f(d) \quad\Longleftrightarrow\quad f(n)=\sum_{d\mid n}\mu(d)F\left(\frac{n}{d}\right).$$

互质对：$1\le x\le n$、$1\le y\le m$ 且 $\gcd(x,y)=1$ 的数量为

$$\sum_{d=1}^{\min(n,m)}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor.$$

GCD 权值：$*$ 表示 Dirichlet 卷积，

$$h(\gcd(x,y))=\sum_{\substack{d\mid x\\ d\mid y}}(h*\mu)(d).$$

阶乘质因子指数：

$$v_p(n!)=\sum_{k\ge 1}\left\lfloor\frac{n}{p^k}\right\rfloor.$$

欧拉降幂：仅在 $\gcd(a,m)=1$ 时直接使用

$$a^b\equiv a^{b \bmod \varphi(m)} \pmod m.$$

不互质时不能无条件套用。

7 多项式与线性代数

7.1 多项式乘法 / 卷积（NTT）

功能：在模 998244353 下作数论变换，快速计算多项式乘法。
约定：模数 998244353、原根 3；系数从常数项开始，归一到 [0,mod)。NTT 长度为 2 幂且 $\le 2^{23}$；结果卷积长度也需满足变换上限。
复杂度：$\mathcal{O}(n\log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon（参考 cp-book）/ CC0-1.0 / 本地回归 / 对拍 bd8f6e

```
namespace poly {
const int mod = 998244353;
using Vec = vector<int>;
int power(int a, i64 b) {
    int ans = 1;
    for (; b; b >>= 1, a = i64(a) * a % mod)
        if (b & 1) ans = i64(ans) * a % mod;
    return ans;
}
void ntt(Vec &a, bool inv) {
    int n = a.size();
    assert(n && (n & (n - 1)) == 0 && n <= (1 << 23));
    for (int i = 1, j = 0; i < n; i ++) {
        int b = n >> 1;
        for (; j & b; b >>= 1)
            j ^= b;
        j ^= b;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        int w0 = power(3, (mod - 1) / len);
        if (inv) w0 = power(w0, mod - 2);
        for (int l = 0; l < n; l += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j ++) {
                int u = a[l + j],
                    v = i64(a[l + j + len / 2]) * w %
                        mod;
                a[l + j] = (u + v) % mod;
                a[l + j + len / 2] =
                    (u - v + mod) % mod;
                w = i64(w) * w0 % mod;
            }
        }
    }
    if (inv) {
        int z = power(n, mod - 2);
        for (int &x : a)
            x = i64(x) * z % mod;
    }
}
Vec conv(Vec a, Vec b) {
    if (a.empty() || b.empty()) return {};
    int m = a.size() + b.size() - 1, n = 1;
    while (n < m)
        n <<= 1;
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i ++)
        a[i] = i64(a[i]) * b[i] % mod;
    ntt(a, true);
    a.resize(m);
    return a;
}
} // namespace poly
```

7.2 任意模数卷积

功能：使用三模 NTT 与 CRT，计算指定正整数模数下的卷积。
约定：mod 为正 int；输入系数在 [0,mod)。三模 CRT 恢复整数后取模，要求原始每项卷积系数 < 三模乘积；变换长度 $\le 2^{24}$。常见长度与 32 位模数可满足此界，仍需检查。
复杂度：$\mathcal{O}(n\log n)$
依赖：§1.1, §6.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 c86273

```
template <int P, int G>
vector<int> ntt_conv(vector<int> a, vector<int> b) {
    auto pw = [](i64 x, int e) {
        i64 z = 1;
        for (; e; e >>= 1, x = x * x % P)
            if (e & 1) z = z * x % P;
        return int(z);
    };
    int m = a.size() + b.size() - 1, n = 1;
    while (n < m)
        n <<= 1;
    assert((P - 1) % n == 0);
    a.resize(n);
    b.resize(n);
    auto transform = [&](vector<int> &v, bool inv) {
        for (int i = 1, j = 0; i < n; i ++) {
            int k = n >> 1;
            for (; j & k; k >>= 1)
                j ^= k;
            j ^= k;
            if (i < j) swap(v[i], v[j]);
        }
        for (int k = 2; k <= n; k <<= 1) {
            int z = pw(G, (P - 1) / k);
            if (inv) z = pw(z, P - 2);
            for (int l = 0; l < n; l += k)
                for (int j = 0, w = 1; j < k / 2;
                     j ++, w = i64(w) * z % P) {
                    int x = v[l + j],
                        y = i64(v[l + j + k / 2]) * w %
                            P;
                    v[l + j] = (i64(x) + y) % P;
                    v[l + j + k / 2] =
                        (i64(x) - y + P) % P;
                }
        }
        if (inv) {
            int z = pw(n, P - 2);
            for (int &x : v)
                x = i64(x) * z % P;
        }
    };
    for (int &x : a)
        x %= P;
    for (int &x : b)
        x %= P;
    transform(a, false);
    transform(b, false);
    for (int i = 0; i < n; i ++)
        a[i] = i64(a[i]) * b[i] % P;
    transform(a, true);
```

```
    a.resize(m);
    return a;
}
vector<int> conv_mod(const vector<int> &a,
                     const vector<int> &b, int mod) {
    if (a.empty() || b.empty()) return {};
    const i64 p = 167772161, q = 469762049,
              r = 1224736769;
    auto x = ntt_conv<p, 3>(a, b),
         y = ntt_conv<q, 3>(a, b),
         z = ntt_conv<r, 3>(a, b);
    i64 ip = inverse(p, q), ipq = inverse(p * q % r, r);
    vector<int> ans(x.size());
    for (int i = 0; i < (int)x.size(); i ++) {
        i64 t = (y[i] - x[i] + q) % q * ip % q;
        i128 v = x[i] + i128(p) * t;
        i64 u = (z[i] - (i64)(v % r) + r) % r * ipq % r;
        ans[i] = (v + i128(p) * q * u) % mod;
    }
    return ans;
}
```

7.3 形式幂级数求逆 / ln / exp / 幂

功能：提供求导、积分、乘法逆、对数、指数与非负整数次幂。
约定：模 998244353、n>=1；inv 要求常数项非零；ln 常数项 1，exp 常数项 0，pow 指数非负。全部返回模 x^n 结果；精度必须小于模数且所有中间卷积满足 NTT 长度界。
复杂度：$\mathcal{O}(n \log n)$（当前 ln/exp Newton 实现常数较大）
依赖：§1.1, §7.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 d45b1d

```
namespace poly {
Vec cut(Vec a, int n) {
    a.resize(n);
    return a;
}
Vec derivative(const Vec &a) {
    Vec b(max(0, (int)a.size() - 1));
    for (int i = 1; i < (int)a.size(); i ++)
        b[i - 1] = i64(a[i]) * i % mod;
    return b;
}
Vec integral(const Vec &a) {
    Vec b(a.size() + 1), inv(a.size() + 1);
    if (a.size()) inv[1] = 1;
    for (int i = 2; i <= (int)a.size(); i ++)
        inv[i] =
            mod - i64(mod / i) * inv[mod % i] % mod;
    for (int i = 0; i < (int)a.size(); i ++)
        b[i + 1] = i64(a[i]) * inv[i + 1] % mod;
    return b;
}
Vec inverse(const Vec &a, int n) {
    assert(n >= 1 && !a.empty() && a[0]);
    Vec b = { power(a[0], mod - 2) };
    for (int m = 2; m / 2 < n; m *= 2) {
        int k = min(m, n);
        Vec c = conv(cut(a, k), b);
        c.resize(k);
        for (int &x : c)
            x = x ? mod - x : 0;
        c[0] = (c[0] + 2) % mod;
        b = cut(conv(b, c), k);
    }
    return cut(b, n);
}
Vec logarithm(const Vec &a, int n) {
    assert(!a.empty() && a[0] == 1 && n >= 1);
    if (n == 1) return { 0 };
    return cut(
        integral(cut(conv(derivative(a), inverse(a, n)),
                     n - 1)),
        n);
}
Vec exponential(const Vec &a, int n) {
    assert(n >= 1 && (a.empty() || a[0] == 0));
    Vec b = { 1 };
    for (int m = 2; m / 2 < n; m *= 2) {
        int k = min(m, n);
        Vec c = cut(a, k), d = logarithm(b, k);
        for (int i = 0; i < k; i ++)
            c[i] = (c[i] - d[i] + mod) % mod;
        c[0] ++;
        b = cut(conv(b, c), k);
    }
    return cut(b, n);
}
Vec fps_power(const Vec &a, u64 e, int n) {
    assert(n >= 1);
    if (!e) {
        Vec b(n);
        b[0] = 1;
        return b;
    }
    int l = 0;
    while (l < (int)a.size() && !a[l])
        l ++;
    if (l == (int)a.size() || u128(l) * e >= (u64)n)
        return Vec(n);
    int shift = u128(l) * e, k = n - shift;
    Vec b(a.begin() + l, a.end());
    int iv = power(b[0], mod - 2),
        c = power(b[0], e % (mod - 1));
    for (int &x : b)
        x = i64(x) * iv % mod;
    b = logarithm(b, k);
    for (int &x : b)
        x = i64(x) * (e % mod) % mod;
    b = exponential(b, k);
    for (int &x : b)
        x = i64(x) * c % mod;
    b.insert(b.begin(), shift, 0);
    return b;
}
} // namespace poly
```

7.4 形式幂级数平方根

功能：计算模素数形式幂级数的一支平方根，并判断是否存在。
约定：模 998244353，支持前导零；不存在平方根返回 nullopt，返回任意一支模 x^n 根。依赖 Tonelli-Shanks；除移位后的可确定部分外，高次项按零延拓计算。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1, §7.3, §6.5（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 695887

```
namespace poly {
optional<Vec> fps_sqrt(const Vec &a, int n) {
    assert(n >= 1);
    int l = 0;
    while (l < (int)a.size() && !a[l])
        l ++;
    if (l == (int)a.size() || l >= n) return Vec(n);
    if (l & 1) return nullopt;
    auto z = mod_sqrt(a[l], mod);
    if (!z) return nullopt;
    int shift = l / 2, k = n - shift;
    Vec c(a.begin() + l, a.end()), b = { int(*z) };
    for (int m = 2; m / 2 < k; m *= 2) {
        int t = min(m, k);
        Vec d = cut(conv(cut(c, t), inverse(b, t)), t);
        b.resize(t);
        for (int i = 0; i < t; i ++)
            b[i] = i64((b[i] + d[i]) % mod) *
                   ((mod + 1) / 2) % mod;
    }
    b = cut(b, k);
    b.insert(b.begin(), shift, 0);
    return b;
}
} // namespace poly
```

7.5 多项式除法 / 取模 / 多点求值 / 插值

功能：提供多项式除法 / 余数、批量点值求值及点值插值。
约定：系数归一，低次在前；除数非零。ProductTree 支持空点集，求值可重复点，插值要求点两两不同。所有中间 NTT 长度满足上限。
复杂度：除法 $\mathcal{O}(n \log n)$；多点操作 $\mathcal{O}(n \log^2 n)$
依赖：§1.1, §7.3（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 8dd872

```
namespace poly {
void trim(Vec &a) {
    while (!a.empty() && !a.back())
        a.pop_back();
}
pair<Vec, Vec> divmod(Vec a, Vec b) {
    trim(a);
    trim(b);
    assert(!b.empty());
    if (a.size() < b.size()) return { {}, a };
    int k = a.size() - b.size() + 1;
    Vec x = a, y = b;
    reverse(x.begin(), x.end());
    reverse(y.begin(), y.end());
    Vec q = cut(conv(cut(x, k), inverse(y, k)), k);
    reverse(q.begin(), q.end());
    Vec t = conv(b, q);
    a.resize(b.size() - 1);
    for (int i = 0; i < (int)a.size(); i ++)
        a[i] = (a[i] - t[i] + mod) % mod;
    trim(a);
    return { q, a };
}
struct ProductTree {
    int n;
    vector<Vec> tr;
    Vec xs;
    ProductTree(const Vec &x)
        : n(x.size()), tr(4 * n + 4), xs(x) {
        if (n) build(1, 0, n);
    }
    void build(int p, int l, int r) {
        if (r - l == 1) {
            tr[p] = { (mod - xs[l]) % mod, 1 };
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m);
        build(p * 2 + 1, m, r);
        tr[p] = conv(tr[p * 2], tr[p * 2 + 1]);
    }
    void eval(int p, int l, int r, const Vec &a,
              Vec &ans) const {
        Vec b = divmod(a, tr[p]).second;
        if (r - l == 1) {
            ans[l] = b.empty() ? 0 : b[0];
            return;
        }
        int m = (l + r) / 2;
        eval(p * 2, l, m, b, ans);
        eval(p * 2 + 1, m, r, b, ans);
    }
    Vec evaluate(const Vec &a) const {
        Vec ans(n);
        if (n) eval(1, 0, n, a, ans);
        return ans;
    }
    Vec combine(int p, int l, int r,
                const Vec &v) const {
        if (r - l == 1) return { v[l] };
        int m = (l + r) / 2;
        Vec a = conv(combine(p * 2, l, m, v),
                     tr[p * 2 + 1]);
        Vec b = conv(combine(p * 2 + 1, m, r, v),
                     tr[p * 2]);
        a.resize(max(a.size(), b.size()));
        for (int i = 0; i < (int)b.size(); i ++)
            a[i] = (a[i] + b[i]) % mod;
        return a;
    }
    Vec interpolate(const Vec &ys) const {
        assert(ys.size() == xs.size());
        if (!n) return {};
        Vec d = evaluate(derivative(tr[1])), v(n);
        for (int i = 0; i < n; i ++) {
            assert(d[i]);
            v[i] =
                i64(ys[i]) * power(d[i], mod - 2) % mod;
        }
        return cut(combine(1, 0, n, v), n);
    }
};
} // namespace poly
```

7.6 连续点值插值

功能：由连续整数点上的值，求低次多项式在任意点的值。
约定：给定次数 <=n 的多项式在 0..n 的点值，求 x 处值；模 998244353，n<mod，输入点值归一。
复杂度：$\mathcal{O}(n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 fd30e5

```
int interpolate_consecutive(const vector<int> &y,
                            i64 x) {
    const int mod = 998244353;
    auto pw = [&](i64 a, int b) {
        i64 z = 1;
        for (; b; b >>= 1, a = a * a % mod)
            if (b & 1) z = z * a % mod;
        return int(z);
    };
    int n = y.size() - 1;
    assert(n >= 0 && n < mod);
    x = (x % mod + mod) % mod;
    if (x <= n) return y[x];
    vector<i64> pre(n + 2, 1), suf(n + 2, 1),
        fac(n + 1, 1), inv(n + 1);
    for (int i = 0; i <= n; i ++)
        pre[i + 1] = pre[i] * (x - i + mod) % mod;
    for (int i = n; i >= 0; i --)
        suf[i] = suf[i + 1] * (x - i + mod) % mod;
    for (int i = 1; i <= n; i ++)
        fac[i] = fac[i - 1] * i % mod;
    inv[n] = pw(fac[n], mod - 2);
    for (int i = n; i; i --)
        inv[i - 1] = inv[i] * i % mod;
    i64 ans = 0;
    for (int i = 0; i <= n; i ++) {
        i64 v = i64(y[i]) * pre[i] % mod * suf[i + 1] %
                mod * inv[i] % mod * inv[n - i] % mod;
        ans = (ans + ((n - i) & 1 ? mod - v : v)) % mod;
    }
    return ans;
}
```

7.7 FWT / FMT 公式与正交补

约定：下标是 k 位集合，$N = 2^k$，$|S|$ 表示置位数，$\langle S, T\rangle = |S \cap T| \bmod 2$。Transforms.h 的系数模 998244353，输入长度必须为非零的二次幂，系数先归一化到 $[0, 998244353)$。这里 FMT 指子集 Zeta / Möbius 变换，不同资料的命名可能不同。

XOR 变换（Walsh–Hadamard）：令 $\hat{a} = H(a)$，则

$$\hat{a}[S] = \sum_{T \subseteq U} (-1)^{\langle S,T\rangle} a[T], \qquad U = \{0, \dots, k-1\}.$$

$$a[S] = N^{-1} \sum_{T \subseteq U} (-1)^{\langle S,T\rangle} \hat{a}[T].$$

其中 $H^2 = NI$。每层把当前位为 0、1 的一对值 (u, v) 变成 $(u+v, u-v)$；逆变换可每层乘 2^{-1}，也可像代码一样重复正变换后统一乘 N^{-1}，两种归一化不能同时做。接口：`fwt(a,0)` / `fwt(a,0,true)`。XOR 逆变换要求 2 可逆，本模板的奇素数模满足条件。

OR 变换 / 子集 FMT：正变换是子集 Zeta，逆变换是子集 Möbius。

$$\hat{a}[S] = \sum_{T \subseteq S} a[T], \qquad a[S] = \sum_{T \subseteq S} (-1)^{|S|-|T|} \hat{a}[T].$$

每层正变换 $(u,v)\mapsto(u,u+v)$，逆变换 $(u,v)\mapsto(u,v-u)$，不需要除以 N。接口：`fwt(a,1)` 与 `sos(a)` 等价；`fwt(a,1,true)` 与 `sos(a,true)` 等价。也就是枚举位 b，对含 b 的 S 执行 $a[S]\mathrel{+}=a[S\setminus\{b\}]$，反演时改为减法。

AND 变换 / 超集 FMT：正变换是超集 Zeta，逆变换是超集 Möbius。

$$\hat a[S]=\sum_{T\supseteq S}a[T],\qquad a[S]=\sum_{T\supseteq S}(-1)^{|T|-|S|}\hat a[T].$$

每层正变换 $(u,v)\mapsto(u+v,v)$，逆变换 $(u,v)\mapsto(u-v,v)$，不需要归一化。接口：`fwt(a,2)` / `fwt(a,2,true)`。所有蝶形公式中的 u,v 均为更新前的值；减法在模意义下处理。

位运算卷积：取 $\circ\in\{\oplus,\cup,\cap\}$，分别对应 XOR、OR、AND。若 $\mathcal{T}$ 是对应的变换，则

$$c[S]=\sum_{A\circ B=S}a[A]b[B],\qquad c=\mathcal{T}^{-1}(\mathcal{T}(a)\cdot\mathcal{T}(b)).$$

乘号表示逐项相乘。调用顺序是分别变换 a、b，执行 `a[i]=i64(a[i])*b[i]%mod`，再逆变换 a。每次变换时间 $O(k2^k)$、额外空间 $O(1)$。OR 允许 $A\cap B\neq\emptyset$，不能直接替代下面的不交子集卷积。

不交子集卷积：`convolution(a,b)` 求 $c[S]=\sum_{T\subseteq S}a[T]b[S\setminus T]$。按集合大小分层再做子集 FMT：

$$A_i[S]=\sum_{\substack{T\subseteq S\\|T|=i}}a[T],\qquad B_i[S]=\sum_{\substack{T\subseteq S\\|T|=i}}b[T].$$

$$C_r[S]=\sum_{i=0}^{r}A_i[S]B_{r-i}[S],\qquad c[S]=(\mu C_{|S|})[S].$$

这里 μ 是子集 Möbius 逆变换。反演保证并集恰为 S，再取层数 $|S|$ 保证两集合不交。时间 $O(k^2 2^k)$，空间 $O(k2^k)$。参考 Björklund 等的 Fourier meets Möbius: fast subset convolution（arXiv:cs/0611101）；链接见 SOURCES.md。

线性基的正交补：在 $\mathbb{F}_2^k$ 中，把异或值看作位向量，内积为 $\langle x,y\rangle=\operatorname{popcount}(x\,\&\,y)\bmod 2$。给定子空间 V，所求通常是

$$V^\perp=\{y:\ \langle x,y\rangle=0\ \text{对所有}x\in V\},\qquad \dim V^\perp=k-\dim V.$$

XorBasis.h 没有专用的正交补接口，`reduced()` 只做消元化简。但 supplement/GaussBinary.h 已能求齐次解空间：把 V 的生成元逐行作为系数，右侧全填 0，调用 `gauss_binary(rows,k)`，返回的 `basis` 就是正交补的一组基。每行必须有 $\lceil(k+1)/64\rceil$ 个 64 位块；k=64 时应把生成元 x 编为 `{x,0}`，第 64 位是右端项。返回向量的第 j 项对应原掩码的第 j 位。

正交补中的向量均与 V 正交，但这组基的向量之间不保证两两正交。例如在 $\mathbb{F}_2^2$ 中，$V=\operatorname{span}\{11_2\}$ 满足 $V^\perp=V$，所以不能套用实数空间的正交直和结论。实数向量的 Gram–Schmidt 正交化是另一件事，当前库没有专门实现。

与 FWT 的联系：令 1_V 为子空间 V 的示性函数，则

$$H(1_V)[S]=\begin{cases}|V|, & S\in V^\perp,\\ 0, & S\notin V^\perp.\end{cases}$$

若 $S\notin V^\perp$，取 $v\in V$ 使 $\langle S,v\rangle=1$，将 x 与 $x\oplus v$ 配对，符号恰好抵消。这能解释为何异或线性空间计数会同时出现 FWT 与正交补：求正交补本身只需消元，无须枚举 2^k 个状态。

7.8 位运算卷积 / 子集和 / 子集卷积

功能：计算 XOR / OR / AND 卷积、子集和变换及不交子集卷积。
约定：数组长度 2^k，系数模 998244353。type=0/1/2 对应 XOR/OR/AND；SOS 为子集和，inv 为反演；convolution 求 $\sum_{T\subseteq S}a[T]b[S\setminus T]$。
复杂度：FWT/SOS $\mathcal{O}(k2^k)$，子集卷积 $\mathcal{O}(k^2 2^k)$，空间 $\mathcal{O}(k2^k)$
依赖：§1.1, §7.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 846e7e

```cpp
namespace subset {
const int mod = 998244353;
void fwt(vector<int> &a, int type, bool inv = false) {
    int n = a.size();
    assert(n && (n & (n - 1)) == 0);
    for (int d = 1; d < n; d *= 2)
        for (int l = 0; l < n; l += 2 * d)
            for (int j = 0; j < d; j ++) {
                int &x = a[l + j], &y = a[l + j + d],
                    u = x, v = y;
                if (type == 0)
                    x = (u + v) % mod,
                    y = (u - v + mod) % mod;
                if (type == 1)
                    y = (v + (inv ? mod - u : u)) % mod;
                if (type == 2)
                    x = (u + (inv ? mod - v : v)) % mod;
            }
    if (type == 0 && inv) {
        int z = poly::power(n, mod - 2);
        for (int &x : a)
            x = i64(x) * z % mod;
    }
}
void sos(vector<int> &a, bool inv = false) {
    for (int b = 1; b < (int)a.size(); b *= 2)
        for (int s = 0; s < (int)a.size(); s ++)
            if (s & b)
                a[s] = (a[s] + (inv ? mod - a[s ^ b]
                                    : a[s ^ b])) %
                       mod;
}
vector<int> convolution(const vector<int> &a,
                        const vector<int> &b) {
    int n = a.size(), k = __lg(n);
    assert(n && (n & (n - 1)) == 0 &&
           b.size() == a.size());
    vector<vector<int>> x(k + 1, vector<int>(n)), y = x,
                                                  z = x;
    for (int s = 0; s < n; s ++)
        x[__builtin_popcount(s)][s] = a[s],
        y[__builtin_popcount(s)][s] = b[s];
    for (int i = 0; i <= k; i ++)
        sos(x[i]), sos(y[i]);
    for (int i = 0; i <= k; i ++)
        for (int j = 0; j <= i; j ++)
            for (int s = 0; s < n; s ++)
                z[i][s] = (z[i][s] +
                           i64(x[j][s]) * y[i - j][s]) %
                          mod;
    for (int i = 0; i <= k; i ++)
        sos(z[i], true);
    vector<int> c(n);
    for (int s = 0; s < n; s ++)
        c[s] = z[__builtin_popcount(s)][s];
    return c;
}
} // namespace subset
```

7.9 线性递推猜测 / 第 k 项（BM）

功能：由已知项猜测常系数线性递推，再求递推的第 k 项。
约定：模 998244353；递推 $s_i=\sum_{j=1}^{d}c_{j-1}s_{i-j}$。BM 至少给 2d 项才能保证恢复真实 d 阶递推，有限样本拟合不等于证明。k 从 0 起，空递推视后续为零。
复杂度：BM $\mathcal{O}(L^2)$，第 k 项 $\mathcal{O}(d^2\log k)$
依赖：§1.1, §7.1（递归收齐依赖）。

BlueLagoon（参考 cp-book, koosaga）/ CC0-1.0 / 本地回归 / 对拍 49e9ae

```cpp
namespace poly {
Vec berlekamp_massey(const Vec &s) {
    Vec c = { 1 }, b = { 1 };
    int l = 0, m = 1, last = 1;
    for (int i = 0; i < (int)s.size(); i ++) {
        int d = s[i];
        for (int j = 1; j <= l; j ++)
            d = (d + i64(c[j]) * s[i - j]) % mod;
        if (!d) {
            m ++;
            continue;
        }
        Vec old = c;
```

```
        int coef = i64(d) * power(last, mod - 2) % mod;
        c.resize(max(c.size(), b.size() + m));
        for (int j = 0; j < (int)b.size(); j ++)
            c[j + m] = (c[j + m] -
                        i64(coef) * b[j] % mod + mod) %
                       mod;
        if (2 * l <= i)
            l = i + 1 - l, b = old, last = d, m = 1;
        else
            m ++;
    }
    c.resize(l + 1);
    c.erase(c.begin());
    for (int &x : c)
        x = x ? mod - x : 0;
    return c;
}
int linear_rec(Vec init, const Vec &c, u64 k) {
    if (k < init.size()) return init[k];
    int n = c.size();
    if (!n) return 0;
    assert(init.size() >= (size_t)n);
    auto mul = [&](Vec a, Vec b) {
        Vec d(2 * n - 1);
        for (int i = 0; i < n; i ++)
            for (int j = 0; j < n; j ++)
                d[i + j] =
                    (d[i + j] + i64(a[i]) * b[j]) % mod;
        for (int i = 2 * n - 2; i >= n; i --)
            for (int j = 1; j <= n; j ++)
                d[i - j] =
                    (d[i - j] + i64(d[i]) * c[j - 1]) %
                    mod;
        d.resize(n);
        return d;
    };
    Vec a(n), b(n);
    a[0] = 1;
    if (n == 1)
        b[0] = c[0];
    else
        b[1] = 1;
    for (; k; k >>= 1, b = mul(b, b))
        if (k & 1) a = mul(a, b);
    int ans = 0;
    for (int i = 0; i < n; i ++)
        ans = (ans + i64(a[i]) * init[i]) % mod;
    return ans;
}
} // namespace poly
```

7.10 实数 / 模素数高斯消元与行列式

功能：求实数 / 素数模线性方程组的解，并计算方阵行列式。
约定：增广矩阵 n×(m+1)，返回 0 无解 / 1 唯一 / 2 多解，自由变量取 0。模版要求 p 为素数、系数归一；determinant 输入方阵。实数阈值须按规模调整。
复杂度：$\mathcal{O}(nm\min(n,m))$；行列式 $\mathcal{O}(n^3)$
依赖：§1.1, §6.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 77da40

```
int gauss_real(vector<vector<double>> a,
               vector<double> &ans, double eps = 1e-9) {
    int n = a.size(), m = n ? a[0].size() - 1 : 0,
        r = 0;
    vector<int> pos(m, -1);
    for (int c = 0; c < m && r < n; c ++) {
        int p = r;
        for (int i = r; i < n; i ++)
            if (abs(a[i][c]) > abs(a[p][c])) p = i;
        if (abs(a[p][c]) < eps) continue;
        swap(a[p], a[r]);
        pos[c] = r;
        double v = a[r][c];
        for (int j = c; j <= m; j ++)
            a[r][j] /= v;
        for (int i = 0; i < n; i ++)
            if (i != r) {
                v = a[i][c];
                for (int j = c; j <= m; j ++)
                    a[i][j] -= v * a[r][j];
            }
        r ++;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i ++)
        if (pos[i] != -1) ans[i] = a[pos[i]][m];
    for (int i = r; i < n; i ++)
        if (abs(a[i][m]) > eps) return 0;
    return r == m ? 1 : 2;
}
int gauss_mod(vector<vector<int>> a, vector<int> &ans,
              int p) {
    int n = a.size(), m = n ? a[0].size() - 1 : 0,
        r = 0;
    vector<int> pos(m, -1);
    for (int c = 0; c < m && r < n; c ++) {
        int k = r;
        while (k < n && !a[k][c])
            k ++;
        if (k == n) continue;
        swap(a[k], a[r]);
        pos[c] = r;
        i64 v = pow_mod(a[r][c], p - 2, p);
        for (int j = c; j <= m; j ++)
            a[r][j] = a[r][j] * v % p;
        for (int i = 0; i < n; i ++)
            if (i != r) {
                v = a[i][c];
                for (int j = c; j <= m; j ++)
                    a[i][j] = (a[i][j] -
                               v * a[r][j] % p + p) %
                              p;
            }
        r ++;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i ++)
        if (pos[i] != -1) ans[i] = a[pos[i]][m];
    for (int i = r; i < n; i ++)
        if (a[i][m]) return 0;
    return r == m ? 1 : 2;
}
int determinant(vector<vector<int>> a, int p) {
    int n = a.size();
    i64 ans = 1;
    for (int c = 0; c < n; c ++) {
        int k = c;
        while (k < n && !a[k][c])
            k ++;
        if (k == n) return 0;
        if (k != c)
            swap(a[k], a[c]), ans = (p - ans) % p;
        ans = ans * a[c][c] % p;
        i64 inv = pow_mod(a[c][c], p - 2, p);
        for (int i = c + 1; i < n; i ++) {
            i64 v = a[i][c] * inv % p;
            for (int j = c; j < n; j ++)
                a[i][j] =
                    (a[i][j] - v * a[c][j] % p + p) % p;
        }
    }
    return ans;
}
```

7.11 异或线性基

功能：维护可表示的异或值，求最大异或和及去重后的第 k 小异或值。
约定：unsigned 64 位；kth 是去重后所有可表示异或值的 0-based 第 k 小，包含空集得到的 0。若题目要求非空子集，需另记线性相关性。
复杂度：插入 / 最大值 $\mathcal{O}(64)$，kth $\mathcal{O}(64^2)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 8baff9

```cpp
struct XorBasis {
    array<u64, 64> b{};
    bool insert(u64 x) {
        for (int i = 63; i >= 0; i --)
            if (x >> i & 1) {
                if (!b[i]) {
                    b[i] = x;
                    return true;
                }
                x ^= b[i];
            }
        return false;
    }
    bool contains(u64 x) const {
        for (int i = 63; i >= 0; i --)
            if (x >> i & 1) x ^= b[i];
        return x == 0;
    }
    u64 maximum(u64 x = 0) const {
        for (int i = 63; i >= 0; i --)
            x = max(x, x ^ b[i]);
        return x;
    }
    vector<u64> reduced() const {
        auto a = b;
        for (int i = 0; i < 64; i ++)
            if (a[i])
                for (int j = i + 1; j < 64; j ++)
                    if (a[j] >> i & 1) a[j] ^= a[i];
        vector<u64> v;
        for (u64 x : a)
            if (x) v.push_back(x);
        return v;
    }
    u64 kth(u64 k) const {
        auto v = reduced();
        assert(v.size() == 64 ||
               k < (1ULL << v.size()));
        u64 ans = 0;
        for (int i = 0; i < (int)v.size(); i ++)
            if (k >> i & 1) ans ^= v[i];
        return ans;
    }
};
```

7.12 生成树计数（Matrix-Tree）

功能：统计无向生成树、有向根向 / 根出树；带权时求边权积之和。
约定：0-based，n>=1、素数模；自环忽略。spanning_trees 无向无权；weighted_spanning_trees 的边为 (u,v,w)，权可按模归一；directed_spanning_trees(n,root,edges,mod,toward) 为根向树（每个点沿原边到根），toward=false 为从根出发的树。n=1 返回 1；根删行列。例：0→1、1→2 的单位权边，root=0,toward=false 返回 1。
复杂度：$\mathcal{O}(n^3)$
依赖：§1.1, §7.10（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 66d640

```cpp
int weighted_spanning_trees(
    int n, const vector<tuple<int, int, int>> &ed,
    int mod) {
    assert(n >= 1);
    vector<vector<int>> a(n, vector<int>(n));
    auto add = [&](int u, int v, i64 x) {
        a[u][v] = (a[u][v] + x + mod) % mod;
    };
    for (auto [u, v, w] : ed)
        if (u != v) {
            w = (i64(w) % mod + mod) % mod;
            add(u, u, w), add(v, v, w), add(u, v, -w),
                add(v, u, -w);
        }
    a.pop_back();
    for (auto &r : a)
        r.pop_back();
    return determinant(a, mod);
}

int directed_spanning_trees(
    int n, int root,
    const vector<tuple<int, int, int>> &ed, int mod,
    bool toward = true) {
    assert(n >= 1 && 0 <= root && root < n);
    vector<vector<int>> a(n, vector<int>(n));
    for (auto [u, v, w] : ed)
        if (u != v) {
            if (!toward) swap(u, v);
            w = (i64(w) % mod + mod) % mod;
            a[u][u] = (i64(a[u][u]) + w) % mod;
            a[u][v] = (i64(a[u][v]) - w + mod) % mod;
        }
    a.erase(a.begin() + root);
    for (auto &r : a)
        r.erase(r.begin() + root);
    return determinant(a, mod);
}

int spanning_trees(int n,
                   const vector<pair<int, int>> &ed,
                   int mod) {
    assert(n >= 1);
    vector<vector<int>> a(n, vector<int>(n));
    for (auto [u, v] : ed)
        if (u != v) {
            a[u][u] = (a[u][u] + 1) % mod;
            a[v][v] = (a[v][v] + 1) % mod;
            a[u][v] = (a[u][v] + mod - 1) % mod;
            a[v][u] = (a[v][u] + mod - 1) % mod;
        }
    a.pop_back();
    for (auto &r : a)
        r.pop_back();
    return determinant(a, mod);
}
```

7.13 线性规划最优化（单纯形法）

功能：求非负变量在线性不等式约束下的线性目标最大值。
约定：最大化 $c^{\mathsf{T}}x$，约束 $Ax \le b$、$x \ge 0$，要求至少一条约束。solve(x) 返回 -inf 无解、+inf 无界或最优值；数值稳定性依赖建模尺度，需检查约束残差。
复杂度：$\mathcal{O}(nm \cdot pivot$数$)$，最坏指数
依赖：§1.1（递归收齐依赖）。

KACTL / MIT / 本地回归 / 对拍 154dc5

```cpp
using T =
    double; // long double, Rational, double + mod<P>...
using vd = vector<T>;
using vvd = vector<vd>;

const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
#define ltj(X)                                          \
    if (s == -1 || MP(X[j], N[j]) < MP(X[s], N[s]))     \
    s = j

struct LPSolver {
    int m, n;
    vector<int> N, B;
    vvd D;

    LPSolver(const vvd &A, const vd &b, const vd &c)
        : m((int)(b).size()), n((int)(c).size()),
          N(n + 1), B(m), D(m + 2, vd(n + 2)) {
        for (int i = 0; i < m; i ++)
            for (int j = 0; j < n; j ++)
                D[i][j] = A[i][j];
        for (int i = 0; i < m; i ++) {
            B[i] = n + i;
            D[i][n] = -1;
            D[i][n + 1] = b[i];
        }
        for (int j = 0; j < n; j ++) {
```

```
            N[j] = j;
            D[m][j] = -c[j];
        }
        N[n] = -1;
        D[m + 1][n] = 1;
    }

    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        for (int i = 0; i < m + 2; i ++)
            if (i != r && abs(D[i][s]) > eps) {
                T *b = D[i].data(), inv2 = b[s] * inv;
                for (int j = 0; j < n + 2; j ++)
                    b[j] -= a[j] * inv2;
                b[s] = a[s] * inv2;
            }
        for (int j = 0; j < n + 2; j ++)
            if (j != s) D[r][j] *= inv;
        for (int i = 0; i < m + 2; i ++)
            if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }

    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            for (int j = 0; j < n + 1; j ++)
                if (N[j] != -phase) ltj(D[x]);
            if (D[x][s] >= -eps) return true;
            int r = -1;
            for (int i = 0; i < m; i ++) {
                if (D[i][s] <= eps) continue;
                if (r == -1 ||
                    MP(D[i][n + 1] / D[i][s], B[i]) <
                        MP(D[r][n + 1] / D[r][s], B[r]))
                    r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }

    T solve(vd &x) {
        int r = 0;
        for (int i = 1; i < m; i ++)
            if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -eps) {
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] < -eps)
                return -inf;
            for (int i = 0; i < m; i ++)
                if (B[i] == -1) {
                    int s = 0;
                    for (int j = 1; j < n + 1; j ++)
                        ltj(D[i]);
                    pivot(i, s);
                }
        }
        bool ok = simplex(1);
        x = vd(n);
        for (int i = 0; i < m; i ++)
            if (B[i] < n) x[B[i]] = D[i][n + 1];
        return ok ? D[m][n + 1] : inf;
    }
};

#undef MP
#undef ltj
```

8 计算几何

8.1 二维点与向量

功能：提供二维点向量的算术、点积、叉积、长度与方向操作。
约定：Point<i64> 用于整数谓词，Point<double> 用于构造与距离；dot/cross/dist2 均以 T 计算，先估计溢出。unit/normal 要求非零向量。
复杂度：$\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 0d8235

```
template <class T> int sgn(T x) {
    return (x > 0) - (x < 0);
}
template <class T> struct Point {
    using P = Point;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const {
        return tie(x, y) < tie(p.x, p.y);
    }
    bool operator==(P p) const {
        return tie(x, y) == tie(p.x, p.y);
    }
    P operator+(P p) const {
        return P(x + p.x, y + p.y);
    }
    P operator-(P p) const {
        return P(x - p.x, y - p.y);
    }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; }
    T cross(P p) const { return x * p.y - y * p.x; }
    T cross(P a, P b) const {
        return (a - *this).cross(b - *this);
    }
    T dist2() const { return x * x + y * y; }
    double dist() const {
        return sqrt((double)dist2());
    }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const {
        return *this / dist();
    } // makes dist()=1
    P perp() const {
        return P(-y, x);
    } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the
    // origin
    P rotate(double a) const {
        return P(x * cos(a) - y * sin(a),
                 x * sin(a) + y * cos(a));
    }
    friend ostream &operator<<(ostream &os, P p) {
        return os << "(" << p.x << "," << p.y << ")";
    }
};
```

8.2 点在线段上

功能：判断一个点是否在线段上，包含端点。
约定：整数点精确判断；浮点点在线段上用 segDist<=eps。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 69edb6

```
template <class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 &&
           (s - p).dot(e - p) <= 0;
}
```

8.3 有向直线侧别

功能：判断点在有向直线左侧、右侧还是线上。
约定：返回左 1 / 线上 0 / 右-1；eps 版本使用到直线的距离，要求起终点不同。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 15f286

```
template <class P> int sideOf(P s, P e, P p) {
    return sgn(s.cross(e, p));
}

template <class P>
int sideOf(P s, P e, P p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}
```

8.4 直线交点

功能：求两条直线交点，并区分平行与重合。
约定：返回 1 唯一 / 0 平行 / -1 重合；用 Point<double> 构造非整数交点。方向向量非零；精确零比较需按题目调整。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 3c039b

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return { -(s1.cross(e1, s2) == 0), P(0, 0) };
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return { 1, (s1 * p + e1 * q) / d };
}
```

8.5 线段交点

功能：求两条线段的交点或重叠区间端点。
约定：返回 0 / 1 / 2 个点，2 个表示重叠区间端点。整数点仅适用于交点也为整数的场景；一般构造用浮点并调整 eps。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1, §8.2（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 18df25

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
         oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint
    // point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return { (a * ob - b * oa) / (ob - oa) };
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return { (s).begin(), (s).end() };
}
```

8.6 点到直线的投影 / 轴对称

功能：求点在直线上的投影，或关于该直线的对称点。
约定：a!=b，Point<double>；refl=false 投影，true 对称。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 6b54d8

```
template <class P>
P lineProj(P a, P b, P p, bool refl = false) {
    P v = b - a;
    return p - v.perp() * (1 + refl) * v.cross(p - a) /
                   v.dist2();
}
```

8.7 点到直线距离

功能：求点到有向直线的带符号距离。
约定：带符号距离，左正右负；a!=b。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 d05ad0

```
template <class P> double lineDist(P a, P b, P p) {
    return (double)(b - a).cross(p - a) /
           (b - a).dist();
}
```

8.8 点到线段距离

功能：求点到线段的最短距离。
约定：DP=Point<double>；零长度线段可处理。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 3820a0

```
using DP = Point<double>;
double segDist(DP s, DP e, DP p) {
    if (s == e) return (p - s).dist();
    auto d = (e - s).dist2(),
         t = min(d, max(.0, (p - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```

8.9 点在多边形内

功能：判断点是否位于简单多边形内部，可选择是否包含边界。
约定：整数谓词，简单多边形；strict 控制是否含边界。浮点使用时需补误差判断；空多边形返回 false。
复杂度：$\mathcal{O}(n)$
依赖：§1.1, §8.1, §8.2, §8.8（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 1587f8

```
template <class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = (int)(p).size();
    for (int i = 0; i < n; i ++) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        // or: if (segDist(p[i], q, a) <= eps) return
        // !strict;
        cnt ^= ((a.y < p[i].y) - (a.y < q.y)) *
                   a.cross(p[i], q) >
               0;
    }
    return cnt;
}
```

8.10 多边形有向面积

功能：使用叉积求多边形两倍有向面积。
约定：返回 2 倍有向面积，CCW 为正；输入非空。i64 乘法可能溢出，可将点类型改 i128。
复杂度：$\mathcal{O}(n)$
依赖：§1.1, §8.1（递归收齐依赖）。
KACTL / CC0 / 本地回归 / 对拍 58bf75

```
template <class T> T polygonArea2(vector<Point<T>> &v) {
    T a = v.back().cross(v[0]);
    for (int i = 0; i < (int)(v).size() - 1; i ++)
        a += v[i].cross(v[i + 1]);
    return a;
}
```

8.11 凸包

功能：求二维点集的逆时针凸包，剔除边上中间共线点。
约定：IP=Point<i64>，返回 CCW 凸包，去掉边上中间点，允许空集 / 重复点 / 全共线。整数叉积不得溢出。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / Unlicense / 本地回归 / 对拍 a6ff1f

```
using IP = Point<i64>;
vector<IP> convexHull(vector<IP> pts) {
    if ((int)(pts).size() <= 1) return pts;
    sort((pts).begin(), (pts).end());
    vector<IP> h((int)(pts).size() + 1);
    int s = 0, t = 0;
    for (int it = 2; it --;
         s = --t, reverse((pts).begin(), (pts).end()))
        for (IP p : pts) {
            while (t >= s + 2 &&
                   h[t - 2].cross(h[t - 1], p) <= 0)
                t --;
            h[t ++] = p;
        }
    return { h.begin(),
             h.begin() + t - (t == 2 && h[0] == h[1]) };
}
```

8.12 凸包直径 / 最远点对（旋转卡壳）

功能：求凸多边形中欧氏距离最远的一对顶点。
约定：输入非空 CCW 凸包，无相邻重复与中间共线点；返回最远点对。平方距离 i64 不得溢出。
复杂度：$\mathcal{O}(n)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / Boost Software License / 本地回归 / 对拍 47df7b

```
using IP = Point<i64>;
array<IP, 2> hullDiameter(vector<IP> S) {
    int n = (int)(S).size(), j = n < 2 ? 0 : 1;
    pair<i64, array<IP, 2>> res({ 0, { S[0], S[0] } });
    for (int i = 0; i < j; i ++)
        for (;; j = (j + 1) % n) {
            res = max(res, { (S[i] - S[j]).dist2(),
                             { S[i], S[j] } });
            if ((S[(j + 1) % n] - S[j])
                    .cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

8.13 最近点对

功能：求二维整数点集中欧氏距离最近的一对点。
约定：IP 整数点，至少 2 点；平方距离及坐标差不溢出，重复点允许。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 b97cc0

```
using IP = Point<i64>;
pair<IP, IP> closest(vector<IP> v) {
    assert((int)(v).size() > 1);
    set<IP> S;
    sort((v).begin(), (v).end(),
         [](IP a, IP b) { return a.y < b.y; });
    pair<i64, pair<IP, IP>> ret{ LLONG_MAX,
                                 { IP(), IP() } };
    int j = 0;
    for (IP p : v) {
        IP d{ 1 + (i64)sqrt(ret.first), 0 };
        while (v[j].y <= p.y - d.x)
            S.erase(v[j ++]);
        auto lo = S.lower_bound(p - d),
             hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(
                ret, { (*lo - p).dist2(), { *lo, p } });
        S.insert(p);
    }
    return ret.second;
}
```

8.14 点在凸多边形内的查询

功能：对给定凸包判断点是否在内部，可选择是否包含边界。
约定：IP；非空 CCW 严格凸包；strict 控制含边界，小于 3 点也能处理。
复杂度：$\mathcal{O}(\log n)$
依赖：§1.1, §8.1, §8.3, §8.2（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 462104

```
using IP = Point<i64>;

bool inHull(const vector<IP> &l, IP p,
            bool strict = true) {
    int a = 1, b = (int)(l).size() - 1, r = !strict;
    if ((int)(l).size() < 3)
        return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r ||
        sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

8.15 凸包极值点与直线交

功能：求凸包在指定方向的极值顶点，以及直线穿过的边。
约定：严格 CCW 凸多边形，至少 3 点；extrVertex 求方向投影最大点，lineHull 返回被穿过的边编号。具体返回编码见本章速查。
复杂度：$\mathcal{O}(\log n)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / Boost Software License / 本地回归 / 对拍 48c5af

```
#define cmp(i, j)                                       \
    sgn(dir.perp().cross(poly[(i) % n] - poly[(j) % n]))
#define extr(i)                                         \
    cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P> &poly, P dir) {
    int n = (int)(poly).size(), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m))
             ? hi
             : lo) = m;
    }
    return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P> &poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return { -1, -1 };
    array<int, 2> res;
    for (int i = 0; i < 2; i ++) {
        int lo = endB, hi = endA,
            n = (int)(poly).size();
        while ((lo + 1) % n != hi) {
            int m =
                ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
```

```
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return { res[0], -1 };
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch (
            (res[0] - res[1] + (int)(poly).size() + 1) %
            (int)(poly).size()) {
            case 0:
                return { res[0], res[0] };
            case 2:
                return { res[1], res[1] };
        }
    return res;
}

#undef cmp
#undef extr
#undef cmpL
```

8.16 两圆交点

功能：求两个圆的交点。
约定：DP，半径非负；相同圆（无限交点）须调用前单独处理。返回两交点，相切时重复；精度边界按尺度调整。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 568109

```
using DP = Point<double>;
bool circleInter(DP a, DP b, double r1, double r2,
                 pair<DP, DP> *out) {
    if (a == b) {
        assert(r1 != r2);
        return false;
    }
    DP vec = b - a;
    double d2 = vec.dist2(), sum = r1 + r2,
           dif = r1 - r2,
           p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2),
           h2 = r1 * r1 - p * p * d2;
    if (sum * sum < d2 || dif * dif > d2) return false;
    DP mid = a + vec * p,
       per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = { mid + per, mid - per };
    return true;
}
```

8.17 圆与直线相交

功能：求圆与直线的交点。
约定：Point<double>，半径非负，a!=b；返回 0/1/2 个交点。h2 接近零时应按题目尺度设 eps。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 bc8899

```
template <class P>
vector<P> circleLine(P c, double r, P a, P b) {
    P ab = b - a,
      p = a + ab * (c - a).dot(ab) / ab.dist2();
    double s = a.cross(b, c),
           h2 = r * r - s * s / ab.dist2();
    if (h2 < 0) return {};
    if (h2 == 0) return { p };
    P h = ab.unit() * sqrt(h2);
    return { p - h, p + h };
}
```

8.18 圆的公切线

功能：求两圆公切线的切点对，也可求过给定点的圆切线。
约定：Point<double>；r2 取负求内公切线，r2=0 求过点切线；返回切点对，相切时两点相同需另用垂直圆心连线方向表示切线。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 4b62fe

```
template <class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2,
                            double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(),
           h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : { -1, 1 }) {
        P v =
            (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({ c1 + v * r1, c2 + v * r2 });
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

8.19 三角形外接圆

功能：求非共线三点确定的外接圆圆心和半径。
约定：DP，三个点不共线。ccCenter 返回圆心，ccRadius 返回半径。
复杂度：$\mathcal{O}(1)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 f11084

```
using DP = Point<double>;
double ccRadius(DP A, DP B, DP C) {
    return (B - A).dist() * (C - B).dist() *
           (A - C).dist() / abs((B - A).cross(C - A)) /
           2;
}
DP ccCenter(DP A, DP B, DP C) {
    DP b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() /
                   b.cross(c) / 2;
}
```

8.20 最小覆盖圆

功能：求覆盖全部给定点、半径最小的圆。
约定：DP；返回圆心和半径，输入非空。随机增量，允许共线与重复点；浮点误差按尺度调整。
复杂度：期望 $\mathcal{O}(n)$
依赖：§1.1, §8.19（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 979b88

```
pair<DP, double> mec(vector<DP> ps) {
    shuffle((ps).begin(), (ps).end(), mt19937(time(0)));
    DP o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    for (int i = 0; i < (int)(ps).size(); i ++)
        if ((o - ps[i]).dist() > r * EPS) {
            o = ps[i], r = 0;
            for (int j = 0; j < i; j ++)
                if ((o - ps[j]).dist() > r * EPS) {
                    o = (ps[i] + ps[j]) / 2;
                    r = (o - ps[i]).dist();
                    for (int k = 0; k < j; k ++)
                        if ((o - ps[k]).dist() >
                            r * EPS) {
                            o = ccCenter(ps[i], ps[j],
                                         ps[k]);
                            r = (o - ps[i]).dist();
                        }
                }
        }
    return { o, r };
}
```

8.21 圆与多边形交面积

功能：求圆与简单多边形相交区域的面积。
约定：DP；简单 CCW 多边形、无连续重复点、r>=0。返回有向交面积；CW 输入需取负或反序。
复杂度：$\mathcal{O}(n)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 62d1d3

```
using DP = Point<double>;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(DP c, double r, vector<DP> ps) {
    auto tri = [&](DP p, DP q) {
        auto r2 = r * r / 2;
        DP d = q - p;
        auto a = d.dot(p) / d.dist2(),
             b = (p.dist2() - r * r) / d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)),
             t = min(1., -a + sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        DP u = p + d * s, v = q + d * (t - 1);
        return arg(p, u) * r2 + u.cross(v) / 2 +
               arg(v, q) * r2;
    };
    auto sum = 0.0;
    for (int i = 0; i < (int)(ps).size(); i ++)
        sum += tri(ps[i] - c,
                   ps[(i + 1) % (int)(ps).size()] - c);
    return sum;
}

#undef arg
```

8.22 三维向量

功能：提供三维向量点积、叉积、距离、归一化与绕轴旋转。
约定：Point3D<i64/double>；dot、cross、距离、绕轴旋转；归一化要求非零向量。
复杂度：$\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 独立编译 3f3fda

```
template <class T> struct Point3D {
    using P = Point3D;
    T x, y, z;
    explicit Point3D(T x = 0, T y = 0, T z = 0)
        : x(x), y(y), z(z) {}
    bool operator<(P p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z);
    }
    bool operator==(P p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z);
    }
    P operator+(P p) const {
        return P(x + p.x, y + p.y, z + p.z);
    }
    P operator-(P p) const {
        return P(x - p.x, y - p.y, z - p.z);
    }
    P operator*(T d) const {
        return P(x * d, y * d, z * d);
    }
    P operator/(T d) const {
        return P(x / d, y / d, z / d);
    }
    T dot(P p) const {
        return x * p.x + y * p.y + z * p.z;
    }
    P cross(P p) const {
        return P(y * p.z - z * p.y, z * p.x - x * p.z,
                 x * p.y - y * p.x);
    }
    T dist2() const { return x * x + y * y + z * z; }
    double dist() const {
        return sqrt((double)dist2());
    }
    // Azimuthal angle (longitude) to x-axis in interval
    // [-pi, pi]
    double phi() const { return atan2(y, x); }
    // Zenith angle (latitude) to the z-axis in interval
    // [0, pi]
    double theta() const {
        return atan2(sqrt(x * x + y * y), z);
    }
    P unit() const {
        return *this / (T)dist();
    } // makes dist()=1
    // returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }
    // returns point rotated 'angle' radians ccw around
    // axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle);
        P u = axis.unit();
        return u * dot(u) * (1 - c) + (*this) * c -
               cross(u) * s;
    }
};
```

8.23 三维凸包

功能：求一般位置三维点集的凸包，输出外向三角面。
约定：至少 4 点且任意 4 点不共面；输出外向三角面。含共面输入先选择适合的退化处理方案，不可直接套用。
复杂度：$\mathcal{O}(n^2)$
依赖：§1.1, §8.22（递归收齐依赖）。

KACTL / Not specified upstream; retain attribution / 本地回归 / 对拍 6ccad0

```
using P3 = Point3D<double>;

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};

struct F {
    P3 q;
    int a, b, c;
};

vector<F> hull3d(const vector<P3> &A) {
    assert((int)(A).size() >= 4);
    vector<vector<PR>> E(
        (int)(A).size(),
        vector<PR>((int)(A).size(), { -1, -1 }));
#define E(x, y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i])) q = q * -1;
        F f{ q, i, j, k };
        E(a, b).ins(k);
        E(a, c).ins(j);
        E(b, c).ins(i);
        FS.push_back(f);
    };
    for (int i = 0; i < 4; i ++)
        for (int j = i + 1; j < 4; j ++)
            for (int k = j + 1; k < 4; k ++)
                mf(i, j, k, 6 - i - j - k);

    for (int i = 4; i < (int)(A).size(); i ++) {
        for (int j = 0; j < (int)(FS).size(); j ++) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a, b).rem(f.c);
                E(a, c).rem(f.b);
                E(b, c).rem(f.a);
                swap(FS[j --], FS.back());
                FS.pop_back();
            }
```

```
        }
        int nw = (int)(FS).size();
        for (int j = 0; j < nw; j ++) {
            F f = FS[j];
#define C(a, b, c)                                          \
    if (E(a, b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c);
            C(a, c, b);
            C(b, c, a);
        }
    }
    for (F &it : FS)
        if ((A[it.b] - A[it.a])
                .cross(A[it.c] - A[it.a])
                .dot(it.q) <= 0)
            swap(it.c, it.b);
    return FS;
};

#undef E
#undef C
```

8.24 点到凸多边形的切线

功能：求经过给定点、与凸多边形相切的两侧切点。
约定：c 为 CCW 强凸多边形且末尾重复首点，至少 2 个不同顶点。p 不等于任何顶点；up=1/-1 求两侧切点，严格内部返回-1，共线切线可能任选端点。
复杂度：$\mathcal{O}(\log n)$
依赖：§1.1, §8.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 04683e

```
using IP = Point<i64>;
int convex_tangent(const vector<IP> &c, IP p,
                   int up = 1) {
    auto sign = [&](i64 x) {
        return x > 0 ? up : x == 0 ? 0 : -up;
    };
    auto local = [&](IP a, IP b, IP d) {
        return sign(p.cross(a, b)) <= 0 &&
               sign(p.cross(b, d)) >= 0;
    };
    int n = c.size() - 1, l = 0, r = n;
    if (local(c[1], c[0], c[n - 1])) return 0;
    while (l + 1 < r) {
        int m = (l + r) / 2;
        if (local(c[m - 1], c[m], c[m + 1])) return m;
        if (sign(p.cross(c[l], c[l + 1])) < 0) {
            if (sign(p.cross(c[m], c[m + 1])) > 0)
                r = m;
            else if (sign(p.cross(c[m], c[l])) > 0)
                l = m;
            else
                r = m;
        } else {
            if (sign(p.cross(c[m], c[m + 1])) < 0)
                l = m;
            else if (sign(p.cross(c[m], c[l])) < 0)
                l = m;
            else
                r = m;
        }
    }
    if (l && local(c[l - 1], c[l], c[l + 1])) return l;
    if (r != n && local(c[r - 1], c[r], c[r + 1]))
        return r;
    return -1;
}
```

8.25 凸多边形闵可夫斯基和

功能：求两个凸多边形点向量之和形成的凸多边形。
约定：输入 CCW 严格凸包且不重复首点；允许空、点、线段退化，退化时用凸包收尾。坐标和与叉积须不溢出。
复杂度：非退化 $\mathcal{O}(n+m)$，退化 $\mathcal{O}((n+m)\log(n+m))$
依赖：§1.1, §8.11（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 3ab648

```
using IP = Point<i64>;
vector<IP> minkowski(vector<IP> a, vector<IP> b) {
    if (a.empty() || b.empty()) return {};
    if (a.size() < 3 || b.size() < 3) {
        vector<IP> c;
        for (IP x : a)
            for (IP y : b)
                c.push_back(x + y);
        return convexHull(c);
    }
    auto rotate_min = [](vector<IP> &p) {
        auto it = min_element(
            p.begin(), p.end(), [](IP x, IP y) {
                return tie(x.y, x.x) < tie(y.y, y.x);
            });
        rotate(p.begin(), it, p.end());
    };
    rotate_min(a);
    rotate_min(b);
    int n = a.size(), m = b.size();
    a.push_back(a[0]);
    b.push_back(b[0]);
    vector<IP> c;
    int i = 0, j = 0;
    IP p = a[0] + b[0];
    c.push_back(p);
    while (i < n || j < m) {
        IP x = i < n ? a[i + 1] - a[i] : IP(),
           y = j < m ? b[j + 1] - b[j] : IP();
        i64 z = x.cross(y);
        bool ai = i < n && (j == m || z >= 0),
             bj = j < m && (i == n || z <= 0);
        if (ai) p = p + x, i ++;
        if (bj) p = p + y, j ++;
        c.push_back(p);
    }
    c.pop_back();
    return c;
}
```

8.26 半平面交

功能：求若干有向直线左侧半平面的交，并与指定边界框求交。
约定：保留每条有向直线的左侧；起终点不同。返回与 $[-\text{bound}, \text{bound}]^2$ 的交，不自动判无界；bound 必须覆盖题目全部合法解。空交返回空；低维交可能为空或含重复顶点，需另判面积与维度；eps 是叉积量纲，需按方向长度调整。
复杂度：$\mathcal{O}(n\log n)$
依赖：§1.1, §8.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 5e58dc

```
using DP = Point<double>;
struct HalfPlane {
    DP p, v;
    double angle;
    HalfPlane(DP a, DP b)
        : p(a), v(b - a), angle(atan2(v.y, v.x)) {}
};
vector<DP> halfplane(vector<HalfPlane> a, double bound,
                     double eps = 1e-9) {
    assert(bound > 0);
    vector<DP> box = { DP(-bound, -bound),
                       DP(bound, -bound),
                       DP(bound, bound),
                       DP(-bound, bound) };
    for (int i = 0; i < 4; i ++)
        a.emplace_back(box[i], box[(i + 1) % 4]);
    auto out = [&](const HalfPlane &h, DP p) {
        return h.v.cross(p - h.p) < -eps;
    };
    auto inter = [](const HalfPlane &a,
                    const HalfPlane &b) {
        return a.p + a.v * (b.v.cross(a.p - b.p) /
                            a.v.cross(b.v));
    };
```

```
    sort(a.begin(), a.end(),
         [](const auto &x, const auto &y) {
             return x.angle < y.angle;
         });
    deque<HalfPlane> q;
    for (auto h : a) {
        while (q.size() > 1 &&
               out(h, inter(q[q.size() - 2], q.back())))
            q.pop_back();
        while (q.size() > 1 &&
               out(h, inter(q[0], q[1])))
            q.pop_front();
        if (!q.empty() &&
            abs(h.v.cross(q.back().v)) < eps) {
            if (h.v.dot(q.back().v) < 0) return {};
            if (out(h, q.back().p))
                q.pop_back();
            else
                continue;
        }
        q.push_back(h);
    }
    while (q.size() > 2 &&
           out(q.front(),
               inter(q[q.size() - 2], q.back())))
        q.pop_back();
    while (q.size() > 2 &&
           out(q.back(), inter(q[0], q[1])))
        q.pop_front();
    if (q.size() < 3) return {};
    vector<DP> ans;
    for (int i = 0; i < (int)q.size(); i ++)
        ans.push_back(
            inter(q[i], q[(i + 1) % q.size()]));
    return ans;
}
```

8.27 几何类型与返回编码速查

整数接口使用 IP=Point<i64>；构造接口使用 DP=Point<double>；两者可同时包含。泛型函数按传入点类型实例化。

Point<T> 的 cross/dot/dist2 直接用 T，坐标较大时 Point<i128> 或独立 i128 叉积；不能仅在乘法完成后转类型。

浮点模板中的 ==0、h2<0 是基准实现。近退化题目要统一改为按尺度的 eps；不同量纲不能共用阈值。

lineHull 返回：(-1,-1) 不交；(i,-1) 触顶点 i；(i,i) 沿边 i；(i,j) 穿过边 i 和 j，边 i 表示 i→(i+1)%n。

凸包必须统一 CCW、去重复点、去中间共线点；需要保留共线点的题目不能直接使用严格凸包查询。

三维凸包本实现要求一般位置（任意四点不共面），不能把扰动后结果当精确退化处理。

圆的相切、重合、零半径分别处理；公切线切点相同时，需要使用显式方向表达那条切线。

9 优化、计数与公式速查

9.1 决策单调 DP 的分治优化

功能：在最优决策单调的前提下，加速一层 DP 的转移。
约定：dp[i]=min(0<=j<i) get(i,j)；get 通常含上一层 dp[j]。必须证明最左最优决策随 i 不减，且 get 不依赖当前尚未求出的 dp。返回 dp[0]=0；不可达用小于 LLONG_MAX 的有限 INF。
复杂度：每层 $\mathcal{O}(n \log n)$ oracle 调用
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 a3a28f

```
template <class F> vector<i64> divide_dp(int n, F get) {
    vector<i64> dp(n + 1);
    auto rec = [&](auto self, int l, int r, int lo,
                   int hi) -> void {
        if (l > r) return;
        int m = (l + r) / 2, k = -1;
        i64 best = LLONG_MAX;
        for (int j = lo; j <= min(hi, m - 1); j ++) {
            i64 v = get(m, j);
            if (v < best) best = v, k = j;
        }
        assert(k != -1);
        dp[m] = best;
        self(self, l, m - 1, lo, k);
        self(self, m + 1, r, k, hi);
    };
    if (n) rec(rec, 1, n, 0, n - 1);
    return dp;
}
```

9.2 区间 DP 四边形不等式优化（Knuth）

功能：用相邻区间最优决策界，缩小区间 DP 的分割点枚举范围。
约定：半开区间 dp[l][r]=min(l<k<r) dp[l][k]+dp[k][r]+cost(l,r)，单点代价 0。充分条件：cost 区间包含单调且满足四边形不等式，保证 opt[l][r-1]<=opt[l][r]<=opt[l+1][r]；和不得溢出。
复杂度：$\mathcal{O}(n^2)$ 时间 / 空间
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 9db544

```
template <class F> i64 knuth_dp(int n, F cost) {
    vector<vector<i64>> dp(n + 1, vector<i64>(n + 1));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1));
    for (int i = 0; i < n; i ++)
        opt[i][i + 1] = i + 1;
    for (int len = 2; len <= n; len ++)
        for (int l = 0; l + len <= n; l ++) {
            int r = l + len;
            dp[l][r] = LLONG_MAX;
            int lo = max(l + 1, opt[l][r - 1]),
                hi = min(r - 1, opt[l + 1][r]);
            for (int k = lo; k <= hi; k ++) {
                i64 v =
                    dp[l][k] + dp[k][r] + cost(l, r);
                if (v < dp[l][r])
                    dp[l][r] = v, opt[l][r] = k;
            }
        }
    return dp[0][n];
}
```

9.3 全单调矩阵行最小值（SMAWK）

功能：通过矩阵元素查询接口，求全单调矩阵每行最左最小值的位置。
约定：离线 n×m 全单调矩阵，每行求最左最小列。全单调要求每个子矩阵行最优列也单调；只有原矩阵行最优列单调不足以使用。get 为 O(1) oracle。
复杂度：$\mathcal{O}(n+m)$ oracle 调用
依赖：§1.1（递归收齐依赖）。

BlueLagoon（参考 cp-book）/ CC0-1.0 / 本地回归 / 对拍 05a61d

```
template <class F>
vector<int> smawk(int n, int m, F get) {
    assert(n >= 0 && (n == 0 || m >= 1));
    vector<int> rows(n), cols(m), ans(n, -1);
    iota(rows.begin(), rows.end(), 0);
    iota(cols.begin(), cols.end(), 0);
    auto rec = [&](auto self, const vector<int> &r,
                   const vector<int> &c) -> void {
        if (r.empty()) return;
        vector<int> st;
        for (int x : c) {
            while (!st.empty() &&
                   get(r[st.size() - 1], x) <
                       get(r[st.size() - 1], st.back()))
                st.pop_back();
            if (st.size() < r.size()) st.push_back(x);
        }
        vector<int> odd;
        for (int i = 1; i < (int)r.size(); i += 2)
            odd.push_back(r[i]);
        self(self, odd, st);
```

```
        int p = 0;
        for (int i = 0; i < (int)r.size(); i += 2) {
            int stop = i + 1 < (int)r.size()
                           ? ans[r[i + 1]]
                           : st.back();
            int best = st[p];
            while (true) {
                if (get(r[i], st[p]) < get(r[i], best))
                    best = st[p];
                if (st[p] == stop) break;
                p ++;
            }
            ans[r[i]] = best;
        }
    };
    rec(rec, rows, cols);
    return ans;
}
```

9.4 数值积分（自适应 Simpson）

功能：用递归分段的 Simpson 公式近似计算有限区间定积分。
约定：平滑函数有限区间积分，eps>0。depth 是递归上限，达到上限时误差目标不保证；不适合未经处理的奇点、剧烈振荡或狭窄尖峰。
复杂度：依赖函数与目标误差，最坏 $\mathcal{O}(2^{\text{depth}})$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 097cf3

```
template <class F>
double integrate(F f, double l, double r,
                 double eps = 1e-8, int depth = 25) {
    auto area = [](double l, double r, double a,
                   double b, double c) {
        return (r - l) * (a + 4 * b + c) / 6;
    };
    auto rec = [&](auto self, double l, double r,
                   double a, double b, double c,
                   double s, double eps,
                   int dep) -> double {
        double m = (l + r) / 2, d = f((l + m) / 2),
               e = f((m + r) / 2);
        double x = area(l, m, a, d, b),
               y = area(m, r, b, e, c), z = x + y - s;
        if (!dep || abs(z) <= 15 * eps)
            return x + y + z / 15;
        return self(self, l, m, a, d, b, x, eps / 2,
                    dep - 1) +
               self(self, m, r, b, e, c, y, eps / 2,
                    dep - 1);
    };
    double a = f(l), b = f((l + r) / 2), c = f(r);
    return rec(rec, l, r, a, b, c, area(l, r, a, b, c),
               eps, depth);
}
```

9.5 全覆盖循环序列（De Bruijn）

功能：构造循环序列，使每种长度 n 的字母串恰好出现一次。
约定：返回字母表 0..k-1、阶数 n 的循环序列，长度 k^n；线性展开时重复开头 n-1 个字符。输出规模须可存储，k=1 特判。
复杂度：$\mathcal{O}(k^n)$
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 903458

```
vector<int> de_bruijn(int k, int n) {
    assert(k >= 1 && n >= 1);
    if (k == 1) return { 0 };
    vector<int> a(n + 1), ans;
    auto dfs = [&](auto self, int t, int p) -> void {
        if (t > n) {
            if (n % p == 0)
                for (int i = 1; i <= p; i ++)
                    ans.push_back(a[i]);
            return;
        }
        a[t] = a[t - p];
        self(self, t + 1, p);
        for (int i = a[t - p] + 1; i < k; i ++)
            a[t] = i, self(self, t + 1, t);
    };
    dfs(dfs, 1, 1);
    return ans;
}
```

9.6 SG、Nim 与常见博弈

无环正常玩法：

$$\mathrm{SG}(x) = \mathrm{mex}\{\mathrm{SG}(y) : x \to y\}.$$

无合法操作时 $\mathrm{SG}(x) = 0$；独立子游戏的和对应 SG 异或。

SG 定理要求无环 / 终止且是无偏游戏；循环游戏可能存在平局，需反向拓扑判 win/lose/draw。

普通 Nim：所有堆大小异或非零则先手胜；取最后一个获胜。

Misère Nim（取最后一个输）：若全为 1，堆数偶数先手胜；否则按普通 Nim。

Wythoff：排序 $a \le b$，令 $k = b - a$；败态满足

$$a = \lfloor k\phi \rfloor, \qquad \phi = \frac{1+\sqrt{5}}{2}.$$

大整数须防浮点取整误差。

减法游戏先计算足够多 SG；观察周期只是猜测，需要证明或适用的周期定理。

有向无环图 SG：先求拓扑序，逆序计算，mex 使用时间戳清理。

9.7 组合与数学公式速查

Burnside：群 G 作用下的轨道数为

$$N = \frac{1}{|G|}\sum_{g\in G}\mathrm{fix}(g).$$

模运算除以 $|G|$ 要求可逆，否则使用整数或更高模数计算。

Pólya：置换 g 的长度 j 循环数为 $c_j(g)$，循环指标为

$$Z_G(x_1,x_2,\ldots) = \frac{1}{|G|}\sum_{g\in G}\prod_{j\ge 1}x_j^{c_j(g)}.$$

m 色染色时，各 x_j 代入 m。

Catalan：$C_0 = 1$，

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}.$$

第二类 Stirling：$S(0,0) = 1$，

$$S(n,k) = S(n-1,k-1) + kS(n-1,k).$$

第一类无符号 Stirling：$c(0,0) = 1$，

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k).$$

Bell：$B_n = \sum_{k=0}^{n} S(n,k)$。

错排：$D_0 = 1$，$D_1 = 0$，

$$D_n = (n-1)(D_{n-1} + D_{n-2}).$$

分拆数：$P_0 = 1$，负下标为 0。令广义五边形数 $g_k^{\pm} = k(3k \pm 1)/2$，则

$$P_n = \sum_{k\ge 1}(-1)^{k-1}(P_{n-g_k^-} + P_{n-g_k^+}).$$

Cayley：$n \ge 2$ 个有标号点的树为 n^{n-2}；Prüfer 码长度为 $n-2$。

Pick：简单格点多边形面积 $A = I + B/2 - 1$；有 h 个孔时 $A = I + B/2 - 1 + h$，边界计数包括所有边界。

格点线段的边界间隔数为 $\gcd(|\Delta x|, |\Delta y|)$；简单格点多边形满足

$$B = \sum_{\text{edges}}\gcd(|\Delta x|, |\Delta y|).$$

Euler：连通平面图 $V-E+F=2$（含外面）；c 个连通块时 $V-E+F=1+c$。

Knuth 充分条件：$a\le b\le c\le d$ 时，

$$w(b,c)\le w(a,d),$$
$$w(a,c)+w(b,d)\le w(a,d)+w(b,c).$$

Monge：以下不等式推出全单调与最小决策单调。

$$A(i,j)+A(i+1,j+1)$$
$$\le A(i,j+1)+A(i+1,j).$$

常用求和：

$$\sum_{i=1}^{n} i=\frac{n(n+1)}{2},\qquad \sum_{i=1}^{n} i^2=\frac{n(n+1)(2n+1)}{6}.$$

内存：`int[N]` 约 $4N$ 字节，`i64[N]` 约 $8N$；`vector` 有额外容量和对象开销。邻接表双向边算两份。

复杂度预算以考场机器测量为准；大常数、随机算法与哈希不能只看渐进式。

10 常用补充

10.1 二维树状数组 / 矩形求和

功能：预登记更新坐标后，动态加点权并查询二维前缀和 / 矩形和。
约定：0-based 点坐标任意 i64；先 reserve 所有可能更新的点，再 build。sum(x,y) 统计横纵坐标均严格小于 x,y 的点。重复 reserve 允许；不能更新未登记点。
调用：登记点 (1,2) 后 build；`f.add(1,2,5);`；`f.rect(0,0,2,3)` 得 5。
复杂度：$\mathcal{O}(log^2 n)$ 更新 / 查询，$\mathcal{O}(n\log n)$ 空间
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 c9da4f

```
struct Fenwick2D {
    vector<pair<i64, i64>> pts;
    vector<i64> xs;
    vector<vector<i64>> ys, tr;
    void reserve(i64 x, i64 y) {
        pts.emplace_back(x, y);
    }
    void build() {
        xs.clear();
        for (auto [x, y] : pts)
            xs.push_back(x);
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()),
                 xs.end());
        int n = xs.size();
        ys.assign(n + 1, {});
        tr.assign(n + 1, {});
        for (auto [x, y] : pts)
            for (int i = lower_bound(xs.begin(),
                                     xs.end(), x) -
                         xs.begin() + 1;
                 i <= n; i += i & -i)
                ys[i].push_back(y);
        for (int i = 1; i <= n; i ++) {
            sort(ys[i].begin(), ys[i].end());
            ys[i].erase(
                unique(ys[i].begin(), ys[i].end()),
                ys[i].end());
            tr[i].assign(ys[i].size() + 1, 0);
        }
    }
    void add(i64 x, i64 y, i64 v) {
        int k = lower_bound(xs.begin(), xs.end(), x) -
                xs.begin();
        assert(k < (int)xs.size() && xs[k] == x);
        for (int i = k + 1; i <= (int)xs.size();
             i += i & -i) {
            int j = lower_bound(ys[i].begin(),
                                ys[i].end(), y) -
                    ys[i].begin();
            assert(j < (int)ys[i].size() &&
                   ys[i][j] == y);
            for (j ++; j < (int)tr[i].size();
                 j += j & -j)
                tr[i][j] += v;
        }
    }
    i64 sum(i64 x, i64 y) const {
        i64 ans = 0;
        for (int i =
                 lower_bound(xs.begin(), xs.end(), x) -
                 xs.begin();
             i; i -= i & -i)
            for (int j = lower_bound(ys[i].begin(),
                                     ys[i].end(), y) -
                         ys[i].begin();
                 j; j -= j & -j)
                ans += tr[i][j];
        return ans;
    }
    i64 rect(i64 l, i64 d, i64 r, i64 u) const {
        return sum(r, u) - sum(l, u) - sum(r, d) +
               sum(l, d);
    }
};
```

10.2 莫队 / 静态区间查询

功能：重排离线区间查询，通过两端加入 / 删除元素维护答案。
约定：查询半开区间 [l,r)，id 必须为 0..q-1 的排列。初始统计为空；add/del 接收下标，read(id) 保存当前答案。仅适用于两端插删均可维护的状态。
调用：`qs={{0,3,0},{1,4,1}};`；`mo(n,qs,add,del,read);`；三个回调分别加入下标、删除下标、保存答案。
复杂度：$\mathcal{O}((n+q)\sqrt{n})$ 次状态操作
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 8c03e1

```
struct MoQuery {
    int l, r, id;
};
template <class Add, class Del, class Read>
void mo(int n, vector<MoQuery> qs, Add add, Del del,
        Read read) {
    int b = max(1, (int)sqrt(max(n, 1)));
    sort(qs.begin(), qs.end(), [&](auto a, auto c) {
        int x = a.l / b, y = c.l / b;
        if (x != y) return x < y;
        return x & 1 ? a.r > c.r : a.r < c.r;
    });
    int l = 0, r = 0;
    for (auto q : qs) {
        assert(0 <= q.l && q.l <= q.r && q.r <= n);
        while (l > q.l)
            add(--l);
        while (r < q.r)
            add(r ++);
        while (l < q.l)
            del(l ++);
        while (r > q.r)
            del(--r);
        read(q.id);
    }
}
```

10.3 倍增祖先 / 路径第 k 点

功能：求某点的第 k 级祖先，以及两点路径上指定偏移的顶点。
约定：以 HLD 的父亲 / 深度初始化；kth_path 的 k 为从 u 开始的 0-based 偏移，越界返回 -1。jump 不允许越过根。
调用：`BinaryLifting b(h);`；`b.kth_path(u,v,0)` 为 u，`b.jump(v,1)` 为父亲（根返回-1）。
复杂度：$\mathcal{O}(n\log n)$ 预处理，$\mathcal{O}(\log n)$ 查询
依赖：§1.1, §2.10（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 294ec9

```cpp
struct BinaryLifting {
    vector<vector<int>> up;
    vector<int> dep;
    BinaryLifting(const HLD &h) : dep(h.dep) {
        int n = h.n, lg = 1;
        while ((1LL << lg) <= n)
            lg ++;
        up.assign(lg, vector<int>(n));
        for (int v = 0; v < n; v ++)
            up[0][v] = h.p[v] < 0 ? v : h.p[v];
        for (int j = 1; j < lg; j ++)
            for (int v = 0; v < n; v ++)
                up[j][v] = up[j - 1][up[j - 1][v]];
    }
    int jump(int v, int k) const {
        if (k < 0 || k > dep[v]) return -1;
        for (int j = 0; k; k >>= 1, j ++)
            if (k & 1) v = up[j][v];
        return v;
    }
    int lca(int u, int v) const {
        if (dep[u] < dep[v]) swap(u, v);
        u = jump(u, dep[u] - dep[v]);
        if (u == v) return u;
        for (int j = (int)up.size() - 1; j >= 0; j --)
            if (up[j][u] != up[j][v])
                u = up[j][u], v = up[j][v];
        return up[0][u];
    }
    int kth_path(int u, int v, int k) const {
        int w = lca(u, v), a = dep[u] - dep[w],
            b = dep[v] - dep[w];
        if (k < 0 || k > a + b) return -1;
        return k <= a ? jump(u, k) : jump(v, a + b - k);
    }
};
```

10.4 模矩阵乘幂与实数 / 模矩阵求逆

功能：计算模矩阵乘法 / 快速幂，以及实数 / 素数模矩阵逆。
约定：模运算要求 1<mod<=INT_MAX，系数归一。inverse_mod 要求素数；奇异返回 nullopt。实数版 eps 按量纲调整；空矩阵逆为空。乘幂只接受方阵。
调用：`matrix::power({{1,1},{1,0}},10,mod)` 的 [0][1]为 F(10)；`matrix::inverse_mod(a,mod)` 先检查 optional。
复杂度：乘法 / 逆 $\mathcal{O}(n^3)$，幂 $\mathcal{O}(n^3 \log k)$
依赖：§1.1, §6.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 0f62d7

```cpp
namespace matrix {
using Mat = vector<vector<int>>;
Mat identity(int n, int mod) {
    Mat a(n, vector<int>(n));
    for (int i = 0; i < n; i ++)
        a[i][i] = 1 % mod;
    return a;
}
Mat multiply(const Mat &a, const Mat &b, int mod) {
    int n = a.size(), k = b.size(),
        m = k ? b[0].size() : 0;
    assert(!n || (int)a[0].size() == k);
    Mat c(n, vector<int>(m));
    for (int i = 0; i < n; i ++)
        for (int j = 0; j < k; j ++)
            if (a[i][j])
                for (int t = 0; t < m; t ++)
                    c[i][t] = (c[i][t] +
                               i64(a[i][j]) * b[j][t]) %
                              mod;
    return c;
}
Mat power(Mat a, u64 k, int mod) {
    int n = a.size();
    for (auto &r : a)
        assert((int)r.size() == n);
    Mat b = identity(n, mod);
    for (; k; k >>= 1, a = multiply(a, a, mod))
        if (k & 1) b = multiply(b, a, mod);
    return b;
}
optional<Mat> inverse_mod(Mat a, int mod) {
    int n = a.size();
    Mat b = identity(n, mod);
    for (int i = 0; i < n; i ++) {
        int p = i;
        while (p < n && !a[p][i])
            p ++;
        if (p == n) return nullopt;
        swap(a[p], a[i]);
        swap(b[p], b[i]);
        int x = pow_mod(a[i][i], mod - 2, mod);
        for (int j = 0; j < n; j ++)
            a[i][j] = i64(a[i][j]) * x % mod,
            b[i][j] = i64(b[i][j]) * x % mod;
        for (int k = 0; k < n; k ++)
            if (k != i) {
                int x = a[k][i];
                for (int j = 0; j < n; j ++) {
                    a[k][j] =
                        (a[k][j] -
                         i64(x) * a[i][j] % mod + mod) %
                        mod;
                    b[k][j] =
                        (b[k][j] -
                         i64(x) * b[i][j] % mod + mod) %
                        mod;
                }
            }
    }
    return b;
}
optional<vector<vector<double>>>
inverse_real(vector<vector<double>> a,
             double eps = 1e-9) {
    int n = a.size();
    vector<vector<double>> b(n, vector<double>(n));
    for (int i = 0; i < n; i ++)
        b[i][i] = 1;
    for (int i = 0; i < n; i ++) {
        int p = i;
        for (int k = i; k < n; k ++)
            if (abs(a[k][i]) > abs(a[p][i])) p = k;
        if (abs(a[p][i]) <= eps) return nullopt;
        swap(a[p], a[i]);
        swap(b[p], b[i]);
        double x = a[i][i];
        for (int j = 0; j < n; j ++)
            a[i][j] /= x, b[i][j] /= x;
        for (int k = 0; k < n; k ++)
            if (k != i) {
                double x = a[k][i];
                for (int j = 0; j < n; j ++)
                    a[k][j] -= x * a[i][j],
                    b[k][j] -= x * b[i][j];
            }
    }
    return b;
}
} // namespace matrix
```

10.5 异或方程组 / 解空间（GF(2)）

功能：求二元域线性方程组的一个特解与全部自由方向。
约定：m 个变量，下标 0..m-1，右侧为第 m 位；每行长度 ceil((m+1)/64)。返回可行性、一个特解与齐次解空间基，所有解为特解 XOR 基的任意组合。
调用：方程 x0 XOR x1=1 编码为 `{{7}}`；`gauss_binary({{7}},2)` 返回特解与一个自由方向。
复杂度：$\mathcal{O}(nm\,\mathrm{ceil}((m+1)/64))$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 b1bfdf

```cpp
struct BinarySolution {
```

```
    bool ok;
    int rank;
    vector<int> value;
    vector<vector<int>> basis;
};
BinarySolution gauss_binary(vector<vector<u64>> a,
                            int m) {
    int n = a.size(), w = (m + 64) / 64, r = 0;
    for (auto &v : a)
        assert((int)v.size() == w);
    auto bit = [&](int i, int j) {
        return (a[i][j / 64] >> (j % 64)) & 1;
    };
    vector<int> where(m, -1);
    for (int j = 0; j < m; j ++) {
        int p = r;
        while (p < n && !bit(p, j))
            p ++;
        if (p == n) continue;
        swap(a[p], a[r]);
        where[j] = r;
        for (int i = 0; i < n; i ++)
            if (i != r && bit(i, j))
                for (int k = 0; k < w; k ++)
                    a[i][k] ^= a[r][k];
        r ++;
    }
    for (int i = r; i < n; i ++)
        if (bit(i, m)) return { false, r, {}, {} };
    BinarySolution out{ true, r, vector<int>(m), {} };
    for (int j = 0; j < m; j ++)
        if (where[j] != -1)
            out.value[j] = bit(where[j], m);
        else {
            vector<int> v(m);
            v[j] = 1;
            for (int k = 0; k < m; k ++)
                if (where[k] != -1)
                    v[k] = bit(where[k], j);
            out.basis.push_back(v);
        }
    return out;
}
```

10.6 最长递增子序列长度 / 恢复（LIS）

功能：恢复一条最长严格递增或非降子序列的原下标。
约定：返回一条子序列的原下标；strict=true 严格递增，false 非降。空输入返回空。
调用：`lis(vector<int>{3,1,2})` 返回下标 {1,2}；非降版本另传 false。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 34783c

```
template <class T>
vector<int> lis(const vector<T> &a,
                bool strict = true) {
    vector<T> val;
    vector<int> last, pre(a.size(), -1);
    for (int i = 0; i < (int)a.size(); i ++) {
        int k =
            (strict ? lower_bound(val.begin(),
                                  val.end(), a[i])
                    : upper_bound(val.begin(),
                                  val.end(), a[i])) -
            val.begin();
        if (k) pre[i] = last[k - 1];
        if (k == (int)val.size())
            val.push_back(a[i]), last.push_back(i);
        else
            val[k] = a[i], last[k] = i;
    }
    vector<int> ans;
    if (!last.empty())
        for (int i = last.back(); i != -1; i = pre[i])
            ans.push_back(i);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

10.7 整数分拆数 / 两类斯特林数

功能：计算整数无序分拆数，以及两类斯特林数的三角表。
约定：任意正 int 模数；partition 返回 p(0)..p(n)。stirling 的 first=true 表示第一类无符号，否则第二类；返回三角表，空位为 0。
调用：`partition_table(5,mod)[5]` 为 7；`stirling_table(3,mod)[3][2]` 为 3。
复杂度：分拆 $\mathcal{O}(n\sqrt{n})$；Stirling $\mathcal{O}(n^2)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 7ff094

```
vector<int> partition_table(int n, int mod) {
    vector<int> p(n + 1);
    p[0] = 1 % mod;
    for (int i = 1; i <= n; i ++)
        for (i64 k = 1; k * (3 * k - 1) / 2 <= i; k ++) {
            int sign = k & 1 ? 1 : -1;
            for (i64 g : { k * (3 * k - 1) / 2,
                           k * (3 * k + 1) / 2 })
                if (g <= i)
                    p[i] =
                        (i64(p[i]) +
                         sign * i64(p[i - g]) + mod) %
                        mod;
        }
    return p;
}
vector<vector<int>> stirling_table(int n, int mod,
                                   bool first = false) {
    vector<vector<int>> s(n + 1, vector<int>(n + 1));
    s[0][0] = 1 % mod;
    for (int i = 1; i <= n; i ++)
        for (int j = 1; j <= i; j ++)
            s[i][j] =
                (s[i - 1][j - 1] +
                 i64(first ? i - 1 : j) * s[i - 1][j]) %
                mod;
    return s;
}
```

10.8 fread 快读快写

功能：用 fread / fwrite 缓冲读写整数，支持 128 位整数及 EOF 检测。
约定：批处理 IO；不要与 cin/scanf 混用。read 整数要求输入在目标类型范围内，EOF 返回 false；支持有符号最小值和 i128。对象含 64KiB 缓冲；输出析构时 flush，交互题显式 flush。
调用：`FastInput in; i64 x;; if(in.read(x)) out.write(x);`，out 为 FastOutput；EOF 时 read 返回 false。
复杂度：$\mathcal{O}$(字符数)
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 e15f37

```
struct FastInput {
    static const int B = 1 << 16;
    FILE *f;
    char buf[B];
    int p = 0, n = 0;
    FastInput(FILE *f = stdin) : f(f) {}
    int get() {
        if (p == n) n = fread(buf, 1, B, f), p = 0;
        return n ? (unsigned char)buf[p ++] : EOF;
    }
    template <class T> bool read(T &x) {
        int c;
        do {
            c = get();
            if (c == EOF) return false;
        } while (c <= 32);
        bool neg = c == '-';
        if (neg || c == '+') c = get();
```

```
        u128 v = 0;
        assert(c >= '0' && c <= '9');
        while (c >= '0' && c <= '9')
            v = v * 10 + c - '0', c = get();
        x = neg ? T(-i128(v - 1)) - 1 : T(v);
        return true;
    }
};
struct FastOutput {
    static const int B = 1 << 16;
    FILE *f;
    char buf[B];
    int n = 0;
    FastOutput(FILE *f = stdout) : f(f) {}
    ~FastOutput() { flush(); }
    void flush() {
        if (n) fwrite(buf, 1, n, f), n = 0;
    }
    void put(char c) {
        if (n == B) flush();
        buf[n ++] = c;
    }
    template <class T>
    void write(T x, char end = '\n') {
        u128 v;
        if (x < 0)
            put('-'), v = u128(-(i128(x) + 1)) + 1;
        else
            v = x;
        char s[50];
        int k = 0;
        do {
            s[k ++] = char('0' + v % 10);
            v /= 10;
        } while (v);
        while (k)
            put(s[--k]);
        if (end) put(end);
    }
};
```

10.9 区间内分母最小的分数

功能：求严格位于给定非负有理数区间内、分母最小的分数。
约定：输入非负有理数 L<R，分母为正，均用 i64；求严格位于 (L, R) 内、分母最小（并列分子最小）的分数。分子 / 分母越过 i64 时返回 nullopt；内部用 i128。
调用：`simplest_fraction(1,3,1,2)` 得 (2,5)，端点均不允许。
复杂度：$\mathcal{O}(\log M)$ 次整除，M 为结果量级
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 dd614a

```
optional<pair<i64, i64>>
simplest_fraction(i64 a, i64 b, i64 c, i64 d) {
    assert(a >= 0 && c >= 0 && b > 0 && d > 0 &&
           i128(a) * d < i128(c) * b);
    i128 l = 0, q = 1, r = 1, s = 0;
    while (true) {
        i128 x = l + r, y = q + s;
        if (x > LLONG_MAX || y > LLONG_MAX)
            return nullopt;
        if (x * b <= i128(a) * y) {
            i128 k = (i128(a) * q - i128(b) * l) /
                     (i128(b) * r - i128(a) * s);
            l += k * r;
            q += k * s;
        } else if (x * d >= i128(c) * y) {
            i128 k = (i128(d) * r - i128(c) * s) /
                     (i128(c) * q - i128(d) * l);
            r += k * l;
            s += k * q;
        } else
            return pair<i64, i64>{ (i64)x, (i64)y };
    }
}
```

10.10 多边形直线裁剪与重心

功能：按直线裁剪凸多边形，并计算简单多边形的面积重心。
约定：cut 输入凸多边形；center 输入简单多边形，均按边界顺序给出。cut 保留有向直线 a→b 左侧，含边界；凸多边形输入时输出单个凸多边形。center 返回面积重心，零面积返回 nullopt。
调用：`polygon_cut(p,a,b)` 保留 a→b 左边；`polygon_center(p)` 为面积重心，非顶点均值。
复杂度：$\mathcal{O}(n)$
依赖：§1.1, §8.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 00ebb9

```
vector<Point<double>>
polygon_cut(const vector<Point<double>> &p,
            Point<double> a, Point<double> b,
            double eps = 1e-9) {
    vector<Point<double>> out;
    int n = p.size();
    for (int i = 0; i < n; i ++) {
        auto u = p[i], v = p[(i + 1) % n];
        double x = (b - a).cross(u - a),
               y = (b - a).cross(v - a);
        if (x >= -eps) out.push_back(u);
        if ((x > eps && y < -eps) ||
            (x < -eps && y > eps))
            out.push_back(u + (v - u) * (x / (x - y)));
    }
    return out;
}
optional<Point<double>>
polygon_center(const vector<Point<double>> &p,
               double eps = 1e-9) {
    Point<double> ans;
    double area = 0;
    int n = p.size();
    for (int i = 0; i < n; i ++) {
        auto a = p[i], b = p[(i + 1) % n];
        double w = a.cross(b);
        area += w;
        ans = ans + (a + b) * w;
    }
    if (abs(area) <= eps) return nullopt;
    return ans / (3 * area);
}
```

10.11 曼哈顿距离最小生成树候选边

功能：生成线性数量的候选边，配合 Kruskal 求点集曼哈顿距离 MST。
约定：0-based 整数点，可重复；返回 (距离,u,v)，至多 4n 条。对结果跑 Kruskal 得完整图 Manhattan MST。坐标变换、距离、比较中间值不得溢出 i64。
调用：`auto ed=manhattanMST(points);`；ed 元素为 (距离,u,v)，再按距离排序跑 Kruskal。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 0bf55b

```
typedef Point<i64> P;
vector<tuple<i64, int, int>>
manhattanMST(vector<P> ps) {
    vector<int> id(int(ps.size()));
    iota(id.begin(), id.end(), 0);
    vector<tuple<i64, int, int>> edges;
    for (int k = 0; k < 4; k ++) {
        sort(id.begin(), id.end(), [&](int i, int j) {
            return (ps[i] - ps[j]).x <
                   (ps[j] - ps[i]).y;
        });
        map<i64, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-ps[i].y);
                 it != sweep.end(); sweep.erase(it ++)) {
                int j = it->second;
                P d = ps[i] - ps[j];
                if (d.y > d.x) break;
```

```
                edges.push_back({ d.y + d.x, i, j });
            }
            sweep[-ps[i].y] = i;
        }
        for (P &p : ps)
            if (k & 1)
                p.x = -p.x;
            else
                swap(p.x, p.y);
    }
    return edges;
}
```

11 进阶附录

11.1 有理函数第 k 项 / 快速递推（Bostan-Mori）

功能：求有理生成函数的指定系数，也可加速常系数线性递推取项。
约定：输入系数均归一到 [0,998244353)。模 998244353，Q[0]非零；返回 [x^k]P/Q，k 为 u64。linear_rec_fast 的 c[i] 表示 a[n]=Σc[i]a[n-1-i]；init 长度等于阶数。
调用：`poly::bostan_mori({0,1},{1,mod-1,mod-1},10)` 得 55；即 $\frac{x}{1-x-x^2}$。
复杂度：$\mathcal{O}(d \log d \log k)$
依赖：§1.1, §7.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 d4da33

```
namespace poly {
int bostan_mori(Vec p, Vec q, u64 k) {
    assert(!q.empty() && q[0]);
    if (p.empty()) return 0;
    while (k) {
        Vec qm = q;
        for (int i = 1; i < (int)qm.size(); i += 2)
            qm[i] = qm[i] ? mod - qm[i] : 0;
        Vec u = conv(p, qm), v = conv(q, qm), np, nq;
        for (int i = k & 1; i < (int)u.size(); i += 2)
            np.push_back(u[i]);
        for (int i = 0; i < (int)v.size(); i += 2)
            nq.push_back(v[i]);
        p = move(np);
        q = move(nq);
        k >>= 1;
    }
    return p.empty()
               ? 0
               : i64(p[0]) * power(q[0], mod - 2) % mod;
}
int linear_rec_fast(const Vec &init, const Vec &c,
                    u64 k) {
    int d = c.size();
    assert((int)init.size() == d);
    if (!d) return 0;
    Vec q(d + 1);
    q[0] = 1;
    for (int i = 0; i < d; i ++)
        q[i + 1] = c[i] ? mod - c[i] : 0;
    Vec p = conv(init, q);
    p.resize(d);
    return bostan_mori(p, q, k);
}
} // namespace poly
```

11.2 一侧固定的在线卷积

功能：固定一侧系数，另一侧逐项输入时立即返回当前卷积系数。
约定：输入系数均归一到 [0,998244353)。模 998244353。构造时固定核 a，声明输入长度 n；push(x) 依次输入 b[k]，立即返回 $\sum_{i=0}^{k} a_i b_{k-i}$。无未来项访问；k<n。不是双侧均在线的 relaxed 卷积。
调用：`poly::OnlineConvolution c({1,2},3);`；依次 push(3),push(4),push(5) 得 3,10,13。
复杂度：总 $\mathcal{O}(nlog^2n)$，空间 $\mathcal{O}(n)$
依赖：§1.1, §7.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 cd59d8

```
namespace poly {
struct OnlineConvolution {
    Vec a, b, c;
    int n;
    OnlineConvolution(Vec a, int n)
        : a(move(a)), c(n), n(n) {
        if ((int)this->a.size() > n) this->a.resize(n);
    }
    int push(int x) {
        int k = b.size();
        assert(k < n);
        b.push_back(x);
        if (!a.empty())
            c[k] = (c[k] + i64(a[0]) * x) % mod;
        for (int z = 1; ((k + 1) & (z - 1)) == 0 &&
                        z < (int)a.size() && k + 1 < n;
             z <<= 1) {
            int r = min(2 * z, (int)a.size());
            Vec u(a.begin() + z, a.begin() + r),
                v(b.end() - z, b.end());
            Vec w = conv(u, v);
            for (int j = 0;
                 j < (int)w.size() && k + 1 + j < n;
                 j ++)
                c[k + 1 + j] =
                    (c[k + 1 + j] + i64(w[j])) % mod;
        }
        return c[k];
    }
};
} // namespace poly
```

11.3 形式幂级数复合 / 多项式平移

功能：计算 f(g(x)) 的截断系数，以及多项式 f(x+c) 的系数。
约定：输入系数均归一到 [0,998244353)。模 998244353，compose 返回 f(g) modx^n，要求 g[0]=0；当前为分块版本，不冒充准线性实现。taylor_shift 返回 f(x+c)，长度小于模数，卷积满足 NTT 上限。
调　用：`poly::compose({1,2,3},{0,1,1},4)` 得 {1,2,5,6}；`taylor_shift({1,2,3},1)` 得 {6,8,3}。
复杂度：复合 $\mathcal{O}(n^2 + n\sqrt{n}\log n)$，空间 $\mathcal{O}(n\sqrt{n})$；平移 $\mathcal{O}(n \log n)$
依赖：§1.1, §7.3（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 e731e6

```
namespace poly {
Vec compose(const Vec &f, const Vec &g, int n) {
    assert(n >= 0 && (g.empty() || g[0] == 0));
    if (!n) return {};
    Vec base(g.begin(),
             g.begin() + min(n, (int)g.size()));
    int b = max(1, (int)sqrt(n));
    vector<Vec> pw(b + 1, Vec(n));
    pw[0][0] = 1;
    for (int j = 1; j <= b; j ++)
        pw[j] = cut(conv(pw[j - 1], base), n);
    Vec ans(n);
    int blocks = (min(n, (int)f.size()) + b - 1) / b;
    for (int t = blocks - 1; t >= 0; t --) {
        ans = cut(conv(ans, pw[b]), n);
        for (int j = 0;
             j < b && t * b + j < min(n, (int)f.size());
             j ++)
            for (int i = 0; i < n; i ++)
                ans[i] = (ans[i] + i64(f[t * b + j]) *
                                   pw[j][i]) %
                         mod;
    }
    return ans;
}
Vec taylor_shift(const Vec &a, int c) {
    int n = a.size();
    if (!n) return {};
    assert(n < mod);
    Vec fac(n, 1), inv(n, 1), u(n), v(n);
    for (int i = 1; i < n; i ++)
        fac[i] = i64(fac[i - 1]) * i % mod;
    inv[n - 1] = power(fac[n - 1], mod - 2);
```

```
    for (int i = n - 1; i; i --)
        inv[i - 1] = i64(inv[i]) * i % mod;
    int p = 1;
    for (int i = 0; i < n; i ++)
        u[n - 1 - i] = i64(a[i]) * fac[i] % mod,
                  v[i] = i64(p) * inv[i] % mod,
                  p = i64(p) * c % mod;
    Vec w = conv(u, v), out(n);
    for (int i = 0; i < n; i ++)
        out[i] = i64(w[n - 1 - i]) * inv[i] % mod;
    return out;
}
} // namespace poly
```

11.4 无权拟阵交

功能：在两个无权拟阵中求元素个数最多的公共独立集。
约定：地集 0..n-1；oracle 接收元素下标列表，返回是否独立。两个 oracle 必须确实对应拟阵；输出最大公共独立集。不支持带权；oracle 不得修改外部状态。
调用：均匀拟阵例：`auto a=[](auto s){return s.size()<=2;};` `matroid_intersection(5,a,a)` 返回 2 个元素。实题替换独立性 oracle。
复杂度：每次增广 $\mathcal{O}(n^2)$ 次 oracle，至多 n 次
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 bdbd5d

```
template <class A, class B>
vector<int> matroid_intersection(int n, A a, B b) {
    vector<int> in(n);
    for (;;) {
        vector<int> cur;
        for (int i = 0; i < n; i ++)
            if (in[i]) cur.push_back(i);
        vector<int> pre(n, -1);
        queue<int> q;
        int end = -1;
        auto trial = [&](int add, int del) {
            vector<int> s;
            for (int x : cur)
                if (x != del) s.push_back(x);
            if (add >= 0) s.push_back(add);
            return s;
        };
        for (int i = 0; i < n; i ++)
            if (!in[i] && a(trial(i, -1)))
                pre[i] = -2, q.push(i);
        while (!q.empty() && end < 0) {
            int x = q.front();
            q.pop();
            if (!in[x]) {
                if (b(trial(x, -1))) {
                    end = x;
                    break;
                }
                for (int y : cur)
                    if (pre[y] == -1 && b(trial(x, y)))
                        pre[y] = x, q.push(y);
            } else
                for (int y = 0; y < n; y ++)
                    if (!in[y] && pre[y] == -1 &&
                        a(trial(y, x)))
                        pre[y] = x, q.push(y);
        }
        if (end < 0) return cur;
        for (int x = end; x >= 0; x = pre[x])
            in[x] ^= 1;
    }
}
```

11.5 素数模离散 k 次根

功能：求模素数幂方程的一个 k 次根，或判断无解。
约定：p 为素数且 <=LLONG_MAX，k>=1，返回任意一个 x 满足 $x^k \equiv a \pmod p$；无解 nullopt。BSGS 的 sqrt(p) 内存是实际限制；不是合数模开根。
调用：`kth_root(4,2,7)` 返回 2 或 5；`kth_root(3,2,7)` 无解。
复杂度：期望 $\mathcal{O}(\sqrt{p})$，另含 p-1 分解
依赖：§1.1, §6.5, §6.4（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 7a37dd

```
optional<u64> kth_root(u64 a, u64 k, u64 p) {
    assert(k >= 1 && p <= LLONG_MAX && is_prime(p));
    a %= p;
    if (a == 0 || p == 2) return a;
    u64 g = primitive_root(p), m = p - 1;
    auto z = bsgs(g, a, p);
    if (!z) return nullopt;
    u64 d = gcd(k, m);
    if (*z % d) return nullopt;
    m /= d;
    u64 t = m == 1
                ? 0
                : mul_mod(*z / d,
                          inverse((k / d) % m, m), m);
    return pow_mod(g, t, p);
}
```

11.6 正整数幂塔取模

功能：求右结合正整数幂塔模任意正模数的值，支持不互质底数。
约定：a 非空且每个底数 >=1，mod 正且 <=LLONG_MAX。返回 a[0]^(a[1]^(...)) mod mod；支持不互质，用扩展欧拉及精确截断判断。零底数未定义于此接口。
调用：`tower::tetration({2,3,2},1000)` 为 512；括号为右结合：2^{3^2}。
复杂度：欧拉链 $\mathcal{O}(\log \mathrm{mod})$ 层；每层含整数分解
依赖：§1.1, §6.3（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 c3732b

```
namespace tower {
u64 phi(u64 m) {
    vector<u64> ps;
    factor(m, ps);
    sort(ps.begin(), ps.end());
    ps.erase(unique(ps.begin(), ps.end()), ps.end());
    for (u64 p : ps)
        m = m / p * (p - 1);
    return m;
}
u64 cap(const vector<u64> &a, int i, u64 lim) {
    if (lim <= 1) return lim;
    if (i == (int)a.size() - 1) return min(a[i], lim);
    if (a[i] == 1) return 1;
    u64 v = 1, need = 0;
    while (v < lim) {
        need ++;
        v = u128(v) * a[i] >= lim ? lim : v * a[i];
    }
    u64 e = cap(a, i + 1, need);
    v = 1;
    while (e -- && v < lim)
        v = u128(v) * a[i] >= lim ? lim : v * a[i];
    return v;
}
u64 calc(const vector<u64> &a, int i, u64 mod) {
    if (mod == 1) return 0;
    if (i == (int)a.size() - 1) return a[i] % mod;
    u64 p = phi(mod), e = calc(a, i + 1, p);
    if (cap(a, i + 1, p) >= p) e += p;
    return pow_mod(a[i] % mod, e, mod);
}
u64 tetration(const vector<u64> &a, u64 mod) {
    assert(!a.empty() && mod >= 1 && mod <= LLONG_MAX);
    for (u64 x : a)
        assert(x >= 1);
    return calc(a, 0, mod);
}
} // namespace tower
```

11.7 有上下界最小费用环流（允许负环）

功能：求满足流量上下界的最低费用环流，允许有限容量负费用环。
约定：0-based，有限整数容量 0<=lo<=hi。先求可行流，再消残量负环；solve 返回 i128 费用或 nullopt，flow 为各原边流量。允许自环和平行边。必须检查总容量 / 余额不超过 i64。消环版适合中小规模，不替代大规模 cost scaling。
调　　用：`MinCostCirculation f(1);; f.add(0,0,0,3,-2);; f.solve()` 为-6，`f.flow[0]` 为 3。
复杂度：可行性 Dinic；优化 $\mathcal{O}(Cnm)$，C 为消环次数，整数费用时伪多项式
依赖：§1.1, §4.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 a13ceb

```
struct MinCostCirculation {
    struct Edge {
        int u, v;
        i64 lo, hi, cost;
    };
    struct Arc {
        int u, v;
        i64 cap, cost;
    };
    int n;
    vector<Edge> ed;
    vector<i64> flow;
    MinCostCirculation(int n) : n(n) {}
    void add(int u, int v, i64 lo, i64 hi, i64 cost) {
        assert(0 <= lo && lo <= hi &&
               cost != LLONG_MIN);
        ed.push_back({ u, v, lo, hi, cost });
    }
    optional<i128> solve() {
        Dinic d(n + 2);
        vector<i64> bal(n);
        vector<int> id;
        for (auto e : ed) {
            id.push_back(d.add(e.u, e.v, e.hi - e.lo));
            bal[e.u] -= e.lo;
            bal[e.v] += e.lo;
        }
        i64 need = 0;
        for (int v = 0; v < n; v ++)
            if (bal[v] > 0)
                d.add(n, v, bal[v]), need += bal[v];
            else if (bal[v] < 0)
                d.add(v, n + 1, -bal[v]);
        if (d.flow(n, n + 1, need) != need)
            return nullopt;
        vector<Arc> a;
        flow.resize(ed.size());
        for (int i = 0; i < (int)ed.size(); i ++) {
            auto e = ed[i];
            i64 f = d.used(e.u, id[i]);
            a.push_back(
                { e.u, e.v, e.hi - e.lo - f, e.cost });
            a.push_back({ e.v, e.u, f, -e.cost });
        }
        for (;;) {
            vector<i128> dist(n);
            vector<int> pre(n, -1);
            int x = -1;
            for (int k = 0; k < n; k ++) {
                x = -1;
                for (int j = 0; j < (int)a.size();
                     j ++) {
                    auto e = a[j];
                    if (e.cap &&
                        dist[e.v] > dist[e.u] + e.cost)
                        dist[e.v] = dist[e.u] + e.cost,
                        pre[e.v] = j, x = e.v;
                }
                if (x < 0) break;
            }
            if (x < 0) break;
            for (int k = 0; k < n; k ++)
                x = a[pre[x]].u;
            int v = x;
            vector<int> cyc;
            i64 f = LLONG_MAX;
            do {
                int j = pre[v];
                cyc.push_back(j);
                f = min(f, a[j].cap);
                v = a[j].u;
            } while (v != x);
            for (int j : cyc)
                a[j].cap -= f, a[j ^ 1].cap += f;
        }
        i128 cost = 0;
        for (int i = 0; i < (int)ed.size(); i ++)
            flow[i] = ed[i].lo + a[2 * i + 1].cap,
            cost += i128(flow[i]) * ed[i].cost;
        return cost;
    }
};
```

11.8 K 短路 / K 最短游走（Eppstein）

功能：求非负权有向图前 k 条最短游走费用，允许重复顶点和边。
约定：非负 i64 边权；允许重复点、重复边，平行边视为不同选择。返回按费用排序的至多 k 条游走；s=t 包含空游走；零环可产生无限多条同费用结果。距离与返回费用必须 <LLONG_MAX/4。
调用：`KShortest g(2);`；add(0,1,2)、add(1,1,1) 后 `g.solve(0,1,3)` 得 {2,3,4}。
复杂度：$\mathcal{O}((n+m)\log n + m\log n + k\log k)$，持久堆空间 $\mathcal{O}(m\log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 1a2a9a

```
struct KShortest {
    struct Edge {
        int u, v;
        i64 w;
    };
    struct Node {
        int l = 0, r = 0, rank = 1, to = 0;
        i64 w = 0;
    };
    int n;
    vector<Edge> ed;
    vector<Node> tr;
    KShortest(int n) : n(n) {}
    void add(int u, int v, i64 w) {
        assert(w >= 0);
        ed.push_back({ u, v, w });
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (tr[a].w > tr[b].w) swap(a, b);
        int c = tr.size();
        tr.push_back(tr[a]);
        int r = merge(tr[c].r, b);
        tr[c].r = r;
        if (tr[tr[c].l].rank < tr[r].rank)
            swap(tr[c].l, tr[c].r);
        tr[c].rank = tr[tr[c].r].rank + 1;
        return c;
    }
    vector<i64> solve(int s, int t, int k) {
        const i64 inf = LLONG_MAX / 4;
        vector<vector<int>> rev(n), out(n);
        for (int i = 0; i < (int)ed.size(); i ++)
            rev[ed[i].v].push_back(i),
                out[ed[i].u].push_back(i);
        vector<i64> d(n, inf);
        vector<int> parent(n, -1), order, h(n);
        priority_queue<pair<i64, int>,
                       vector<pair<i64, int>>,
                       greater<pair<i64, int>>>
            q;
        d[t] = 0;
        q.push({ 0, t });
        while (!q.empty()) {
            auto [x, u] = q.top();
```

```
            q.pop();
            if (x != d[u]) continue;
            order.push_back(u);
            for (int j : rev[u]) {
                auto e = ed[j];
                if (e.w < inf - x && d[e.u] > x + e.w)
                    d[e.u] = x + e.w, parent[e.u] = j,
                    q.push({ d[e.u], e.u });
            }
        }
        if (k <= 0 || d[s] == inf) return {};
        tr.assign(1, {});
        tr[0].rank = 0;
        for (int u : order) {
            if (parent[u] >= 0)
                h[u] = h[ed[parent[u]].v];
            for (int j : out[u])
                if (j != parent[u] &&
                    d[ed[j].v] < inf) {
                    auto e = ed[j];
                    i128 delta =
                        i128(e.w) + d[e.v] - d[u];
                    assert(0 <= delta && delta < inf);
                    int a = tr.size();
                    tr.push_back(
                        { 0, 0, 1, e.v, (i64)delta });
                    h[u] = merge(h[u], a);
                }
        }
        vector<i64> ans{ d[s] };
        priority_queue<pair<i64, int>,
                       vector<pair<i64, int>>,
                       greater<pair<i64, int>>>
            pq;
        auto push = [&](i128 x, int v) {
            if (v) {
                assert(x < inf);
                pq.push({ (i64)x, v });
            }
        };
        push(i128(d[s]) + (h[s] ? tr[h[s]].w : 0),
             h[s]);
        while ((int)ans.size() < k && !pq.empty()) {
            auto [x, u] = pq.top();
            pq.pop();
            ans.push_back(x);
            for (int v : { tr[u].l, tr[u].r })
                if (v)
                    push(i128(x) - tr[u].w + tr[v].w,
                         v);
            int v = h[tr[u].to];
            if (v) push(i128(x) + tr[v].w, v);
        }
        return ans;
    }
};
```

11.9 字符串极大周期段（runs）

功能：枚举无法向左右延伸、长度至少两倍最小周期的周期段。
约定：输入小写字母；返回 (最小周期,l,r)，区间 [l,r)，长度 >=2p，左右不能在该周期下延伸。基于两种字典序 Lyndon 根及前后 LCP；结果去重。
调用：`enumerate_runs("ababab")` 含 {2,0,6}；输出最小周期与半开区间。
复杂度：$\mathcal{O}(n \log n)$ 时间 / 空间
依赖：§1.1, §5.6（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 b54c45

```
vector<array<int, 3>> enumerate_runs(string s) {
    int n = s.size();
    if (!n) return {};
    SuffixLCP fw(s);
    string rev = s;
    reverse(rev.begin(), rev.end());
    SuffixLCP bw(rev);
    vector<pair<int, int>> intervals;
    for (int pass = 0; pass < 2; pass ++) {
        SuffixArray sa(s);
        vector<int> rank(n + 1);
        for (int i = 0; i <= n; i ++)
            rank[sa.sa[i]] = i;
        vector<int> st{ n };
        for (int i = n - 1; i >= 0; i --) {
            while (rank[st.back()] > rank[i])
                st.pop_back();
            intervals.emplace_back(i, st.back());
            st.push_back(i);
        }
        for (char &c : s)
            c = 'a' + 'z' - c;
    }
    vector<array<int, 3>> ans;
    for (auto [x, y] : intervals) {
        int p = y - x, l = x - bw.lcp(n - x, n - y),
            r = y + fw.lcp(x, y);
        if (r - l >= 2 * p) ans.push_back({ p, l, r });
    }
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(), ans.end()),
              ans.end());
    return ans;
}
```

11.10 一般图 Δ+1 边染色

功能：用不超过 Δ+1 种颜色给一般简单图边染色。
约定：简单无向图，0-based，无自环 / 平行边。输出按输入边顺序的颜色，范围 0..Δ。不能用于一般多重图。
调用：`edgeColoring(3,{{0,1},{1,2},{2,0}})` 返回三种颜色，顺序对应输入边。
复杂度：$\mathcal{O}(nm)$
依赖：§1.1（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 987355

```
vector<int> edgeColoring(int N,
                         vector<pair<int, int>> eds) {
    vector<int> cc(N + 1), ret(int(eds.size())), fan(N),
        free(N), loc;
    for (pair<int, int> e : eds)
        ++cc[e.first], ++cc[e.second];
    int u, v,
        ncols = *max_element(cc.begin(), cc.end()) + 1;
    vector<vector<int>> adj(N, vector<int>(ncols, -1));
    for (pair<int, int> e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0,
            i = 0;
        while (d = free[v],
               !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1;
             cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd],
                 adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i],
                e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : { fan[0], u, end })
            for (int &z = free[y] = 0; adj[y][z] != -1;
                 z ++)
                ;
    }
```

```
    for (int i = 0; i < int(eds.size()); i ++)
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;)
            ++ret[i];
    return ret;
}
```

11.11 弦图识别与完美消除序

功能：判断简单图是否为弦图，是则返回完美消除序。
约定：非空简单无向图；两端邻接均存在。getPEO 返回消除序，无弦图返回空；后继邻点必须成团。
调用：`getPEO({{1,2},{0,2},{0,1}})` 返回三角形的某个消除序；无弦四环返回空。
复杂度：$\mathcal{O}(m \log n + m \log n)$（邻接排序 / 检查）
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 bcaf38

```
// given simple, nonempty graph
// in O(m log n), returns permutation of vertices
// (positive) or empty vector (negative) such that for
// p[i], all j > i such that p[j] is adjacent to p[i],
// constitutes cliques. for each undir edge (s, e), both
// s \in gph[e], e \in gph[s] should hold
vector<int> getPEO(vector<vector<int>> gph) {
    int n = int(gph.size());
    vector<int> cnt(n), idx(n);
    for (int i = 0; i < n; i ++)
        sort(gph[i].begin(), gph[i].end());
    priority_queue<pair<int, int>> pq;
    for (int i = 0; i < n; i ++)
        pq.emplace(cnt[i], i);
    vector<int> ord;
    while (!pq.empty()) {
        int x = pq.top().second, y = pq.top().first;
        pq.pop();
        if (cnt[x] != y || idx[x]) continue;
        ord.push_back(x);
        idx[x] = n + 1 - ord.size();
        for (auto &i : gph[x]) {
            if (!idx[i]) {
                cnt[i] ++;
                pq.emplace(cnt[i], i);
            }
        }
    }
    reverse(ord.begin(), ord.end());
    for (auto &i : ord) {
        int minBef = 1e9;
        for (auto &j : gph[i]) {
            if (idx[j] > idx[i])
                minBef = min(minBef, idx[j]);
        }
        minBef --;
        if (minBef < n) {
            minBef = ord[minBef];
            for (auto &j : gph[i]) {
                if (idx[j] > idx[minBef] &&
                    !binary_search(gph[minBef].begin(),
                                   gph[minBef].end(),
                                   j)) {
                    return vector<int>();
                }
            }
        }
    }
    return ord;
}
```

11.12 Nim 积 / 64 位 nimber 乘法

功能：计算 nimber 的乘法，配合异或加法进行 Nim 数运算。
约定：第一次使用先 nimbers::init()；product 的入参显式转换 u64，避免重载歧义。加法为 XOR；普通整数乘法与 nimber 乘法不同。
调用：`nimbers::init()`；后 `nimbers::product(u64(2),u64(2))` 为 3。
复杂度：初始化 $\mathcal{O}(2^{16})$，每次 $\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 c5638c

```
typedef unsigned short u16;
typedef unsigned int u32;
typedef unsigned long long u64;

namespace nimbers {
constexpr u32 n2f[16] = { 0x0001u, 0x071cu, 0x6bd1u,
                          0x1224u, 0x6ba8u, 0x1333u,
                          0x1553u, 0x0007u, 0x071eu,
                          0x0925u, 0xc586u, 0x5dbdu,
                          0xc463u, 0x5efdu, 0x2aa1u,
                          0x155au },
              f2n[16] = { 0x0001u, 0x0102u, 0x0183u,
                          0x8041u, 0x015cu, 0x5f24u,
                          0xde2cu, 0x957eu, 0x01f4u,
                          0xf7d8u, 0x76b0u, 0x5d52u,
                          0xa977u, 0x20d1u, 0xc1a4u,
                          0x271fu };
inline u32 nimber2field(u32 x) {
    u32 y = 0;
    for (; x; x &= x - 1)
        y ^= n2f[__builtin_ctz(x)];
    return y;
}
inline u32 field2nimber(u32 x) {
    u32 y = 0;
    for (; x; x &= x - 1)
        y ^= f2n[__builtin_ctz(x)];
    return y;
}
inline u32 __builtin_double(u32 x) {
    return x << 1 ^ (x < 32768 ? 0 : 0x16babu);
}

u16 ln[65536], exp[196605], *Hexp = exp + 62133,
                            *H2exp = exp + 58731;

inline void init() {
    int i;
    for (*exp = i = 1; i < 65535; ++i)
        exp[i] = __builtin_double(exp[i - 1]);
    for (i = 1; i < 65535; ++i)
        exp[i] = field2nimber(exp[i]), ln[exp[i]] = i;
    memcpy(exp + 65535, exp, 131070);
    memcpy(exp + 131070, exp, 131070);
}

inline u16 product(u16 A, u16 B) {
    return A && B ? exp[ln[A] + ln[B]] : 0;
}
inline u16 H(u16 A) {
    return A ? Hexp[ln[A]] : 0;
}
inline u16 H2(u16 A) {
    return A ? H2exp[ln[A]] : 0;
}
inline u16 Hproduct(u16 A, u16 B) {
    return A && B ? Hexp[ln[A] + ln[B]] : 0;
}

inline u32 product(u32 A, u32 B) {
    u16 a = A & 65535, b = B & 65535, c = A >> 16,
        d = B >> 16, e = product(a, b);
    return u32(product(u16(a ^ c), u16(b ^ d)) ^ e)
                << 16 |
           (Hproduct(c, d) ^ e);
}

inline u32 H(u32 A) {
    u16 a = A & 65535, b = A >> 16;
    return H(u16(a ^ b)) << 16 | H2(b);
}

inline u64 product(u64 A, u64 B) {
    u32 a = A & UINT_MAX, b = B & UINT_MAX, c = A >> 32,
        d = B >> 32, e = product(a, b);
```

```
    return u64(product(a ^ c, b ^ d) ^ e) << 32 |
           (H(product(c, d)) ^ e);
}
} // namespace nimbers
```

11.13 一般图最大权匹配（带权带花树）

功能：求允许点不匹配的一般图最大总权匹配，并恢复配对。
约定：namespace weighted_blossom，1-based，0<=n<=500。init 后 add_edge 正权边；零 / 负权边可忽略，因为允许点不匹配。solve 返回 (总权,边数)，match[u]为伙伴。不是最大权完美匹配或优先最大基数；总权 / 对偶中间值需不溢出 i64。全局工作区约数十 MiB，单实例。
调用：先 `weighted_blossom::init(3);`;add_edge(1,2,5)、add_edge(2,3,7) 后 solve 返回 (7,1)。
复杂度：$\mathcal{O}(n^3)$，空间 $\mathcal{O}(n^2)$
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 147b8d

```
namespace weighted_blossom {
// N^3 (but fast in practice)
static const i64 INF = LLONG_MAX / 4;
static const int N = 514;
struct edge {
    int u, v;
    i64 w;
    edge() {}
    edge(int ui, int vi, i64 wi)
        : u(ui), v(vi), w(wi) {}
};
int n, n_x;
edge g[N * 2][N * 2];
i64 lab[N * 2];
int match[N * 2], slack[N * 2], st[N * 2], pa[N * 2];
int flo_from[N * 2][N + 1], S[N * 2], vis[N * 2];
vector<int> flo[N * 2];
queue<int> q;
i64 e_delta(const edge &e) {
    return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2;
}
void update_slack(int u, int x) {
    if (!slack[x] ||
        e_delta(g[u][x]) < e_delta(g[slack[x]][x]))
        slack[x] = u;
}
void set_slack(int x) {
    slack[x] = 0;
    for (int u = 1; u <= n; ++u)
        if (g[u][x].w > 0 && st[u] != x &&
            S[st[u]] == 0)
            update_slack(u, x);
}
void q_push(int x) {
    if (x <= n)
        q.push(x);
    else
        for (size_t i = 0; i < flo[x].size(); i ++)
            q_push(flo[x][i]);
}
void set_st(int x, int b) {
    st[x] = b;
    if (x > n)
        for (size_t i = 0; i < flo[x].size(); ++i)
            set_st(flo[x][i], b);
}
int get_pr(int b, int xr) {
    int pr = find(flo[b].begin(), flo[b].end(), xr) -
             flo[b].begin();
    if (pr % 2 == 1) {
        reverse(flo[b].begin() + 1, flo[b].end());
        return (int)flo[b].size() - pr;
    } else
        return pr;
}
void set_match(int u, int v) {
    match[u] = g[u][v].v;
    if (u <= n) return;
    edge e = g[u][v];
    int xr = flo_from[u][e.u], pr = get_pr(u, xr);
    for (int i = 0; i < pr; ++i)
        set_match(flo[u][i], flo[u][i ^ 1]);
    set_match(xr, v);
    rotate(flo[u].begin(), flo[u].begin() + pr,
           flo[u].end());
}
void augment(int u, int v) {
    for (;;) {
        int xnv = st[match[u]];
        set_match(u, v);
        if (!xnv) return;
        set_match(xnv, st[pa[xnv]]);
        u = st[pa[xnv]], v = xnv;
    }
}
int get_lca(int u, int v) {
    static int t = 0;
    for (++t; u || v; swap(u, v)) {
        if (u == 0) continue;
        if (vis[u] == t) return u;
        vis[u] = t;
        u = st[match[u]];
        if (u) u = st[pa[u]];
    }
    return 0;
}
void add_blossom(int u, int lca, int v) {
    int b = n + 1;
    while (b <= n_x && st[b])
        ++b;
    if (b > n_x) ++n_x;
    lab[b] = 0, S[b] = 0;
    match[b] = match[lca];
    flo[b].clear();
    flo[b].push_back(lca);
    for (int x = u, y; x != lca; x = st[pa[y]])
        flo[b].push_back(x),
            flo[b].push_back(y = st[match[x]]),
            q_push(y);
    reverse(flo[b].begin() + 1, flo[b].end());
    for (int x = v, y; x != lca; x = st[pa[y]])
        flo[b].push_back(x),
            flo[b].push_back(y = st[match[x]]),
            q_push(y);
    set_st(b, b);
    for (int x = 1; x <= n_x; ++x)
        g[b][x].w = g[x][b].w = 0;
    for (int x = 1; x <= n; ++x)
        flo_from[b][x] = 0;
    for (size_t i = 0; i < flo[b].size(); ++i) {
        int xs = flo[b][i];
        for (int x = 1; x <= n_x; ++x)
            if (g[b][x].w == 0 ||
                e_delta(g[xs][x]) < e_delta(g[b][x]))
                g[b][x] = g[xs][x], g[x][b] = g[x][xs];
        for (int x = 1; x <= n; ++x)
            if (flo_from[xs][x]) flo_from[b][x] = xs;
    }
    set_slack(b);
}
void expand_blossom(int b) {
    for (size_t i = 0; i < flo[b].size(); ++i)
        set_st(flo[b][i], flo[b][i]);
    int xr = flo_from[b][g[b][pa[b]].u],
        pr = get_pr(b, xr);
    for (int i = 0; i < pr; i += 2) {
        int xs = flo[b][i], xns = flo[b][i + 1];
        pa[xs] = g[xns][xs].u;
        S[xs] = 1, S[xns] = 0;
        slack[xs] = 0, set_slack(xns);
        q_push(xns);
    }
    S[xr] = 1, pa[xr] = pa[b];
    for (size_t i = pr + 1; i < flo[b].size(); ++i) {
        int xs = flo[b][i];
```

```
        S[xs] = -1, set_slack(xs);
    }
    st[b] = 0;
}
bool on_found_edge(const edge &e) {
    int u = st[e.u], v = st[e.v];
    if (S[v] == -1) {
        pa[v] = e.u, S[v] = 1;
        int nu = st[match[v]];
        slack[v] = slack[nu] = 0;
        S[nu] = 0, q_push(nu);
    } else if (S[v] == 0) {
        int lca = get_lca(u, v);
        if (!lca)
            return augment(u, v), augment(v, u), true;
        else
            add_blossom(u, lca, v);
    }
    return false;
}
bool matching() {
    memset(S + 1, -1, sizeof(int) * n_x);
    memset(slack + 1, 0, sizeof(int) * n_x);
    q = queue<int>();
    for (int x = 1; x <= n_x; ++x)
        if (st[x] == x && !match[x])
            pa[x] = 0, S[x] = 0, q_push(x);
    if (q.empty()) return false;
    for (;;) {
        while (q.size()) {
            int u = q.front();
            q.pop();
            if (S[st[u]] == 1) continue;
            for (int v = 1; v <= n; ++v)
                if (g[u][v].w > 0 && st[u] != st[v]) {
                    if (e_delta(g[u][v]) == 0) {
                        if (on_found_edge(g[u][v]))
                            return true;
                    } else
                        update_slack(u, st[v]);
                }
        }
        i64 d = INF;
        for (int b = n + 1; b <= n_x; ++b)
            if (st[b] == b && S[b] == 1)
                d = min(d, lab[b] / 2);
        for (int x = 1; x <= n_x; ++x)
            if (st[x] == x && slack[x]) {
                if (S[x] == -1)
                    d = min(d, e_delta(g[slack[x]][x]));
                else if (S[x] == 0)
                    d = min(d, e_delta(g[slack[x]][x]) /
                                    2);
            }
        for (int u = 1; u <= n; ++u) {
            if (S[st[u]] == 0) {
                if (lab[u] <= d) return 0;
                lab[u] -= d;
            } else if (S[st[u]] == 1)
                lab[u] += d;
        }
        for (int b = n + 1; b <= n_x; ++b)
            if (st[b] == b) {
                if (S[st[b]] == 0)
                    lab[b] += d * 2;
                else if (S[st[b]] == 1)
                    lab[b] -= d * 2;
            }
        q = queue<int>();
        for (int x = 1; x <= n_x; ++x)
            if (st[x] == x && slack[x] &&
                st[slack[x]] != x &&
                e_delta(g[slack[x]][x]) == 0)
                if (on_found_edge(g[slack[x]][x]))
                    return true;
        for (int b = n + 1; b <= n_x; ++b)
            if (st[b] == b && S[b] == 1 && lab[b] == 0)
                expand_blossom(b);
    }
    return false;
}
pair<long long, int> solve() {
    if (!n) return { 0, 0 };
    memset(match + 1, 0, sizeof(int) * n);
    n_x = n;
    int n_matches = 0;
    long long tot_weight = 0;
    for (int u = 0; u <= n; ++u)
        st[u] = u, flo[u].clear();
    i64 w_max = 0;
    for (int u = 1; u <= n; ++u)
        for (int v = 1; v <= n; ++v) {
            flo_from[u][v] = (u == v ? u : 0);
            w_max = max(w_max, g[u][v].w);
        }
    for (int u = 1; u <= n; ++u)
        lab[u] = w_max;
    while (matching())
        ++n_matches;
    for (int u = 1; u <= n; ++u)
        if (match[u] && match[u] < u)
            tot_weight += g[u][match[u]].w;
    return make_pair(tot_weight, n_matches);
}
void add_edge(int ui, int vi, i64 wi) {
    assert(ui != vi && wi > 0 && wi < INF / 4);
    g[ui][vi].w = g[vi][ui].w = max(wi, g[ui][vi].w);
}
void init(int _n) {
    assert(0 <= _n && _n < N);
    n = _n;
    for (int u = 1; u <= n; ++u)
        for (int v = 1; v <= n; ++v)
            g[u][v] = edge(u, v, 0);
}
} // namespace weighted_blossom
```

11.14 实系数多项式乘法 / 卷积（FFT）

功能：作复数快速傅里叶变换，近似计算实系数多项式卷积。
约定：fft 长度为正 2 幂；inverse=true 逆变换并除以 n。convolution 支持实系数，浮点误差依数值规模而变；整数精确计算优先 NTT / CRT。
调用：`real_fft::convolution({1,2},{3,4})` 约为 {3,10,8}；不要直接强制转整数，应按精度条件选择 llround 或 NTT。
复杂度：$\mathcal{O}(n \log n)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 0d3ebb

```
namespace real_fft {
using C = complex<double>;
void fft(vector<C> &a, bool inverse = false) {
    int n = a.size();
    assert(n > 0 && (n & (n - 1)) == 0);
    for (int i = 1, j = 0; i < n; i ++) {
        int b = n >> 1;
        for (; j & b; b >>= 1)
            j ^= b;
        j ^= b;
        if (i < j) swap(a[i], a[j]);
    }
    for (int k = 2; k <= n; k <<= 1) {
        C z = polar(1.0,
                    (inverse ? -2 : 2) * acos(-1) / k);
        for (int i = 0; i < n; i += k) {
            C w = 1;
            for (int j = 0; j < k / 2; j ++, w *= z) {
                C u = a[i + j],
                  v = w * a[i + j + k / 2];
                a[i + j] = u + v;
                a[i + j + k / 2] = u - v;
            }
        }
    }
```

```
    if (inverse)
        for (C &x : a)
            x /= n;
}
vector<double> convolution(const vector<double> &a,
                           const vector<double> &b) {
    if (a.empty() || b.empty()) return {};
    int m = a.size() + b.size() - 1, n = 1;
    while (n < m)
        n <<= 1;
    vector<C> x(n), y(n);
    copy(a.begin(), a.end(), x.begin());
    copy(b.begin(), b.end(), y.begin());
    fft(x);
    fft(y);
    for (int i = 0; i < n; i ++)
        x[i] *= y[i];
    fft(x, true);
    vector<double> out(m);
    for (int i = 0; i < m; i ++)
        out[i] = x[i].real();
    return out;
}
} // namespace real_fft
```

11.15 有向最小生成树 / 选边恢复（快速版）

功能：求指定根的最小树形图，并恢复父亲和原输入边编号。
约定：namespace arborescence；0-based，dmst(n,root,edges) 返回 cost, parent,edge 或 nullopt。edge[v] 是原输入边编号，根为-1；允许负权 / 平行边。权值、对偶偏移、总权不得溢出 i64；堆递归注意栈。
调用：`arborescence::dmst(3,0,{{0,1,4},{1,2,2},{0,2,9}})` 总权 6，edge 为 {-1,0,1}；检查 optional。
复杂度：$\mathcal{O}(m \log n)$，空间 $\mathcal{O}(n+m)$
依赖：§1.1, §2.8（递归收齐依赖）。

KACTL / CC0 / 本地回归 / 对拍 a28f57

```
namespace arborescence {

struct Edge {
    int a, b;
    i64 w;
    int id = -1;
};
struct Node { /// lazy skew heap node
    Edge key;
    Node *l, *r;
    i64 delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() {
        prop();
        return key;
    }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? a : b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node *&a) {
    a->prop();
    a = merge(a->l, a->r);
}

struct Result {
    i64 cost;
    vector<int> parent, edge;
};
optional<Result> dmst(int n, int r, vector<Edge> g) {
    RollbackUF uf(n);
    vector<Node *> heap(n);
    vector<unique_ptr<Node>> pool;
    for (int j = 0; j < int(g.size()); j ++) {
        Edge e = g[j];
        e.id = j;
        pool.emplace_back(
            new Node{ e, nullptr, nullptr, 0 });
        heap[e.b] = merge(heap[e.b], pool.back().get());
    }
    i64 res = 0;
    vector<int> seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, { -1, -1, 0, -1 });
    deque<tuple<int, int, vector<Edge>>> cycs;
    for (int s = 0; s < n; s ++) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return nullopt;
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi ++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] ==
                s) { /// found cycle, contract
                Node *cyc = 0;
                int end = qi, time = uf.time();
                do
                    cyc = merge(cyc,
                                heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc,
                seen[u] = -1;
                cycs.push_front(
                    { u, time, { &Q[qi], &Q[end] } });
            }
        }
        for (int i = 0; i < qi; i ++)
            in[uf.find(Q[i].b)] = Q[i];
    }

    for (auto &[u, t, comp] :
         cycs) { // restore sol (optional)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto &e : comp)
            in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    for (int i = 0; i < n; i ++)
        par[i] = in[i].a;
    vector<int> ids(n);
    for (int i = 0; i < n; i ++)
        ids[i] = in[i].id;
    return Result{ res, par, ids };
}

} // namespace arborescence
```

11.16 Delaunay 三角剖分

功能：对二维整数点集求 Delaunay 三角剖分，输出逆时针三角形。
约定：namespace delaunay，triangulate(vector<Point<i64>>) 返回按每 3 点分组的逆时针三角形。点不可重复，坐标绝对值 <=10^9；共线返回空，共圆选择任意合法对角线。谓词使用 i128；结果为点坐标，不是点编号。工作区单线程。
调用：三点 (0,0),(2,0),(0,2) 调用 `delaunay::triangulate(p)` 返回一个逆时针三角形，共 3 个点。
复杂度：$\mathcal{O}(n \log n)$ 时间，$\mathcal{O}(n)$ 空间
依赖：§1.1, §8.1（递归收齐依赖）。

KACTL / MIT / 本地回归 / 对拍 5d23d1

```
namespace delaunay {

typedef Point<i64> P;
typedef struct Quad *Q;
typedef __int128_t
```

```
    lll; // (can be i64 if coords are < 2e4)
P arb(LLONG_MAX,
      LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot, o;
    P p = arb;
    bool mark;
    P &F() { return r()->p; }
    Q &r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
};
Q H = nullptr;
vector<unique_ptr<Quad>> pool;

bool circ(P p, P a, P b,
          P c) { // is p in the circumcircle?
    lll p2 = p.dist2(), A = a.dist2() - p2,
        B = b.dist2() - p2, C = c.dist2() - p2;
    return p.cross(a, b) * C + p.cross(b, c) * A +
           p.cross(c, a) * B >
           0;
}
Q makeEdge(P orig, P dest) {
    Q r = H;
    if (!r) {
        Q v[4];
        for (int i = 0; i < 4; i ++) {
            pool.emplace_back(new Quad{});
            v[i] = pool.back().get();
        }
        for (int i = 0; i < 4; i ++)
            v[i]->rot = v[(i + 1) % 4];
        r = v[0];
    }
    H = r->o;
    r->r()->r() = r;
    for (int i = 0; i < 4; i ++)
        r = r->rot, r->mark = false, r->p = arb,
        r->o = i & 1 ? r : r->r();
    r->p = orig;
    r->F() = dest;
    return r;
}
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o);
    swap(a->o, b->o);
}
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q, Q> rec(const vector<P> &s) {
    if (int(s.size()) <= 3) {
        Q a = makeEdge(s[0], s[1]),
          b = makeEdge(s[1], s.back());
        if (int(s.size()) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return { side < 0 ? c->r() : a,
                 side < 0 ? c : b->r() };
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = int(s.size()) / 2;
    tie(ra, A) =
        rec(vector<P>(s.begin(), s.end() - half));
    tie(B, rb) = rec(vector<P>(
        s.begin() + int(s.size()) - half, s.end()));
    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
           (A->p.cross(H(B)) > 0 && (B = B->r()->o)))
        ;
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir)                                   \
    Q e = init->dir;                                        \
    if (valid(e))                                           \
        while (circ(e->dir->F(), H(base), e->F())) {        \
            Q t = e->dir;                                   \
            splice(e, e->prev());                           \
            splice(e->r(), e->r()->prev());                 \
            e->o = H;                                       \
            H = e;                                          \
            e = t;                                          \
        }
    for (;;) {
        DEL(LC, base->r(), o);
        DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) ||
            (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
    pool.clear();
    H = nullptr;
    for (auto p : pts)
        assert(abs(p.x) <= 1000000000LL &&
               abs(p.y) <= 1000000000LL);
    sort(pts.begin(), pts.end());
    assert(unique(pts.begin(), pts.end()) == pts.end());
    if (int(pts.size()) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = { e };
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0)
        e = e->o;
#define ADD                                                 \
    {                                                       \
        Q c = e;                                            \
        do {                                                \
            c->mark = 1;                                    \
            pts.push_back(c->p);                            \
            q.push_back(c->r());                            \
            c = c->next();                                  \
        } while (c != e);                                   \
    }
    ADD;
    pts.clear();
    while (qi < int(q.size()))
        if (!(e = q[qi ++])->mark) ADD;
    pool.clear();
    H = nullptr;
    return pts;
}

#undef H
#undef valid
#undef DEL
#undef ADD

} // namespace delaunay
```

11.17 后缀树 / 最长公共子串（Ukkonen）

功能：构建压缩后缀树，并提供两串最长公共子串查询。
约定：输入 a..z 及两个保留符号 {、|（编码 26/27）；完整后缀树须追加唯一终止符。边区间 [l,r)，根 0，节点 1 是辅助节点。SuffixTree::LCS(s,t) 的原串必须仅小写，返回 (最长公共子串长度,在拼接串中的一个起点)。lcs 递归注意深链栈。
调用：`SuffixTree st("aba{");` 是完整树；`SuffixTree::LCS("abac","bac")` 的 first 为 3。
复杂度：$\mathcal{O}(28n)$ 时间 / 空间
依赖：§1.1（递归收齐依赖）。

KACTL / Not specified upstream; retain attribution / 本地回归 / 对拍 9e5d81

```
struct SuffixTree {
    enum { ALPHA = 28 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    vector<array<int, ALPHA>> t;
    vector<int> l, r, p, s;
    int v = 0, q = 0, m = 2;

    void ukkadd(int i, int c) {
    suff:
        if (r[v] <= q) {
            if (t[v][c] == -1) {
                t[v][c] = m;
                l[m] = i;
                p[m ++] = v;
                v = s[v];
                q = r[v];
                goto suff;
            }
            v = t[v][c];
            q = l[v];
        }
        if (q == -1 || c == toi(a[q]))
            q ++;
        else {
            l[m + 1] = i;
            p[m + 1] = m;
            l[m] = l[v];
            r[m] = q;
            p[m] = p[v];
            t[m][c] = m + 1;
            t[m][toi(a[q])] = v;
            l[v] = q;
            p[v] = m;
            t[p[m]][toi(a[l[m]])] = m;
            v = s[p[m]];
            q = l[m];
            while (q < r[m]) {
                v = t[v][toi(a[q])];
                q += r[v] - l[v];
            }
            if (q == r[m])
                s[m] = v;
            else
                s[m] = m + 2;
            q = r[v] - (q - r[m]);
            m += 2;
            goto suff;
        }
    }

    SuffixTree(string a)
        : a(a), t(2 * a.size() + 5), l(t.size()),
          r(t.size(), a.size()), p(t.size()),
          s(t.size()) {
        for (char c : a)
            assert(0 <= toi(c) && toi(c) < ALPHA);

        for (auto &v : t)
            v.fill(-1);
        t[1].fill(0);
        s[0] = 1;
        l[0] = l[1] = -1;
        r[0] = r[1] = p[0] = p[1] = 0;
        for (int i = 0; i < int(a.size()); i ++)
            ukkadd(i, toi(a[i]));
    }

    // example: find longest common substring (uses
    // ALPHA = 28)
    pair<int, int> best{};
    int lcs(int node, int i1, int i2, int olen) {
        if (l[node] <= i1 && i1 < r[node]) return 1;
        if (l[node] <= i2 && i2 < r[node]) return 2;
        int mask = 0,
            len = node ? olen + (r[node] - l[node]) : 0;
        for (int c = 0; c < ALPHA; c ++)
            if (t[node][c] != -1)
                mask |= lcs(t[node][c], i1, i2, len);
        if (mask == 3)
            best = max(best, { len, r[node] - len });
        return mask;
    }
    static pair<int, int> LCS(string s, string t) {
        SuffixTree st(s + (char)('z' + 1) + t +
                      (char)('z' + 2));
        st.lcs(0, int(s.size()),
               int(s.size()) + 1 + int(t.size()), 0);
        return st.best;
    }
};
```

11.18 离线动态最小生成树（边权修改）

功能：对固定连通图的离线边权修改，求每次修改后的 MST 总权。
约定：固定的连通无向图，0-based，允许自环 / 平行边。updates 为 (边编号,新权)，返回每次修改后的 MST 总权；不支持直接加删边。构造保存初始权，solve 可重复调用。i64 总权不得溢出。
调　　用：`DynamicMST d(3,{{0,1,1},{1,2,2},{0,2,5}});` `d.solve({{2,0}})` 得 {1}。
复杂度：典型 $\mathcal{O}((m+q)\log m \log q)$，回滚与排序常数较大
依赖：§1.1, §2.8（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 2b6e46

```
struct DynamicMST {
    struct Edge {
        int u, v;
        i64 w;
    };
    int n;
    vector<Edge> ed;
    vector<pair<int, i64>> qr;
    vector<int> cnt;
    vector<i64> w, ans;
    RollbackUF uf;
    DynamicMST(int n, vector<Edge> ed)
        : n(n), ed(move(ed)), uf(n) {}
    void contract(int l, int r, vector<int> v,
                  vector<int> &must,
                  vector<int> &maybe) {
        sort(v.begin(), v.end(), [&](int a, int b) {
            return tie(w[a], a) < tie(w[b], b);
        });
        int t = uf.time();
        for (int i = l; i <= r; i ++) {
            auto e = ed[qr[i].first];
            uf.join(e.u, e.v);
        }
        for (int i : v)
            if (uf.join(ed[i].u, ed[i].v))
                must.push_back(i);
        uf.rollback(t);
        for (int i : must)
            uf.join(ed[i].u, ed[i].v);
        for (int i : v)
            if (uf.join(ed[i].u, ed[i].v))
                maybe.push_back(i);
        uf.rollback(t);
    }
    void rec(int l, int r, vector<int> v, i64 base) {
        if (l == r) {
```

```
            int x = qr[l].first;
            w[x] = qr[l].second;
            if (uf.find(ed[x].u) == uf.find(ed[x].v))
                ans[l] = base;
            else {
                i64 best = w[x];
                for (int i : v)
                    best = min(best, w[i]);
                ans[l] = base + best;
            }
            return;
        }
        int mid = (l + r) / 2;
        vector<int> a = v, b = v, must, maybe;
        for (int i = mid + 1; i <= r; i ++)
            if (!--cnt[qr[i].first])
                a.push_back(qr[i].first);
        contract(l, mid, a, must, maybe);
        int t = uf.time();
        i64 cost = base;
        for (int i : must)
            cost += w[i], uf.join(ed[i].u, ed[i].v);
        rec(l, mid, maybe, cost);
        uf.rollback(t);
        for (int i = mid + 1; i <= r; i ++)
            cnt[qr[i].first] ++;
        for (int i = l; i <= mid; i ++)
            if (!--cnt[qr[i].first])
                b.push_back(qr[i].first);
        must.clear();
        maybe.clear();
        contract(mid + 1, r, b, must, maybe);
        cost = base;
        for (int i : must)
            cost += w[i], uf.join(ed[i].u, ed[i].v);
        rec(mid + 1, r, maybe, cost);
        uf.rollback(t);
        for (int i = l; i <= mid; i ++)
            cnt[qr[i].first] ++;
    }
    vector<i64> solve(vector<pair<int, i64>> updates) {
        qr = move(updates);
        ans.assign(qr.size(), 0);
        if (qr.empty()) return ans;
        uf = RollbackUF(n);
        for (auto e : ed)
            uf.join(e.u, e.v);
        assert(n >= 1 && uf.size(0) == n);
        uf.rollback(0);
        w.clear();
        for (auto e : ed)
            w.push_back(e.w);
        cnt.assign(ed.size(), 0);
        for (auto [i, x] : qr) {
            assert(0 <= i && i < (int)ed.size());
            cnt[i] ++;
        }
        vector<int> v;
        for (int i = 0; i < (int)ed.size(); i ++)
            if (!cnt[i]) v.push_back(i);
        rec(0, qr.size() - 1, v, 0);
        return ans;
    }
};
```

11.19 反对称矩阵 Pfaffian

功能：计算偶数阶反对称矩阵的 Pfaffian，其平方等于行列式。
约定：偶数阶反对称矩阵，系数归一到 [0,p)，p 为奇素数。返回 pf(A)，满足 $\mathrm{pf}(A)^2 = \det(A)$；空矩阵返回 1。不是直接的整数匹配计数器。
调用：`pfaffian({{0,3},{98,0}},101)` 为 3；平方等于该矩阵行列式。
复杂度：$\mathcal{O}(n^3)$
依赖：§1.1, §6.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 9c752e

```
int pfaffian(vector<vector<int>> a, int p) {
    int n = a.size();
    assert(n % 2 == 0 && p > 2);
    i64 ans = 1;
    for (int i = 0; i < n; i += 2) {
        int j = i + 1;
        while (j < n && !a[i][j])
            j ++;
        if (j == n) return 0;
        if (j != i + 1) {
            swap(a[j], a[i + 1]);
            for (auto &r : a)
                swap(r[j], r[i + 1]);
            ans = (p - ans) % p;
        }
        int v = a[i][i + 1], inv = pow_mod(v, p - 2, p);
        ans = ans * v % p;
        for (int x = i + 2; x < n; x ++)
            for (int y = x + 1; y < n; y ++) {
                i64 z = (i64(a[i][y]) * a[i + 1][x] -
                         i64(a[i][x]) * a[i + 1][y]) %
                        p * inv % p;
                a[x][y] = (a[x][y] + z + p) % p;
                a[y][x] = a[x][y] ? p - a[x][y] : 0;
            }
    }
    return ans;
}
```

11.20 多项式系数递推猜测与延拓

功能：由样本猜测多项式系数线性递推，并据此延拓后续项。
约 定：namespace p_recursive。find_recurrence_relation(terms,deg,prime,verify) 猜测 $\sum_i c_i(n-i)a_{n-i} = 0$；extended 延拓。样本必须足够、首项多项式在延拓点可逆，否则 assert。拟合成功不证明无限序列规律；务必留独立验证项。
调用：阶乘样本 {1,1,2,6,24,120}、deg=1 可猜测；`extended` 需留足初项。约定为 $\sum_i c_i(n-i)a_{n-i} = 0$。
复杂度：消元约 $\mathcal{O}(N^3)$，延拓 $\mathcal{O}(Nrd)$
依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 91adc2

```
namespace p_recursive {

int mod_inv(int a, int mod) {
    int b = mod, s = 1, t = 0, old_a = a;
    while (b) {
        int q = a / b;
        swap(a %= b, b);
        swap(s -= t * q, t);
    }
    if (a != 1) {
        fprintf(
            stderr,
            "Error: %d^{-1} mod %d does not exist.\n\n",
            old_a, mod);
        assert(0);
    }
    return s < 0 ? s + mod : s;
}

vector<int> extended(int n,
                     const vector<vector<int>> &coeffs,
                     const vector<int> &terms,
                     int mod) {

    vector<int> ret(max<int>(n + 1, terms.size()));
    copy(terms.begin(), terms.end(), ret.begin());
    const int order = coeffs.size() - 1;
    const int deg = coeffs[0].size() - 1;
    assert((int)terms.size() >= order);
    for (int m = terms.size(); m <= n; ++m) {
        int s = 0;
        for (int i = 1; i <= order; ++i) {
            int k = m - i, t = ret[k];
            for (int d = 0; d <= deg; ++d) {
```

```
                s = (s + i64(t) * coeffs[i][d]) % mod;
                t = i64(t) * k % mod;
            }
        }
        int denom = 0, mpow = 1;
        for (int d = 0; d <= deg; ++d) {
            denom = (denom + i64(mpow) * coeffs[0][d]) %
                    mod;
            mpow = i64(mpow) * m % mod;
        }
        ret[m] =
            i64(mod - s) * mod_inv(denom, mod) % mod;
    }
    return ret;
}

vector<vector<int>>
find_recurrence_relation(const vector<int> &terms,
                         const int deg, const int mod,
                         bool verify = true) {

    const int n = terms.size();
    const int B =
        (n + 2) / (deg + 2);     // number of blocks
    const int C = B * (deg + 1); // number of columns
    const int R = n - (B - 1);   // number of rows
    assert(B >= 2);
    assert(R >= C - 1);

    auto mul = [mod](int a, int b) {
        return i64(a) * b % mod;
    };
    auto fixed = [mod](int a) {
        return (a %= mod) < 0 ? a + mod : a;
    };
    auto error = [](int order, int deg) {
        fprintf(stderr,
                "Error: Could not find a recurrence "
                "relation "
                "of order <= %d and degree <= %d.\n\n",
                order, deg);
        assert(0);
    };

    vector<vector<int>> mat(R, vector<int>(C));
    for (int y = 0; y < R; ++y) {
        for (int b = 0; b < B; ++b) {
            for (int d = 0, v = fixed(terms[y + b]);
                 d <= deg; ++d) {
                mat[y][b * (deg + 1) + d] = v;
                v = mul(v, y + b);
            }
        }
    }

    int rank = 0;
    for (int x = 0; x < C; ++x) {
        int pivot = -1;
        for (int y = rank; y < R; ++y)
            if (mat[y][x] != 0) {
                pivot = y;
                break;
            }
        if (pivot < 0) break;
        if (pivot != rank) swap(mat[rank], mat[pivot]);
        int inv = mod_inv(mat[rank][x], mod);
        for (int x2 = x; x2 < C; ++x2)
            mat[rank][x2] = mul(mat[rank][x2], inv);
        for (int y = rank + 1; y < R; ++y)
            if (mat[y][x]) {
                int c = mod - mat[y][x];
                for (int x2 = x; x2 < C; ++x2) {
                    mat[y][x2] =
                        (i64(mat[y][x2]) +
                         i64(c) * mat[rank][x2]) %
                        mod;
                }
            }
        ++rank;
    }

    if (rank == C) error(B - 1, deg);

    for (int y = rank - 1; y >= 0; --y)
        if (mat[y][rank]) {
            assert(mat[y][y] == 1);
            int c = mod - mat[y][rank];
            for (int y2 = 0; y2 < y; ++y2) {
                mat[y2][rank] = (mat[y2][rank] +
                                 i64(c) * mat[y2][y]) %
                                mod;
            }
        }

    int order = rank / (deg + 1);
    vector<vector<int>> ret(order + 1,
                            vector<int>(deg + 1));
    ret[0][rank % (deg + 1)] = 1;
    for (int y = rank - 1; y >= 0; --y) {
        int k = order - y / (deg + 1),
            d = y % (deg + 1);
        ret[k][d] = (mod - mat[y][rank]) % mod;
    }

    if (verify) {
        auto extended_terms =
            extended(n - 1, ret,
                     vector<int>(terms.begin(),
                                 terms.begin() + order),
                     mod);
        for (int i = 0; i < (int)terms.size(); ++i) {
            if (fixed(terms[i] - extended_terms[i]) !=
                0)
                error(B - 1, deg);
        }
    }

    return ret;
}

} // namespace p_recursive
```

11.21 合数模线性递推 / 第 k 项（Reeds-Sloane）

功能：在合数模数下由样本猜测常系数递推，再求第 k 项。

约定：namespace composite_rec，LinearRecurrence(samples,mod) 默认合数分支；仅在模数确为素数时可传 true。1<mod<=10^9，样本非空且归一，建议至少 2d 项并留独立验证；get_nth 返回 u64 下标项。试除分解模数，零序列返回 0；拟合不证明未知序列的无限规律。修正负余数更新、前缀瞬态与素数分支前导零；CRT 用 i128。

调 用：Fibonacci 的充分样本、mod=8：`composite_rec::LinearRecurrence r(samples,8,false);; r.get_nth(10)` 为 7。

复杂度：猜测约 $\mathcal{O}(N^2 \log \mathrm{mod})$ 加试除；取项 $\mathcal{O}(d^2 \log k)$

依赖：§1.1（递归收齐依赖）。

koosaga / Not specified upstream; retain attribution / 本地回归 / 对拍 cdf629

```
namespace composite_rec {

using lint = i64;

struct LinearRecurrence {
    using ll = i64;
    using vec = vector<ll>;

    static void extand(vec &a, ll d, ll value = 0) {
        if (d <= a.size()) return;
        a.resize(d, value);
    }

    static vec BerlekampMassey(const vec &s, ll mod) {
        vec c{ 1 }, b{ 1 };
        int len = 0, gap = 1;
        ll old = 1;
```

```
        auto inv = [&](ll x) {
            ll ans = 1;
            for (ll k = mod - 2; k;
                 k >>= 1, x = x * x % mod)
                if (k & 1) ans = ans * x % mod;
            return ans;
        };
        for (int i = 0; i < (int)s.size(); i ++) {
            ll d = s[i];
            for (int j = 1; j < (int)c.size() && j <= i;
                 j ++)
                d = (d + c[j] * s[i - j]) % mod;
            if (!d) {
                gap ++;
                continue;
            }
            vec t = c;
            ll coef = d * inv(old) % mod;
            c.resize(max(c.size(), b.size() + gap));
            for (int j = 0; j < (int)b.size(); j ++)
                c[j + gap] = (c[j + gap] -
                              coef * b[j] % mod + mod) %
                             mod;
            if (2 * len <= i)
                len = i + 1 - len, b = move(t), old = d,
                gap = 1;
            else
                gap ++;
        }
        c.resize(len + 1);
        return c;
    }
    static void exgcd(ll a, ll b, ll &g, ll &x, ll &y) {
        if (!b)
            x = 1, y = 0, g = a;
        else {
            exgcd(b, a % b, g, y, x);
            y -= x * (a / b);
        }
    }
    static ll crt(const vec &c, const vec &m) {
        ll n = c.size();
        ll M = 1, ans = 0;
        for (ll i = 0; i < n; ++i)
            M *= m[i];
        for (ll i = 0; i < n; ++i) {
            ll x, y, g, tm = M / m[i];
            exgcd(tm, m[i], g, x, y);
            ans = (ans + i128(tm) * x * c[i] % M) % M;
        }
        return (ans + M) % M;
    }
    static vec ReedsSloane(const vec &s, ll mod) {
        assert(mod > 1 && mod <= 1000000000LL &&
               !s.empty());
        auto inverse = [](ll a, ll m) {
            ll d, x, y;
            exgcd(a, m, d, x, y);
            return d == 1 ? (x % m + m) % m : -1;
        };
        auto L = [](const vec &a, const vec &b) {
            ll da = (a.size() > 1 ||
                     (a.size() == 1 && a[0]))
                        ? (ll)a.size() - 1
                        : -1000;
            ll db = (b.size() > 1 ||
                     (b.size() == 1 && b[0]))
                        ? (ll)b.size() - 1
                        : -1000;
            return max(da, db + 1);
        };
        auto prime_power = [&](const vec &s, ll mod,
                               ll p, ll e) {
            // linear feedback shift register mod p^e, p
            // is prime
            vector<vec> a(e), b(e), an(e), bn(e), ao(e),
                bo(e);
            vec t(e), u(e), r(e), to(e, 1), uo(e),
                pw(e + 1);
            ;

            pw[0] = 1;
            for (ll i = pw[0] = 1; i <= e; ++i)
                pw[i] = pw[i - 1] * p;
            for (ll i = 0; i < e; ++i) {
                a[i] = { pw[i] }, an[i] = { pw[i] };
                b[i] = { 0 },
                bn[i] = { s[0] * pw[i] % mod };
                t[i] = s[0] * pw[i] % mod;
                if (t[i] == 0) {
                    t[i] = 1, u[i] = e;
                } else {
                    for (u[i] = 0; t[i] % p == 0;
                         t[i] /= p, ++u[i])
                        ;
                }
            }
            for (ll k = 1; k < s.size(); ++k) {
                for (ll g = 0; g < e; ++g) {
                    if (L(an[g], bn[g]) >
                        L(a[g], b[g])) {
                        ao[g] = a[e - 1 - u[g]];
                        bo[g] = b[e - 1 - u[g]];
                        to[g] = t[e - 1 - u[g]];
                        uo[g] = u[e - 1 - u[g]];
                        r[g] = k - 1;
                    }
                }
                a = an, b = bn;
                for (ll o = 0; o < e; ++o) {
                    ll d = 0;
                    for (ll i = 0;
                         i < a[o].size() && i <= k;
                         ++i) {
                        d = (d + a[o][i] * s[k - i]) %
                            mod;
                    }
                    if (d == 0) {
                        t[o] = 1, u[o] = e;
                    } else {
                        for (u[o] = 0, t[o] = d;
                             t[o] % p == 0;
                             t[o] /= p, ++u[o])
                            ;
                        ll g = e - 1 - u[o];
                        if (L(a[g], b[g]) == 0) {
                            extand(bn[o], k + 1);
                            bn[o][k] =
                                (bn[o][k] + d) % mod;
                        } else {
                            ll coef =
                                t[o] *
                                inverse(to[g], mod) %
                                mod * pw[u[o] - uo[g]] %
                                mod;
                            ll m = k - r[g];
                            extand(an[o],
                                   ao[g].size() + m);
                            extand(bn[o],
                                   bo[g].size() + m);
                            for (ll i = 0;
                                 i < ao[g].size();
                                 ++i) {
                                an[o][i + m] -=
                                    coef * ao[g][i] %
                                    mod;
                                if (an[o][i + m] < 0)
                                    an[o][i + m] += mod;
                            }
                            while (an[o].size() &&
                                   an[o].back() == 0)
                                an[o].pop_back();
                            for (ll i = 0;
                                 i < bo[g].size();
                                 ++i) {
```

```
                                bn[o][i + m] -=
                                    coef * bo[g][i] %
                                    mod;
                                if (bn[o][i + m] < 0)
                                    bn[o][i + m] += mod;
                            }
                            while (bn[o].size() &&
                                   bn[o].back() == 0)
                                bn[o].pop_back();
                        }
                    }
                }
            }
            return make_pair(an[0], bn[0]);
        };

        vector<tuple<ll, ll, ll>> fac;
        for (ll i = 2; i * i <= mod; ++i)
            if (mod % i == 0) {
                ll cnt = 0, pw = 1;
                while (mod % i == 0)
                    mod /= i, ++cnt, pw *= i;
                fac.emplace_back(pw, i, cnt);
            }
        if (mod > 1) fac.emplace_back(mod, mod, 1);
        vector<vec> as;
        ll n = 0;
        for (auto &&x : fac) {
            ll mod, p, e;
            vec a, b;
            tie(mod, p, e) = x;
            auto ss = s;
            for (auto &&x : ss)
                x %= mod;
            tie(a, b) = prime_power(ss, mod, p, e);
            as.emplace_back(a);
            n = max(
                n, max((ll)a.size(), (ll)b.size() + 1));
        }
        vec a(n), c(as.size()), m(as.size());

        for (ll i = 0; i < n; ++i) {
            for (ll j = 0; j < as.size(); ++j) {
                m[j] = get<0>(fac[j]);
                c[j] = i < as[j].size() ? as[j][i] : 0;
            }
            a[i] = crt(c, m);
        }
        return a;
    }

    LinearRecurrence(const vec &s, const vec &c, ll mod)
        : init(s), trans(c), mod(mod), m(s.size()) {}

    LinearRecurrence(const vec &s, ll mod,
                     bool is_prime = false)
        : mod(mod) {
        vec A;
        if (is_prime)
            A = BerlekampMassey(s, mod);
        else
            A = ReedsSloane(s, mod);
        if (A.empty()) A = { 0 };
        m = A.size() - 1;

        trans.resize(m);
        for (ll i = 0; i < m; ++i) {
            trans[i] = (mod - A[i + 1]) % mod;
        }
        init = { s.begin(), s.begin() + m };
    }
    int get_nth(unsigned long long n) {
        int m = trans.size();
        if (!m) return 0;
        vector<int> s(m), t(m);
        s[0] = 1;
        if (m != 1)
            t[1] = 1;
        else
            t[0] = trans[0];
        auto mul = [&](vector<int> v, vector<int> w) {
            int m = v.size();
            vector<int> t(2 * m);
            for (int j = 0; j < m; j ++) {
                for (int k = 0; k < m; k ++) {
                    t[j + k] += 1ll * v[j] * w[k] % mod;
                    if (t[j + k] >= mod)
                        t[j + k] -= mod;
                }
            }
            for (int j = 2 * m - 1; j >= m; j --) {
                for (int k = 1; k <= m; k ++) {
                    t[j - k] +=
                        1ll * t[j] * trans[k - 1] % mod;
                    if (t[j - k] >= mod)
                        t[j - k] -= mod;
                }
            }
            t.resize(m);
            return t;
        };
        while (n) {
            if (n & 1) s = mul(s, t);
            t = mul(t, t);
            n >>= 1;
        }
        lint ret = 0;
        for (int i = 0; i < m; i ++)
            ret += 1ll * s[i] * init[i] % mod;
        return ret % mod;
    }
    vec init, trans;
    ll mod;
    ll m;
};

} // namespace composite_rec
```

12 性能工具与卡常技巧

12.1 性能工具选用与边界

先用正确的算法和普通实现跑满规模数据，再定位耗时热点。以下工具改变常数、内存分配或指令选择，不保证在每个编译器和 CPU 上更快。对拍验证正确性，基准测试比较速度；二者不能互相替代。

取模怎么选：编译期常量模数先直接用 %；编译器可能自行转成乘法和移位。运行时给定且大量复用的 32 位模数可用 Barrett32，接受奇数和偶数，也支持模数 1。Barrett32::reduce 的被除数是 u64，不能把两个 u64 的乘积截断后传入；完整 64 位模乘仍用 Modular.h 的 u128 版本。一次模乘就重建 Barrett 对象，会把预处理除法带回热点。

Montgomery32 仅用于 32 位奇模数。普通数 x 转成 xR mod m，R=2^32；先 encode，一串乘加都在域内做，最终 decode。mul 只接受同一模数的域内数；直接乘两个普通整数会算出错误结果。加减也必须保留域内表示；不要每次乘法都来回转换。对象内的 m、neg、r2 不可修改。

FastDiv.h 对应参考笔记的 64 除 32 位 divl 技巧，返回 32 位商与余数。y 必须非零，且 x 的高 32 位小于 y；这等价于商小于 2^32。它仍使用硬件除法，与倒数取模不同；不满足范围时使用普通 /、% 或 Barrett。GNU x86 以外走可移植回退；定义 BL_PORTABLE 可强制关闭本章的汇编和 AVX2 分支。

位运算统一使用无符号数，避免有符号溢出。lsb_index/msb_index 对零返回 -1；底层 ctz/clz 不能直接传零。popcountll 统计 1 的个数。枚举非零子集用 `for(u64 s=m; s; s=(s-1)&m)`，零子集另算。next_mask 枚举相同 popcount 的掩码；限制在 n 位时，n<64 可检查结果小于 `1ULL<<n`，n=64 则以 nullopt 结束，不能写 `1ULL<<64`。k=0 的空集合单独处理，k=64 的初值为 `~0ULL`。

FastIO.h 已在常用补充章：fread/fwrite 缓冲读写整数，支持 EOF 和 i128。对象各含 64 KiB 缓冲，深递归中不要反复创建。批处理 IO 不与 cin/scanf 混用；交互输出要显式 flush。只是输入量小时，关闭同步的 iostream 往往已足够，先测再换。

哈希：先考虑 unordered_map 的 reserve 和 max_load_factor，减少扩容；RandomHash.h 的 SplitMix64 用于混洗整数键。FlatHash 使用连续数组线性探测，预估 expected 为键数而不是桶数。支持全部 u64 键，负整数可显式转 u64。无单项删除；扩容使旧指针引用失效，所以不要在保存一个元素引用后再插入可能触发扩容的新键。clear 保留容量和旧值占用的资源，适合轻量值；有单项删除或稳定引用要求时另选容器。

内存池：Arena 用于主席树、动态线段树等大量小结点；一次给足容量，make 按类型对齐并调用构造。reset 只回收整块空间，不调用析构，所以禁止 string/vector 等非平凡析构类型。旧指针在 reset 后全部失效。超容量抛异常，不能悄悄写出界；特殊的超对齐 SIMD 对象不在支持范围。结点下标加 vector 也是紧凑表示，但 vector 扩容同样会使指针失效。

SIMD 与展开：Simd.h 用数组异或演示 4 路展开、AVX2 非对齐加载和尾部处理。公开入口 xor_words 自动检测 AVX2 支持，不支持则回退；测试同时比较两条路径。AVX2 函数局部使用 target 属性，不对整份程序开 avx2 或 march=native。输入区间允许完全相同，不允许部分重叠。手动展开会增大代码体积，内存带宽受限时 SIMD 也未必提升；更换运算必须重新处理依赖、溢出、对齐和尾部。

编译选项：基准先固定 `-std=c++20 -O2`。再比较 O3、局部展开或向量化，不把 Ofast/fast-math 当作默认开关；它们可能改变浮点语义，几何判断尤其要重新验证。CPU 专用指令仅在目标环境明确支持或具有运行时分派时使用。函数属性和 CPU 检测的官方文档链接见仓库 SOURCES.md。

栈空间：teamnote 的手写 rsp 换栈依赖 ABI、栈对齐、编译器和平台，不作为本库可直接粘贴的模板。深 DFS 优先改显式栈，大数组移到堆或全局；Linux 本地可在允许的硬限制内用 `ulimit -s` 调整栈，Windows MinGW 的链接选项示例为 `-Wl,--stack,268435456`，仅作用于自己链接的程序。评测机若不接受这些设置，不应依赖它们；扩大栈也要计入内存预算。

计时：用 steady_clock，预先生成输入，把随机数、读写和初始化移出被测区间；构造成本另测。累加并输出校验值以防循环被消去，使用相同输入交替运行基线和候选，多轮比较中位数。报告编译器、选项、CPU、数据规模、模数分布和是否包含预处理。正确性测试覆盖边界，性能测试覆盖真实热点；本章的回归测试不宣称加速倍率。

12.2 动态模数快速取模（Barrett）

功能：预处理模数的倒数，用乘法高位和一次修正求余数。
约定：1<=m<2^32；reduce 接受任意 u64，mul 接受任意 uint32_t。模数固定时复用对象；不要每次乘法都构造。编译期常量模数优先直接用% 并实测。
调用：Barrett32 b(998244353); auto r=b.mul(123,456);
复杂度：构造 $O(1)$（一次除法）；reduce、mul $O(1)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 7b31b1

```
struct Barrett32 {
    uint32_t m; // 模数；构造后不要修改。
    u64 inv;    // floor(2^64/m)，m=1 时单独处理。
    explicit Barrett32(uint32_t mod) : m(mod) {
        assert(m);
        inv = m == 1 ? 0 : u64((u128(1) << 64) / m);
    }
    uint32_t reduce(u64 x) const {
        if (m == 1) return 0;
        u64 q = u128(x) * inv >> 64;
        // q 比真实商至多小 1，q*m<=x，故减法不会下溢。
        u64 r = x - q * m;
        return uint32_t(r >= m ? r - m : r);
    }
    uint32_t mul(uint32_t a, uint32_t b) const {
        return reduce(u64(a) * b);
    }
};
```

12.3 奇模数连续模乘（Montgomery）

功能：在 Montgomery 域中连续乘法，避免每次乘法执行整数除法。
约定：m 为奇数且 1<=m<2^32；R=2^32，域内表示为 xR mod m。encode 将普通数转入，decode 转回；mul/add/sub 的参数必须是同一对象的域内数且小于 m。不可混用不同模数；域内的 1 是 encode(1)。reduce 为内部过程，要求 t<mR。
调 用：Montgomery32 z(998244353); auto a=z.encode(123); auto r=z.decode(z.mul(a,a));
复杂度：构造、转换、单次运算 $O(1)$；pow $O(\log e)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 ee7ebf

```
struct Montgomery32 {
    // m; -m^(-1) mod R; R^2 mod m。
    uint32_t m, neg, r2;
    explicit Montgomery32(uint32_t mod) : m(mod) {
        assert(m & 1);
        uint32_t x = 1;
        for (int i = 0; i < 5; i ++)
            x *= 2 - m * x;
        neg = 0u - x;
        r2 = (u128(1) << 64) % m;
    }
    uint32_t reduce(u64 t) const {
        uint32_t k = uint32_t(t) * neg;
        // t+k*m 的低 32 位为 0；分开加高位，避免 64
        // 位溢出。
        u64 v = (t >> 32) + (u64(k) * m >> 32) +
                (uint32_t(t) != 0);
        return uint32_t(v >= m ? v - m : v);
    }
    uint32_t encode(uint32_t a) const {
        return reduce(u64(a % m) * r2);
    }
    uint32_t decode(uint32_t a) const {
        return reduce(a);
    }
    uint32_t mul(uint32_t a, uint32_t b) const {
        return reduce(u64(a) * b);
    }
    uint32_t add(uint32_t a, uint32_t b) const {
        u64 s = u64(a) + b;
        return uint32_t(s >= m ? s - m : s);
    }
    uint32_t sub(uint32_t a, uint32_t b) const {
        return a >= b ? a - b
                      : uint32_t(u64(a) + m - b);
    }
    // 对外接收普通数 a，返回普通数 a^e mod m。
    uint32_t pow(uint32_t a, u64 e) const {
        uint32_t x = encode(a), r = encode(1);
        for (; e; e >>= 1, x = mul(x, x))
            if (e & 1) r = mul(r, x);
        return decode(r);
    }
};
```

12.4 64 除 32 位商余数（x86 divl）

功能：同时返回无符号除法的商和余数，x86 使用单条 divl。
约定：y>0 且 x<y*2^32，保证商能装入 uint32_t；违反前提可能触发硬件异常。GNU x86 使用汇编，其余架构使用 / 和%。这不是 Barrett；速度需实测。
调用：auto [q,r]=divmod32(10000000000ULL,7);
复杂度：$O(1)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 c31d85

```
inline pair<uint32_t, uint32_t> divmod32(u64 x,
                                         uint32_t y) {
    assert(y && (x >> 32) < y);
#if !defined(BL_PORTABLE) && defined(__GNUC__) &&      \
    (defined(__i386__) || defined(__x86_64__))
    uint32_t q = uint32_t(x), r = x >> 32;
    asm("divl %2" : "+a"(q), "+d"(r) : "rm"(y) : "cc");
```

```
    return { q, r };
#else
    return { uint32_t(x / y), uint32_t(x % y) };
#endif
}
```

12.5 位扫描与等大小子集枚举

功能：提供零值安全的最高 / 最低位查询、lowbit 和下一同位数掩码。
约定：位编号 0..63；lsb_index、msb_index 对 0 返回 -1。next_mask 按数值递增，保持 1 的个数；0 或已是 64 位中最后一个时返回 nullopt。均用无符号数。
调用：auto s=next_mask(7); // 得 11，即 00111 的下一个是 01011。
复杂度：每次操作 $\mathcal{O}(1)$
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 3a5348

```
inline int lsb_index(u64 x) {
    return x ? __builtin_ctzll(x) : -1;
}
inline int msb_index(u64 x) {
    return x ? 63 - __builtin_clzll(x) : -1;
}
inline u64 lowbit(u64 x) {
    return x & (0 - x);
}
inline optional<u64> next_mask(u64 x) {
    if (!x) return nullopt;
    u64 a = lowbit(x), b = x + a;
    if (!b) return nullopt;
    // 分两次右移，避免 ctz(a)+2=64 时出现未定义移位。
    return b | (((b ^ x) >> 2) >> lsb_index(a));
}
```

12.6 整数键开放寻址哈希表

功能：用连续数组、线性探测和随机混洗存储 u64 到 T 的映射。
约定：支持 find、下标插入和 clear，不支持单项删除。所有 u64 键均合法，无哨兵键。容量为 2 的幂，装载率不超过 1/2；扩容会使指针和引用失效，clear 后也不可使用旧引用。T 需可默认构造、复制赋值；不存在的键由 operator[] 插入 T{}。T 不可为 bool，可用 unsigned char 代替。非抗恶意自适应哈希。
调用：FlatHash<int> h(1000); h[42]++; if(auto p=h.find(42)) cout«*p;
复杂度：查找期望 $\mathcal{O}(1)$，插入期望均摊 $\mathcal{O}(1)$；最坏 $\mathcal{O}$(容量)，clear $\mathcal{O}$(容量)
依赖：§1.1, §1.4（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 8ee8ac

```
template <class T> struct FlatHash {
    static_assert(!is_same_v<T, bool>);
    vector<u64> key;
    vector<T> val;
    vector<unsigned char> used;
    // 当前键数；used 区分空位，0 也可作为键。
    size_t cnt = 0;
    // 每个对象固定盐值，已有元素时不可修改。
    u64 seed = rng();
    explicit FlatHash(size_t expected = 0) {
        size_t c = 8;
        while (c / 2 < expected) {
            if (c > numeric_limits<size_t>::max() / 2)
                throw length_error("FlatHash capacity");
            c *= 2;
        }
        key.resize(c), val.resize(c), used.resize(c);
    }
    // 内部：找到 x 的位置或探测链上的第一个空位。
    size_t slot(u64 x) const {
        size_t mask = key.size() - 1;
        size_t p = Hash::mix(x + seed) & mask;
        while (used[p] && key[p] != x)
            p = (p + 1) & mask;
        return p;
    }
    T *find(u64 x) {
        size_t p = slot(x);
        return used[p] ? &val[p] : nullptr;
    }
    const T *find(u64 x) const {
        size_t p = slot(x);
        return used[p] ? &val[p] : nullptr;
    }
    // 内部：容量翻倍后重新散列，原有引用全部失效。
    void grow() {
        FlatHash h(key.size());
        h.seed = seed;
        for (size_t i = 0; i < key.size(); i ++)
            if (used[i]) h[key[i]] = val[i];
        *this = move(h);
    }
    T &operator[](u64 x) {
        size_t p = slot(x);
        if (used[p]) return val[p];
        if (cnt == key.size() / 2) grow(), p = slot(x);
        used[p] = 1, key[p] = x, val[p] = T{};
        cnt ++;
        return val[p];
    }
    size_t size() const { return cnt; }
    void clear() {
        fill(used.begin(), used.end(), 0);
        // 保留容量；T 持有的资源不会在这里全部释放。
        cnt = 0;
    }
};
```

12.7 固定容量内存池（Arena）

功能：一次分配堆内存，按对齐要求顺序创建轻量结点，整池重置。
约定：不扩容、不单独释放；reset 使所有旧指针失效。仅支持可平凡析构的 T，alignof(T)<=alignof(max_align_t)；容量不足抛 bad_alloc。对象不可复制或移动。
调用：Arena a(1«20); auto p=a.make<pair<int,int»(3,4); a.reset();
复杂度：分配 $\mathcal{O}(1)$ 加构造开销；reset $\mathcal{O}(1)$；空间为预设容量
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 3c31e2

```
struct Arena {
    void *buf;
    size_t cap,
        pos = 0; // 字节容量、已消耗字节（含对齐填充）。
    explicit Arena(size_t bytes)
        : buf(::operator new(bytes)), cap(bytes) {}
    ~Arena() { ::operator delete(buf); }
    Arena(const Arena &) = delete;
    Arena &operator=(const Arena &) = delete;
    void reset() { pos = 0; }
    template <class T, class... Args>
    T *make(Args &&...args) {
        static_assert(is_trivially_destructible_v<T>);
        static_assert(alignof(T) <=
                      alignof(max_align_t));
        void *p = static_cast<char *>(buf) + pos;
        size_t space = cap - pos;
        if (!align(alignof(T), sizeof(T), p, space))
            throw bad_alloc();
        T *out = ::new (p) T(forward<Args>(args)...);
        pos = cap - space + sizeof(T);
        return out;
    }
};
```

12.8 数组异或（展开循环 / AVX2）

功能：对 n 个 u64 执行 a[i]^=b[i]；自动选择 AVX2 或普通展开循环。
约定：两段内存须不相交或完全相同；允许非 32 字节对齐，尾部逐项处理。仅 GNU x86 编译 AVX2 分支；xor_words 检测 CPU 后调用，不要直接调用 avx2 内部函数。编译整个程序时不要加 -mavx2/-march=native，否则回退路径也可能含专用指令。
调用：xor_words(a.data(),b.data(),a.size());
复杂度：$\mathcal{O}(n)$，每次 AVX2 操作处理 4 个 u64；不保证小数组加速
依赖：§1.1（递归收齐依赖）。

BlueLagoon / CC0-1.0 / 本地回归 / 对拍 380702

```
#if !defined(BL_PORTABLE) && defined(__GNUC__) &&      \
    (defined(__i386__) || defined(__x86_64__))
#include <immintrin.h>
#endif

inline void xor_words_scalar(u64 *a, const u64 *b,
                             size_t n) {
    size_t i = 0;
    for (; n - i >= 4; i += 4) {
        a[i] ^= b[i];
        a[i + 1] ^= b[i + 1];
        a[i + 2] ^= b[i + 2];
        a[i + 3] ^= b[i + 3];
    }
    for (; i < n; i ++)
        a[i] ^= b[i];
}
#if !defined(BL_PORTABLE) && defined(__GNUC__) &&      \
    (defined(__i386__) || defined(__x86_64__))
__attribute__((target("avx2"))) inline void
xor_words_avx2(u64 *a, const u64 *b, size_t n) {
    size_t i = 0;
    for (; n - i >= 4; i += 4) {
        auto x = _mm256_loadu_si256(
            reinterpret_cast<const __m256i *>(a + i));
        auto y = _mm256_loadu_si256(
            reinterpret_cast<const __m256i *>(b + i));
        _mm256_storeu_si256(
            reinterpret_cast<__m256i *>(a + i),
            _mm256_xor_si256(x, y));
    }
    for (; i < n; i ++)
        a[i] ^= b[i];
}
#endif
inline void xor_words(u64 *a, const u64 *b, size_t n) {
#if !defined(BL_PORTABLE) && defined(__GNUC__) &&      \
    (defined(__i386__) || defined(__x86_64__))
    if (__builtin_cpu_supports("avx2")) {
        xor_words_avx2(a, b, n);
        return;
    }
#endif
    xor_words_scalar(a, b, n);
}
```